

You said:

Conversación wireframes.docx

Document

Estoy desarrollando una aplicacion con spring boot y java 25. Quiero que me generes la parte de front (html, css) teniendo en cuenta los wireframes adjuntados

ChatGPT said:

Thought for 7s

Perfecto — con los wireframes del documento (login SSO, catálogo, detalle/ejemplares, reserva, “mis préstamos”, y pantallas de bibliotecario) te dejo una propuesta de **front en HTML + CSS** lista para meter en **Spring Boot** (idealmente con **Thymeleaf**).

Conversación wireframes

A continuación van:

- Estructura de carpetas recomendada
- CSS común (responsive)
- Plantillas HTML por pantalla (con clases consistentes)
- Un layout simple reutilizable (si usas Thymeleaf)

1) Estructura de proyecto (Spring Boot + templates)

```
src/main/resources/  
  static/  
    css/  
      app.css  
    img/  
      logo.svg    (opcional)  
  templates/  
    fragments/  
      layout.html  
    login.html  
    menu.html  
    catalog.html  
    book-detail.html  
    reserve.html  
    my-area.html  
    librarian-loan.html  
    librarian-return.html  
    librarian-copies.html
```

librarian-professor-delivery.html
librarian-config.html

2) CSS común (app.css)

```
/* src/main/resources/static/css/app.css */

:root{
  --bg: #0b1220;
  --card: #0f1a2f;
  --card-2:#122242;
  --text:#eaf0ff;
  --muted:#b7c3e6;
  --border: rgba(255,255,255,.12);
  --accent:#7aa2ff;
  --accent-2:#34d399;
  --danger:#fb7185;
  --warning:#fbbf24;

  --radius: 14px;
  --shadow: 0 18px 45px rgba(0,0,0,.35);

  --container: 1080px;
  --gap: 16px;
  --pad: 18px;

  --font: ui-sans-serif, system-ui, -apple-system, Segoe UI, Roboto, Helvetica,
  Arial, "Apple Color Emoji","Segoe UI Emoji";
}

*{ box-sizing:border-box; }
html,body{ height:100%; }
body{
  margin:0;
  font-family: var(--font);
  color: var(--text);
  background: radial-gradient(1200px 700px at 10% -10%, rgba(122,162,255,.35),
  transparent 50%),
              radial-gradient(1000px 600px at 90% 0%, rgba(52,211,153,.18),
  transparent 55%),
              var(--bg);
}
```

```
}

a{ color:inherit; text-decoration:none; }
img{ max-width:100%; display:block; }

.container{
  max-width: var(--container);
  margin: 0 auto;
  padding: 22px 16px;
}

.topbar{
  position: sticky;
  top:0;
  z-index: 50;
  backdrop-filter: blur(10px);
  background: rgba(11,18,32,.55);
  border-bottom: 1px solid var(--border);
}

.topbar-inner{
  max-width: var(--container);
  margin:0 auto;
  padding: 14px 16px;
  display:flex;
  align-items:center;
  justify-content:space-between;
  gap: 12px;
}

.brand{
  display:flex;
  align-items:center;
  gap: 10px;
  font-weight: 800;
  letter-spacing:.3px;
}

.brand-badge{
  width: 34px;
  height: 34px;
  border-radius: 10px;
  background: linear-gradient(135deg, rgba(122,162,255,.9), rgba(52,211,153,.7));
  box-shadow: 0 10px 22px rgba(0,0,0,.35);
}
```

```
}
.top-actions{
  display:flex;
  align-items:center;
  gap: 10px;
}

.card{
  background: linear-gradient(180deg, rgba(255,255,255,.06),
  rgba(255,255,255,.03));
  border: 1px solid var(--border);
  border-radius: var(--radius);
  box-shadow: var(--shadow);
}
.card.pad{ padding: var(--pad); }

.grid{
  display:grid;
  gap: var(--gap);
}
.grid.two{ grid-template-columns: 1fr 1fr; }
.grid.three{ grid-template-columns: repeat(3, 1fr); }
@media (max-width: 860px){
  .grid.two, .grid.three{ grid-template-columns: 1fr; }
}

.h1{ font-size: 28px; margin: 0 0 6px; }
.h2{ font-size: 18px; margin: 0 0 10px; color: var(--text); }
.p{ margin: 0; color: var(--muted); line-height: 1.5; }

.divider{
  height:1px;
  background: var(--border);
  margin: 14px 0;
}

.field{
  display:grid;
  gap: 8px;
}
.label{
```

```
    font-size: 13px;
    color: var(--muted);
}
.input, .select{
    width:100%;
    padding: 12px 12px;
    border-radius: 12px;
    border: 1px solid var(--border);
    background: rgba(15,26,47,.65);
    color: var(--text);
    outline: none;
}
.input::placeholder{ color: rgba(183,195,230,.65); }
.input:focus, .select:focus{
    border-color: rgba(122,162,255,.65);
    box-shadow: 0 0 0 4px rgba(122,162,255,.16);
}

.row{
    display:flex;
    gap: 10px;
    align-items:center;
    flex-wrap:wrap;
}
.spread{ justify-content:space-between; }

.btn{
    display:inline-flex;
    align-items:center;
    justify-content:center;
    gap: 8px;
    padding: 11px 14px;
    border-radius: 12px;
    border: 1px solid var(--border);
    background: rgba(255,255,255,.06);
    color: var(--text);
    cursor:pointer;
    font-weight: 650;
}
.btn:hover{ background: rgba(255,255,255,.10); }
.btn.primary{
```

```

border-color: rgba(122,162,255,.55);
background: linear-gradient(135deg, rgba(122,162,255,.85),
rgba(52,211,153,.55));
}
.btn.danger{
border-color: rgba(251,113,133,.55);
background: rgba(251,113,133,.12);
}
.btn.ghost{
background: transparent;
}
.chip{
font-size: 12px;
padding: 6px 10px;
border-radius: 999px;
border: 1px solid var(--border);
background: rgba(255,255,255,.05);
color: var(--muted);
}
.chip.ok{ color: #a7f3d0; border-color: rgba(52,211,153,.35); }
.chip.warn{ color: #fde68a; border-color: rgba(251,191,36,.35); }
.chip.bad{ color: #fecdd3; border-color: rgba(251,113,133,.35); }

.list{
display:grid;
gap: 12px;
}
.item{
display:grid;
grid-template-columns: 66px 1fr auto;
gap: 12px;
align-items:center;
padding: 12px;
border-radius: 14px;
border: 1px solid var(--border);
background: rgba(15,26,47,.45);
}
@media (max-width: 620px){
.item{ grid-template-columns: 56px 1fr; }
.item .right{ grid-column: 1 / -1; justify-content:flex-start; }
}
.thumb{

```

```

width: 66px;
height: 66px;
border-radius: 14px;
background: rgba(122,162,255,.15);
border: 1px solid var(--border);
}
.meta .title{
margin:0;
font-weight: 800;
}
.meta .sub{
margin: 2px 0 0;
color: var(--muted);
font-size: 14px;
}
.right{
display:flex;
gap: 10px;
align-items:center;
justify-content:flex-end;
flex-wrap:wrap;
}

.footer{
color: rgba(183,195,230,.65);
font-size: 13px;
text-align:center;
padding: 18px 10px 30px;
}

```

3) Fragmento Layout (Thymeleaf) — layout.html

```

<!-- src/main/resources/templates/fragments/layout.html -->
<!doctype html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="utf-8"/>
  <meta name="viewport" content="width=device-width,initial-scale=1"/>
  <title th:text="${title} ?: 'Biblioteca'">Biblioteca</title>
  <link rel="stylesheet" th:href="@{/css/app.css}">
</head>
<body>

```

```

<header class="topbar" th:fragment="topbar(userName, role)">
  <div class="topbar-inner">
    <a class="brand" th:href="@{/menu}">
      <span class="brand-badge" aria-hidden="true"></span>
      <span>BIBLIOTECA UXX</span>
    </a>
    <div class="top-actions">
      <span class="chip" th:text="${userName} ?: 'Usuario'">Usuario</span>
      <span class="chip" th:text="${role} ?: 'Rol'">Rol</span>
      <a class="btn ghost" th:href="@{/logout}">Cerrar sesión</a>
    </div>
  </div>
</header>

<main class="container" th:fragment="content">
  <!-- content -->
</main>

<footer class="footer" th:fragment="footer">
  © Universidad X - 2025
</footer>
</body>
</html>

```

4) Pantallas

4.1 Login SSO — login.html (CU01)

```

<!-- src/main/resources/templates/Login.html -->
<!doctype html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="utf-8"/>
  <meta name="viewport" content="width=device-width,initial-scale=1"/>
  <title>Login - Biblioteca</title>
  <link rel="stylesheet" th:href="@{/css/app.css}">
</head>
<body>
  <main class="container">
    <section class="card pad" style="max-width:560px;margin:10vh auto 0;">
      <div class="row" style="justify-content:center;margin-bottom:10px;">
        <span class="brand-badge" aria-hidden="true"></span>

```



```

    </div>
    <h1 class="h1" style="text-align:center;">BIBLIOTECA UXX</h1>
    <p class="p" style="text-align:center;">Accede con tu SSO institucional</p>
    <div class="divider"></div>

    <a class="btn primary" th:href="@{/oauth2/authorization/sso}"
style="width:100%;">
        Iniciar sesión con SSO institucional
    </a>

    <div class="divider"></div>
    <p class="p" style="text-align:center;font-size:13px;">
        Si tienes problemas de acceso, contacta con soporte.
    </p>
</section>

<footer class="footer">© Universidad X - 2025</footer>
</main>
</body>
</html>

```

4.2 Menú principal — menu.html

```

<!-- src/main/resources/templates/menu.html -->
<!doctype html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="utf-8"/>
    <meta name="viewport" content="width=device-width,initial-scale=1"/>
    <title>Menú - Biblioteca</title>
    <link rel="stylesheet" th:href="@{/css/app.css}">
</head>
<body>
    <header th:replace="fragments/layout :: topbar(${userName}, ${role})"></header>

    <main class="container">
        <section class="card pad">
            <h1 class="h1">Menú principal</h1>
            <p class="p">Selecciona una opción</p>
            <div class="divider"></div>

```

```

<div class="grid two">
  <a class="card pad" th:href="@{/catalog}">
    <h2 class="h2">Buscar en catálogo</h2>
    <p class="p">Localiza obras y consulta ejemplares</p>
  </a>

  <a class="card pad" th:href="@{/my}">
    <h2 class="h2">Mis préstamos y reservas</h2>
    <p class="p">Renovaciones, reservas activas y sanciones</p>
  </a>
</div>

<div class="divider"></div>

<!-- Mostrar solo si es bibliotecario -->
<div th:if="${isLibrarian}">
  <h2 class="h2">Opciones bibliotecario</h2>
  <div class="grid three">
    <a class="card pad" th:href="@{/librarian/loan}">
      <h2 class="h2">Registrar préstamo</h2>
      <p class="p">Buscar usuario y escanear ejemplar</p>
    </a>
    <a class="card pad" th:href="@{/librarian/return}">
      <h2 class="h2">Registrar devolución</h2>
      <p class="p">Escanear y calcular sanción</p>
    </a>
    <a class="card pad" th:href="@{/librarian/copies}">
      <h2 class="h2">Gestionar ejemplares</h2>
      <p class="p">Altas, edición y bajas</p>
    </a>
    <a class="card pad" th:href="@{/librarian/professor-delivery}">
      <h2 class="h2">Envío a despacho</h2>
      <p class="p">Reservas de profesores listas</p>
    </a>
    <a class="card pad" th:href="@{/librarian/config}">
      <h2 class="h2">Configuración</h2>
      <p class="p">Parámetros del sistema</p>
    </a>
  </div>
</div>
</section>

```

```

        <footer th:replace="fragments/layout :: footer"></footer>
    </main>
</body>
</html>

```

4.3 Buscar en catálogo — catalog.html (CU02)

```

<!-- src/main/resources/templates/catalog.html -->
<!doctype html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="utf-8"/>
    <meta name="viewport" content="width=device-width,initial-scale=1"/>
    <title>Catálogo - Biblioteca</title>
    <link rel="stylesheet" th:href="@{/css/app.css}">
</head>
<body>
    <header th:replace="fragments/layout :: topbar(${userName}, ${role})"></header>

    <main class="container">
        <section class="card pad">
            <div class="row spread">
                <div>
                    <h1 class="h1">Catálogo</h1>
                    <p class="p">Busca por título, autor o materia</p>
                </div>
                <a class="btn" th:href="@{/menu}">← Volver</a>
            </div>

            <div class="divider"></div>

            <form class="row" th:action="@{/catalog}" method="get" style="gap:12px;">
                <input class="input" name="q" placeholder="Buscar..." th:value="${q}"
style="flex:1;min-width:220px;">
                <button class="btn primary" type="submit">Buscar</button>
            </form>

            <div class="divider"></div>

            <div class="list">

```

```

<!-- Repetir por resultados -->
<article class="item" th:each="book : ${books}">
  <div class="thumb" aria-hidden="true"></div>
  <div class="meta">
    <p class="title" th:text="${book.title}">Título de la obra</p>
    <p class="sub" th:text="${book.author}">Autor</p>
  </div>
  <div class="right">
    <a class="btn" th:href="@{/books/${book.id}}">Ver ejemplares →</a>
  </div>
</article>

<!-- Estado vacío -->
<div class="card pad" th:if="${#lists.isEmpty(books)}">
  <p class="p">No hay resultados para tu búsqueda.</p>
</div>
</div>
</section>

<footer th:replace="fragments/layout :: footer"></footer>
</main>
</body>
</html>

```

4.4 Detalle de obra + lista de ejemplares — book-detail.html (CU03)

```

<!-- src/main/resources/templates/book-detail.html -->
<!doctype html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="utf-8"/>
  <meta name="viewport" content="width=device-width,initial-scale=1"/>
  <title>Detalle - Biblioteca</title>
  <link rel="stylesheet" th:href="@{/css/app.css}">
</head>
<body>
  <header th:replace="fragments/layout :: topbar(${userName}, ${role})"></header>

  <main class="container">
    <section class="card pad">
      <div class="row spread">
        <div>

```

```

<h1 class="h1" th:text="${book.title}">Título del libro</h1>
<p class="p">
  <span th:text="'Autor: ' + ${book.author}">Autor: X</span>
  .
  <span th:text="'Materia: ' + ${book.subject}">Materia: Y</span>
</p>
</div>
<a class="btn" th:href="@{/catalog}">← Volver</a>
</div>

<div class="divider"></div>

<h2 class="h2">Ejemplares</h2>

<div class="list">
  <article class="card pad" th:each="copy : ${copies}">
    <div class="row spread">
      <div>
        <div class="row" style="gap:8px;">
          <strong th:text="'Ejemplar ID: ' + ${copy.id}">Ejemplar ID:
12345</strong>
          <span class="chip" th:text="${copy.location}">Estante A3</span>
        </div>
        <p class="p" th:text="'¿Referencia?: ' + (${copy.reference} ?
'Sí' : 'No')">¿Referencia?: No</p>
        <p class="p" th:text="'Estado: ' + ${copy.status}">Estado:
DISPONIBLE</p>

        <p class="p" th:if="${copy.status == 'PRESTADO'}"
          th:text="'Dev. prevista: ' + ${copy.dueDate} + ' . Cola de
reservas: ' + ${copy.queue}">
          Dev. prevista: 15/03/2025 . Cola de reservas: 2
        </p>
      </div>

      <div class="row">
        <span class="chip ok" th:if="${copy.status ==
'DISPONIBLE'}">Disponible</span>
        <span class="chip warn" th:if="${copy.status ==
'PRESTADO'}">Prestado</span>

```

```

        <a class="btn primary"
          th:href="@{|/reserve/{copy.id}|}">
          Reservar ejemplar
        </a>
      </div>
    </div>
  </article>
</div>
</section>

<footer th:replace="fragments/layout :: footer"></footer>
</main>
</body>
</html>

```

4.5 Reservar ejemplar — reserve.html (CU04)

```

<!-- src/main/resources/templates/reserve.html -->
<!doctype html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="utf-8"/>
  <meta name="viewport" content="width=device-width,initial-scale=1"/>
  <title>Reservar - Biblioteca</title>
  <link rel="stylesheet" th:href="@{/css/app.css}">
</head>
<body>
  <header th:replace="fragments/layout :: topbar(${userName}, ${role})"></header>

  <main class="container">
    <section class="card pad" style="max-width:760px;">
      <div class="row spread">
        <div>
          <h1 class="h1">Reservar ejemplar</h1>
          <p class="p" th:text="'Libro: ' + ${bookTitle}">Libro: Título
ejemplo</p>
          <p class="p" th:text="'Ejemplar: ' + ${copyId}">Ejemplar: 12345</p>
          <p class="p" th:text="'Estado: ' + ${status}">Estado: Disponible</p>
        </div>
        <a class="btn" th:href="@{|/books/{bookId}|}">← Volver</a>
      </div>
    </section>
  </main>
</body>
</html>

```

```

<div class="divider"></div>

<form th:action="@{/reserve/confirm}" method="post" class="row spread"
style="align-items:flex-end;">
    <input type="hidden" name="copyId" th:value="${copyId}">
    <button class="btn primary" type="submit">Confirmar reserva</button>
    <span class="chip" th:if="${isProfessor}">Profesores: envío a
despacho</span>
</form>

<div class="divider"></div>
<p class="p">Nota: Los profesores tienen envío a despacho.</p>
</section>

<footer th:replace="fragments/layout :: footer"></footer>
</main>
</body>
</html>

```

4.6 Mis préstamos, reservas y sanciones — my-area.html (CU06)

```

<!-- src/main/resources/templates/my-area.html -->
<!doctype html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="utf-8"/>
    <meta name="viewport" content="width=device-width,initial-scale=1"/>
    <title>Mi área - Biblioteca</title>
    <link rel="stylesheet" th:href="@{/css/app.css}">
</head>
<body>
    <header th:replace="fragments/layout :: topbar(${userName}, ${role})"></header>

    <main class="container">
        <section class="card pad">
            <div class="row spread">
                <div>
                    <h1 class="h1">Mis préstamos y reservas</h1>
                    <p class="p">Gestiona tu actividad</p>
                </div>
                <a class="btn" th:href="@{/menu}">← Volver</a>
            </div>

```

```

<div class="divider"></div>

<h2 class="h2">PRÉSTAMOS ACTIVOS</h2>
<div class="list">
  <article class="item" th:each="loan : ${loans}">
    <div class="thumb" aria-hidden="true"></div>
    <div class="meta">
      <p class="title" th:text="${loan.title}">Título: X</p>
      <p class="sub" th:text="'Ejemplar: ' + ${loan.copyId} + ' · Dev.
prevista: ' + ${loan.dueDate}'">
        Ejemplar: 12345 · Dev. prevista: 20/03/2025
      </p>
      <p class="sub" th:if="${loan.reference}">(Ejemplar referencia → NO
renovable)</p>
    </div>
    <div class="right">
      <button class="btn primary" th:disabled="${loan.reference}"
type="button">Renovar</button>
    </div>
  </article>
  <div class="card pad" th:if="${#lists.isEmpty(loans)}">
    <p class="p">No tienes préstamos activos.</p>
  </div>
</div>

<div class="divider"></div>

<h2 class="h2">RESERVAS ACTIVAS</h2>
<div class="list">
  <article class="item" th:each="res : ${reservations}">
    <div class="thumb" aria-hidden="true"></div>
    <div class="meta">
      <p class="title" th:text="${res.title}">Título: Z</p>
      <p class="sub" th:text="'Estado: ' + ${res.status} + ' · Fecha
límite: ' + ${res.deadline}'">
        Estado: Disponible para recoger · Fecha límite: 12/03/2025
      </p>
    </div>
    <div class="right">
      <button class="btn danger" type="button">Cancelar reserva</button>
    </div>
  </article>
</div>

```



```

        </div>
    </article>
    <div class="card pad" th:if="${#lists.isEmpty(reservations)}">
        <p class="p">No tienes reservas activas.</p>
    </div>
</div>

<div class="divider"></div>

<h2 class="h2">SANCIONES</h2>
<div class="card pad">
    <p class="p" th:text="${sanctionsText}">Ninguna sanción activa</p>
</div>
</section>

<footer th:replace="fragments/layout :: footer"></footer>
</main>
</body>
</html>

```

5) Bibliotecario

5.1 Registrar préstamo — librarian-loan.html (CU07)

```

<!-- src/main/resources/templates/Librarian-Loan.html -->
<!doctype html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="utf-8"/>
    <meta name="viewport" content="width=device-width,initial-scale=1"/>
    <title>Registrar préstamo</title>
    <link rel="stylesheet" th:href="@{/css/app.css}">
</head>
<body>
    <header th:replace="fragments/layout :: topbar(${userName}, ${role})"></header>

    <main class="container">
        <section class="card pad">
            <div class="row spread">
                <div>
                    <h1 class="h1">Bibliotecario → Registrar préstamo</h1>

```

```

    <p class="p">1) Buscar usuario · 2) Escanear ejemplar</p>
  </div>
  <a class="btn" th:href="@{/menu}">← Volver</a>
</div>

<div class="divider"></div>

<div class="grid two">
  <div class="card pad">
    <h2 class="h2">1. Buscar usuario</h2>
    <form class="row" method="get" th:action="@{/librarian/loan}">
      <input class="input" name="userQ" placeholder="Nombre/ID..."
th:value="${userQ}" style="flex:1;min-width:220px;">
      <button class="btn primary" type="submit">Buscar</button>
    </form>

    <div class="divider"></div>
    <div th:if="${selectedUser != null}">
      <p class="p">
        <strong th:text="${selectedUser.name}">María López</strong>
        <span class="chip" th:text="${selectedUser.type}">Estudiante</span>
      </p>
    </div>
    <p class="p" th:if="${selectedUser == null}">Sin usuario
seleccionado.</p>
  </div>

  <div class="card pad">
    <h2 class="h2">2. Escanear ejemplar</h2>
    <form method="post" th:action="@{/librarian/loan/scan}" class="field">
      <label class="label">Código de barras / ID ejemplar</label>
      <input class="input" name="copyCode" placeholder="Escanear..." />
      <button class="btn primary" type="submit">Continuar</button>
    </form>

    <div class="divider"></div>

    <div th:if="${copy != null}">
      <p class="p" th:text="'Ejemplar: ' + ${copy.id}">Ejemplar: 12345</p>
      <p class="p" th:text="'Estado: ' + ${copy.status}">Estado:
Disponible</p>

```

```

        <p class="p" th:text="'Reserva para este usuario: ' +
({copy.reservedForUser} ? 'Sí' : 'No')">
            Reserva para este usuario: Sí/No
        </p>
        <p class="p" th:text="'¿Referencia?: ' + ({copy.reference} ? 'Sí' :
'No')}">¿Referencia?: No</p>

        <div class="divider"></div>
        <form method="post" th:action="@{/librarian/loan/confirm}">
            <button class="btn primary" type="submit">Confirmar
préstamo</button>
        </form>
    </div>

    <p class="p" th:if="${copy == null}">Escanea un ejemplar para ver
detalles.</p>
</div>
</div>
</section>

<footer th:replace="fragments/layout :: footer"></footer>
</main>
</body>
</html>

```

5.2 Registrar devolución — librarian-return.html (CU08)

```

<!-- src/main/resources/templates/librarian-return.html -->
<!doctype html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="utf-8"/>
    <meta name="viewport" content="width=device-width,initial-scale=1"/>
    <title>Registrar devolución</title>
    <link rel="stylesheet" th:href="@{/css/app.css}">
</head>
<body>
    <header th:replace="fragments/layout :: topbar(${userName}, ${role})"></header>

    <main class="container">
        <section class="card pad">
            <div class="row spread">

```

```

    <div>
        <h1 class="h1">Bibliotecario → Registrar devolución</h1>
        <p class="p">Escanea el ejemplar y confirma</p>
    </div>
    <a class="btn" th:href="@{/menu}">← Volver</a>
</div>

<div class="divider"></div>

<form class="row" th:action="@{/librarian/return/scan}" method="post"
style="gap:12px;">
    <input class="input" name="copyCode" placeholder="Código..."
style="flex:1;min-width:220px;">
    <button class="btn primary" type="submit">Buscar</button>
</form>

<div class="divider"></div>

<div class="card pad" th:if="${returnInfo != null}">
    <p class="p"><strong th:text="'Ejemplar ' +
${returnInfo.copyId}">Ejemplar 12345</strong></p>
    <p class="p" th:text="'Usuario: ' + ${returnInfo.userName}">Usuario: Juan
Pérez</p>
    <p class="p" th:text="'Fecha límite: ' + ${returnInfo.dueDate}">Fecha
límite: 15/03/2025</p>
    <p class="p" th:text="'Devuelto: ' + ${returnInfo.returnDate}">Devuelto:
17/03/2025</p>
    <p class="p" th:text="'Retraso: ' + ${returnInfo.delayDays} + '
días'">Retraso: 2 días</p>
    <p class="p" th:text="'Sanción generada: ' + ${returnInfo.penaltyDays} +
' días sin reservar'">
        Sanción generada: 4 días sin reservar
    </p>

    <div class="divider"></div>
    <form th:action="@{/librarian/return/confirm}" method="post">
        <button class="btn primary" type="submit">Confirmar devolución</button>
    </form>
</div>

<div class="card pad" th:if="${returnInfo == null}">

```

```

        <p class="p">Escanea un ejemplar para ver la información de
devolución.</p>
    </div>
</section>

    <footer th:replace="fragments/layout :: footer"></footer>
</main>
</body>
</html>

```

5.3 Gestión de ejemplares — librarian-copies.html (CU11)

```

<!-- src/main/resources/templates/librarian-copies.html -->
<!doctype html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="utf-8"/>
    <meta name="viewport" content="width=device-width,initial-scale=1"/>
    <title>Gestión de ejemplares</title>
    <link rel="stylesheet" th:href="@{/css/app.css}">
</head>
<body>
    <header th:replace="fragments/layout :: topbar(${userName}, ${role})"></header>

    <main class="container">
        <section class="card pad">
            <div class="row spread">
                <div>
                    <h1 class="h1">Bibliotecario → Gestión de Ejemplares</h1>
                    <p class="p">Altas, edición y bajas</p>
                </div>
                <a class="btn" th:href="@{/menu}">← Volver</a>
            </div>

            <div class="divider"></div>

            <div class="row">
                <button class="btn primary" type="button">Alta de ejemplar</button>
                <button class="btn" type="button">Editar ejemplar</button>
                <button class="btn danger" type="button">Eliminar ejemplar</button>
            </div>

```

```

<div class="divider"></div>

<div class="card pad">
  <h2 class="h2">Alta de ejemplar</h2>
  <form class="grid two" th:action="@{/librarian/copies/create}"
method="post">
    <div class="field">
      <label class="label">Obra</label>
      <input class="input" name="work" placeholder="ID obra / título...">
    </div>
    <div class="field">
      <label class="label">ID ejemplar</label>
      <input class="input" name="copyId" placeholder="12345">
    </div>
    <div class="field">
      <label class="label">Ubicación</label>
      <input class="input" name="location" placeholder="Estante A3">
    </div>
    <div class="field">
      <label class="label">¿Es referencia?</label>
      <select class="select" name="reference">
        <option value="false">No</option>
        <option value="true">Sí</option>
      </select>
    </div>
    <div class="row" style="grid-column:1/-1;justify-content:flex-end;">
      <button class="btn primary" type="submit">Guardar</button>
    </div>
  </form>
</div>
</section>

<footer th:replace="fragments/layout :: footer"></footer>
</main>
</body>
</html>

```

5.4 Envío a despacho — librarian-professor-delivery.html (CU12)

```

<!-- src/main/resources/templates/librarian-professor-delivery.html -->
<!doctype html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">

```

```

<head>
  <meta charset="utf-8"/>
  <meta name="viewport" content="width=device-width,initial-scale=1"/>
  <title>Envío a despacho</title>
  <link rel="stylesheet" th:href="@{/css/app.css}">
</head>
<body>
  <header th:replace="fragments/layout :: topbar(${userName}, ${role})"></header>

  <main class="container">
    <section class="card pad">
      <div class="row spread">
        <div>
          <h1 class="h1">Bibliotecario → Envío a despacho</h1>
          <p class="p">Reservas de profesores listas para envío</p>
        </div>
        <a class="btn" th:href="@{/menu}">← Volver</a>
      </div>

      <div class="divider"></div>

      <div class="list">
        <article class="card pad" th:each="r : ${profReservations}">
          <div class="row spread">
            <div>
              <p class="p"><strong th:text="'Profesor: ' +
${r.professor}">Profesor: Ana Ruiz</strong></p>
              <p class="p" th:text="'Ejemplar: ' + ${r.copyId} + ' · Estado: ' +
${r.status}">
                Ejemplar: 12345 · Estado: Listo para préstamo
              </p>
            </div>
            <form th:action="@{/librarian/teacher-delivery/confirm}"
method="post">
              <input type="hidden" name="reservationId" th:value="${r.id}">
              <button class="btn primary" type="submit">Registrar préstamo y
marcar envío</button>
            </form>
          </div>
        </article>
      </div>
    </section>
  </main>

```

```

        <div class="card pad" th:if="${#lists.isEmpty(profReservations)}">
            <p class="p">No hay reservas listas para envío.</p>
        </div>
    </div>

    <div class="divider"></div>

    <div class="card pad">
        <h2 class="h2">Estado logístico actual</h2>
        <p class="p" th:text="${logisticsText}">- Ejemplar 12345 → Enviado el
10/03/2025</p>
    </div>
</section>

    <footer th:replace="fragments/layout :: footer"></footer>
</main>
</body>
</html>

```

5.5 Configuración — librarian-config.html (CU13)

```

<!-- src/main/resources/templates/librarian-config.html -->
<!doctype html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="utf-8"/>
    <meta name="viewport" content="width=device-width,initial-scale=1"/>
    <title>Configuración</title>
    <link rel="stylesheet" th:href="@{/css/app.css}">
</head>
<body>
    <header th:replace="fragments/layout :: topbar(${userName}, ${role})"></header>

    <main class="container">
        <section class="card pad" style="max-width:820px;">
            <div class="row spread">
                <div>
                    <h1 class="h1">Bibliotecario → Configuración del sistema</h1>
                    <p class="p">Parámetros globales</p>
                </div>
                <a class="btn" th:href="@{/menu}">← Volver</a>
            </div>

```



```

<div class="divider"></div>

<form class="grid two" th:action="@{/librarian/config/save}" method="post">
  <div class="field">
    <label class="label">Duración préstamo estudiantes</label>
    <input class="input" name="loanDaysStudent" placeholder="15"
th:value="${cfg.loanDaysStudent}">
  </div>
  <div class="field">
    <label class="label">Duración préstamo profesores</label>
    <input class="input" name="loanDaysProfessor" placeholder="30"
th:value="${cfg.loanDaysProfessor}">
  </div>
  <div class="field">
    <label class="label">Días de sanción por día de retraso</label>
    <input class="input" name="penaltyMultiplier" placeholder="2"
th:value="${cfg.penaltyMultiplier}">
  </div>
  <div class="field">
    <label class="label">Duración reservas</label>
    <input class="input" name="reservationDays" placeholder="3"
th:value="${cfg.reservationDays}">
  </div>

  <div class="row" style="grid-column:1/-1;justify-content:flex-end;">
    <button class="btn primary" type="submit">Guardar cambios</button>
  </div>
</form>
</section>

<footer th:replace="fragments/layout :: footer"></footer>
</main>
</body>
</html>

```

Si quieres, te lo dejo “cerrado” al 100% para tu proyecto

Puedo generarte también:

- Controladores Spring Boot + rutas (/menu, /catalog, /books/{id}, etc.)
- Modelos DTO de ejemplo para rellenar las vistas Thymeleaf

- Versión **SPA ligera** (una sola página con navegación por tabs) si prefieres evitar múltiples templates

Dime si estás usando **Thymeleaf** sí o sí (lo más común en Spring MVC) o si tu backend expone API y quieres el front separado.

Sources

You said:

que ventajas tiene Thymeleaf comparado con otras opciones y porque debería usarlo

ChatGPT said:

Thymeleaf te compensa sobre todo si tu app **Spring Boot** es “**server-rendered**” (MVC) y quieres construir pantallas como las de tus wireframes sin montar un frontend separado.

Ventajas claras de Thymeleaf

1) Integración “natural” con Spring MVC

- Resuelve vistas por nombre (`return "catalog";`), bindea modelos (`Model model`) y funciona muy bien con validación (`@Valid + BindingResult`) y mensajes de error en la plantilla.
- Con Spring Security puedes condicionar UI por roles (mostrar “Opciones bibliotecario”, etc.) sin inventarte una capa extra.

2) No necesitas un frontend separado

- No hay build de Node, bundlers, ni deploy de dos artefactos (API + UI).
- Para apps internas / CRUD / portales, reduce fricción y tiempo de entrega.

3) HTML “real” y fácil de maquetar

- Las plantillas son HTML bastante estándar; un diseñador puede abrirlas en el navegador.
- Muy bueno para layouts y fragmentos (`th:replace`), reutilización de topbar/footer, etc.

4) Rendimiento y SEO

- Sale HTML ya renderizado desde el servidor: rápido “first paint”, indexable (si te importa SEO), y funciona bien aunque JS falle.

5) Seguridad y sesión más simple

- Cookies de sesión + CSRF de Spring Security se integran sin complicarte con tokens y CORS.
- Menos superficie de ataque que una SPA grande si el caso de uso es “app de negocio”.

6) Ideal para formularios

- Si tu app tiene mucho formulario (reservas, préstamos, devoluciones, configuración), Thymeleaf + Spring MVC es una combinación muy cómoda: binding, errores por campo, selects, etc.

Por qué usarlo “en tu caso” (lo de tus wireframes)

Tus pantallas son típicas de:

- login SSO
- búsqueda/listado + detalle
- acciones por rol (bibliotecario vs usuario)
- formularios de alta/edición/configuración

Eso encaja perfecto con **server-side rendering + control por roles en servidor**.

Comparado con otras opciones (cuándo NO usarlo)

React/Vue/Angular (SPA)

- ☒ Mejor si necesitas UI muy interactiva (tiempo real, offline-ish, drag&drop pesado, dashboards complejos).
- ☒ Más complejidad: API REST, autenticación/token, CORS, build pipeline, estado en cliente, etc.

JSP

- ☒ Tradicional, funciona.
- ☒ Más antiguo/menos agradable, y hoy en Spring Boot casi siempre prefieren Thymeleaf o algo moderno.

Freemarker / Mustache

- ☒ Más “simples” como motor de plantillas.
- ☒ Menos “Spring-friendly” en formularios y patrones comunes (no es que no se pueda, pero Thymeleaf suele ser más cómodo en apps Spring).

Regla rápida para decidir

- Si vas a hacer una **app web clásica** (pantallas, formularios, roles, sesión) → **Thymeleaf**.
- Si vas a hacer una **UI compleja y muy dinámica** y quieres front separado → **SPA (React/Vue) + API**.

Si me dices si tu arquitectura es **MVC con sesiones** (típico con SSO) o **API + front separado**, te recomiendo el stack exacto y te adapto las plantillas a: Thymeleaf puro, o HTML estático consumiendo endpoints.

You said:

Como funciona thymeleaf

ChatGPT said:

Thymeleaf es un **motor de plantillas**: tú escribes **HTML** con “atributos especiales” (`th:*`) y, cuando llega una petición a tu controlador Spring MVC, el servidor **rellena** esa plantilla con datos (el `Model`) y devuelve **HTML ya renderizado** al navegador.

Flujo típico en Spring Boot

1. El usuario entra a una URL (ej. `/catalog`).
2. Un controlador Spring maneja la petición.
3. El controlador mete datos en un `Model`.
4. Devuelve el nombre de una vista (`"catalog"`).
5. Spring + Thymeleaf buscan `templates/catalog.html`, sustituyen los `th:*` con los datos del `Model` y envían el HTML final.

Ejemplo mínimo

Controlador

```
@GetMapping("/catalog")
public String catalog(@RequestParam(required = false) String q, Model model) {
    model.addAttribute("title", "Catálogo");
    model.addAttribute("q", q);
    model.addAttribute("books", List.of(
        new BookDto(1L, "Clean Code", "Robert C. Martin"),
        new BookDto(2L, "Effective Java", "Joshua Bloch")
    ));
    return "catalog"; // -> templates/catalog.html
}
```

Plantilla (templates/catalog.html)

```
<h1 th:text="${title}">Título por defecto</h1>

<form method="get" th:action="@{/catalog}">
    <input name="q" th:value="${q}" placeholder="Buscar...">
    <button type="submit">Buscar</button>
</form>

<div th:if="${#lists.isEmpty(books)}">No hay resultados</div>

<ul>
    <li th:each="b : ${books}">
        <span th:text="${b.title}">Título</span> -
        <span th:text="${b.author}">Autor</span>
    </li>
</ul>
```

```
<a th:href="@{/books/${b.id}}">Ver</a>
</li>
</ul>
```

Cuando Thymeleaf procesa eso:

- `th:text` **reemplaza** el contenido del elemento
- `th:each` **itera** una lista
- `th:if` **muestra/oculta**
- `th:href` / `th:action` construyen URLs bien (respetan context path)

Atributos Thymeleaf más usados

- `th:text="${...}"` → imprime texto escapado (seguro).
- `th:utext="${...}"` → imprime HTML sin escapar (úsalo con cuidado).
- `th:if` / `th:unless` → condicionales.
- `th:each="x : ${lista}"` → bucles.
- `th:href="@{/ruta}"` y `th:action="@{/ruta}"` → URLs.
- `th:value`, `th:checked`, `th:selected` → formularios.
- `th:replace` / `th:insert` → fragmentos/reutilización.

Fragmentos (layout reutilizable)

Sirve para no repetir header/footer:

```
<!-- fragments/Layout.html -->
<header th:fragment="topbar(user)">
  <span th:text="${user}">Usuario</span>
</header>
```

Y en una página:

```
<header th:replace="fragments/layout :: topbar(${userName})"></header>
```

Formularios y validación (lo que lo hace muy cómodo)

Con Spring Validation puedes mostrar errores por campo:

```
<input th:field="*{email}">
<div th:if="${#fields.hasErrors('email')}" th:errors="*{email}"></div>
```

Esto se apoya en un `@ModelAttribute` validado en el controlador.

¿Necesita JavaScript?

No necesariamente. Thymeleaf funciona perfecto con páginas “clásicas”.

Pero puedes usar JS para partes puntuales (buscador en vivo, modales, etc.).

Si quieres, te pongo un ejemplo completo con:

- DTO + `@ModelAttribute`
- formulario de “Reservar ejemplar”
- manejo de errores y mensajes “flash”
todo en un mini flujo como tus wireframes.

You said:

ahora integra el html con javascript

ChatGPT said:

Vale. La idea es: **mantienes Thymeleaf para render inicial** (rápido y simple) y añades **JavaScript para interacción** (buscar sin recargar, confirmar acciones, toasts, escaneo, etc.). Te dejo una integración “limpia”:

- Un `app.js` global (vanilla)
- Pequeños “módulos” por pantalla usando `data-*` (sin frameworks)
- Acciones por `fetch()` a endpoints Spring (`/api/...`) o a los mismos endpoints MVC si prefieres

1) Añade un JS global en layout

Crea:

```
src/main/resources/static/js/app.js
```

Y en tu `layout.html` (o en cada HTML si no usas layout), mete esto **antes de cerrar**

`</body>`:

```
<script th:src="@{/js/app.js}" defer></script>
```

2) Hooks en HTML (data-attributes)

Catálogo: búsqueda “en vivo” sin recargar

En `catalog.html`, cambia el form por esto (mantiene fallback sin JS):

```
<form class="row" th:action="@{/catalog}" method="get" style="gap:12px;"
      data-catalog-search>
  <input class="input" name="q" placeholder="Buscar..." th:value="${q}"
        style="flex:1;min-width:220px;" data-q>
  <button class="btn primary" type="submit">Buscar</button>
</form>

<div class="divider"></div>
```

```

<div class="list" data-results>
  <!-- plantilla server-render -->
  <article class="item" th:each="book : ${books}">
    <div class="thumb" aria-hidden="true"></div>
    <div class="meta">
      <p class="title" th:text="${book.title}">Título</p>
      <p class="sub" th:text="${book.author}">Autor</p>
    </div>
    <div class="right">
      <a class="btn" th:href="@{/books/${book.id}}">Ver ejemplares →</a>
    </div>
  </article>
</div>

```

Mi área: botones reales (renovar/cancelar) con confirmación

En my-area.html, cambia botones:

```

<button class="btn primary"
  type="button"
  th:attr="data-renew-loan=${loan.id}"
  th:disabled="${loan.reference}">
  Renovar
</button>

<button class="btn danger"
  type="button"
  th:attr="data-cancel-reservation=${res.id}">
  Cancelar reserva
</button>

```

Reservar: confirmar vía fetch (sin recarga)

En reserve.html (tu form):

```

<form th:action="@{/reserve/confirm}" method="post"
  class="row spread" style="align-items:flex-end;"
  data-reserve-form
  th:attr="data-copy-id=${copyId}">
  <input type="hidden" name="copyId" th:value="${copyId}">
  <button class="btn primary" type="submit">Confirmar reserva</button>
</form>

```

Bibliotecario: “escaneo” con enter y autosearch

En librarian-loan.html añade data-scan al input:

```
<input class="input" name="copyCode" placeholder="Escanear..."
      data-scan-input />
```

3) app.js (global) — con fetch, debounce, toasts y confirm

```
// src/main/resources/static/js/app.js
```

```
(function () {
  // ----- Helpers -----
  const $ = (sel, root = document) => root.querySelector(sel);
  const $$ = (sel, root = document) => Array.from(root.querySelectorAll(sel));

  function debounce(fn, ms = 250) {
    let t;
    return (...args) => {
      clearTimeout(t);
      t = setTimeout(() => fn(...args), ms);
    };
  }

  function toast(message, type = "info") {
    let host = $("#toast-host");
    if (!host) {
      host = document.createElement("div");
      host.id = "toast-host";
      host.style.position = "fixed";
      host.style.right = "16px";
      host.style.bottom = "16px";
      host.style.display = "grid";
      host.style.gap = "10px";
      host.style.zIndex = "9999";
      document.body.appendChild(host);
    }

    const el = document.createElement("div");
    el.textContent = message;
    el.style.padding = "12px 14px";
    el.style.borderRadius = "12px";
    el.style.border = "1px solid rgba(255,255,255,.18)";
```



```

el.style.background = "rgba(15,26,47,.92)";
el.style.backdropFilter = "blur(10px)";
el.style.boxShadow = "0 18px 45px rgba(0,0,0,.35)";
el.style.maxWidth = "340px";

if (type === "ok") el.style.outline = "2px solid rgba(52,211,153,.35)";
if (type === "bad") el.style.outline = "2px solid rgba(251,113,133,.35)";
if (type === "warn") el.style.outline = "2px solid rgba(251,191,36,.35)";

host.appendChild(el);
setTimeout(() => el.remove(), 2800);
}

async function confirmDialog(message) {
  // Simple y efectivo; puedes sustituirlo por un modal luego.
  return window.confirm(message);
}

async function api(url, { method = "GET", body, headers } = {}) {
  const opts = { method, headers: headers || {} };

  // Si mandas JSON:
  if (body && !(body instanceof FormData)) {
    opts.headers["Content-Type"] = "application/json";
    opts.body = JSON.stringify(body);
  } else if (body instanceof FormData) {
    opts.body = body;
  }

  // Si usas Spring Security + CSRF, puedes añadir el token aquí (si lo pintas
  // en meta tags)
  const csrf = $('meta[name="_csrf"]')?.content;
  const csrfHeader = $('meta[name="_csrf_header"]')?.content;
  if (csrf && csrfHeader) opts.headers[csrfHeader] = csrf;

  const res = await fetch(url, opts);
  const contentType = res.headers.get("content-type") || "";
  const data = contentType.includes("application/json") ? await res.json() :
  await res.text();

```

```

    if (!res.ok) {
        const msg = typeof data === "string" ? data : (data.message || "Error");
        throw new Error(msg);
    }
    return data;
}

// ----- Feature: Catálogo live search -----
(function initCatalogSearch() {
    const form = document.querySelector("[data-catalog-search]");
    if (!form) return;

    const input = form.querySelector("[data-q]");
    const results = document.querySelector("[data-results]");
    if (!input || !results) return;

    const render = (books) => {
        results.innerHTML = "";
        if (!books.length) {
            results.innerHTML = `<div class="card pad"><p class="p">No hay resultados.</p></div>`;
            return;
        }

        for (const b of books) {
            const el = document.createElement("article");
            el.className = "item";
            el.innerHTML = `
                <div class="thumb" aria-hidden="true"></div>
                <div class="meta">
                    <p class="title"></p>
                    <p class="sub"></p>
                </div>
                <div class="right">
                    <a class="btn">Ver ejemplares →</a>
                </div>
            `;
            el.querySelector(".title").textContent = b.title;
            el.querySelector(".sub").textContent = b.author;
            el.querySelector("a").href = `/books/${b.id}`;
            results.appendChild(el);
        }
    };

```

```

    }
  };

  const doSearch = debounce(async () => {
    const q = input.value.trim();
    // Si está vacío, puedes dejar el server-render o mostrar vacío
    if (q.length === 0) return;

    try {
      // Endpoint recomendado: GET /api/catalog?q=...
      const data = await api(`/api/catalog?q=${encodeURIComponent(q)}`);
      // Espera algo como: [{id,title,author}, ...]
      render(Array.isArray(data) ? data : (data.items || []));
    } catch (e) {
      toast(e.message, "bad");
    }
  }, 250);

  input.addEventListener("input", doSearch);
})();

// ----- Feature: Reservar (fetch + toast) -----
(function initReserve() {
  const form = document.querySelector("[data-reserve-form]");
  if (!form) return;

  form.addEventListener("submit", async (ev) => {
    ev.preventDefault();

    const copyId = form.getAttribute("data-copy-id");
    const ok = await confirmDialog(`¿Confirmar reserva del ejemplar
${copyId}?`);
    if (!ok) return;

    try {
      // Endpoint recomendado: POST /api/reservations {copyId}
      await api("/api/reservations", { method: "POST", body: { copyId } });
      toast("Reserva confirmada ✅", "ok");
      // Redirige a "Mi área" o recarga
      window.location.href = "/my";
    }
  });
})();

```

```

    } catch (e) {
      toast(e.message, "bad");
    }
  });
})();

// ----- Feature: Renovar préstamo / Cancelar reserva -----
(function initMyAreaActions() {
  const renewBtns = $$("[data-renew-loan]");
  renewBtns.forEach((btn) => {
    btn.addEventListener("click", async () => {
      const loanId = btn.getAttribute("data-renew-loan");
      const ok = await confirmDialog("¿Renovar este préstamo?");
      if (!ok) return;

      try {
        // Endpoint recomendado: POST /api/loans/{id}/renew
        await api(`/api/loans/${encodeURIComponent(loanId)}/renew`, { method:
"POST" });
        toast("Préstamo renovado ", "ok");
        window.location.reload();
      } catch (e) {
        toast(e.message, "bad");
      }
    });
  });

  const cancelBtns = $$("[data-cancel-reservation]");
  cancelBtns.forEach((btn) => {
    btn.addEventListener("click", async () => {
      const resId = btn.getAttribute("data-cancel-reservation");
      const ok = await confirmDialog("¿Cancelar esta reserva?");
      if (!ok) return;

      try {
        // Endpoint recomendado: DELETE /api/reservations/{id}
        await api(`/api/reservations/${encodeURIComponent(resId)}`, { method:
"DELETE" });
        toast("Reserva cancelada", "ok");
        window.location.reload();
      } catch (e) {

```

```

        toast(e.message, "bad");
    }
    });
});
})();

// ----- Feature: Bibliotecario scan (enter = enviar) -----
(function initScanUX() {
    const input = document.querySelector("[data-scan-input]");
    if (!input) return;

    input.addEventListener("keydown", (ev) => {
        if (ev.key === "Enter") {
            // deja que el form haga submit normal (fallback)
            // o podrías interceptar y hacer fetch si te interesa
        }
    });
})();
})();
})();

```

4) Qué endpoints necesitas en Spring (súper rápido)

Para que el JS funcione como está, lo ideal es añadir endpoints JSON:

- GET /api/catalog?q=... → lista de libros
- POST /api/reservations body { copyId }
- POST /api/loans/{id}/renew
- DELETE /api/reservations/{id}

Si no quieres API, también se puede hacer con MVC “clásico” (POST normal + redirect), y usar JS solo para confirmar/toasts; pero con esos 4 endpoints te queda muy bien.

Si me dices si estás usando **Spring Security con CSRF** (muy probable con SSO), te adapto el `layout.html` para pintar los meta tags de CSRF y que el `fetch()` funcione sin 403.

You said:

como puedo hacer que cuando vaya a localhost:8080/ me habra la pagina principal

ChatGPT said:

En Spring Boot la URL / no abre nada automáticamente. Tú tienes que decirle qué vista debe mostrar cuando alguien entra a:

<http://localhost:8080/>

Lo normal es que ahí mandes al login o al menú principal según si el usuario está autenticado.

Ejemplo básico:

```
@Controller
public class HomeController {

    @GetMapping("/")
    public String home() {
        return "redirect:/menu";
    }
}
```

Con eso:

- localhost:8080/ → redirige a /menu
- /menu será tu pantalla principal.

Y tu controlador del menú:

```
@GetMapping("/menu")
public String menu(Model model) {
    model.addAttribute("title", "Menú principal");
    return "menu";
}
```

Si quieres que primero vaya al login:

```
@Controller
public class HomeController {

    @GetMapping("/")
    public String home() {
        return "redirect:/login";
    }
}
```

Si usas Spring Security (con SSO seguro que sí), lo ideal es:

- / → si NO está autenticado → login
- / → si está autenticado → menú

Ejemplo típico:

```
@Controller
public class HomeController {

    @GetMapping("/")
    public String root(Authentication auth) {
        if (auth == null || !auth.isAuthenticated()) {
            return "redirect:/login";
        }
        return "redirect:/menu";
    }
}
```

Alternativa aún más limpia: hacerlo en SecurityConfig

```
@Bean
SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/", "/login").permitAll()
            .anyRequest().authenticated()
        )
        .formLogin(login -> login
            .loginPage("/login")
            .defaultSuccessUrl("/menu", true)
        );
    return http.build();
}
```

Y tu controlador raíz solo:

```
@GetMapping("/")
public String root() {
    return "redirect:/menu";
}
```

Resumen mental rápido:

URL visitada	Qué pasa
/	Redirige a /menu o /login
/login	Carga login.html
/menu	Carga tu pantalla principal

Así cuando abras el navegador y pongas:

<http://localhost:8080/>

verás directamente la página principal de tu aplicación.

You said:



tengo así los controles pero `home()` no me funciona, devuelve literalmente ese string

ChatGPT said:

El problema está en que usaste `@RestController`.

`@RestController` significa:

“Todo lo que devuelvas es el cuerpo de la respuesta HTTP”.

Por eso Spring no interpreta `"redirect:/menu"` como una redirección, sino que lo imprime tal cual en el navegador.

Para trabajar con vistas (Thymeleaf) necesitas usar `@Controller`.

Cambia esto:

```
@RestController
@RequestMapping("/")
class GeneralController {
```

por esto:

```
@Controller
@RequestMapping("/")
class GeneralController {
```

Y deja el resto igual:

```
@Controller
@RequestMapping("/")
class GeneralController {

    @GetMapping
    public String home() {
        return "redirect:/menu";
    }

    @GetMapping("/menu")
    public String menu(Model model) {
```



```

        model.addAttribute("title", "Menú principal");
        return "menu";
    }

    @GetMapping("/logout")
    public String logout() {
        return "Logged out";
    }
}

```

Ahora el flujo será:

URL	Resultado
localhost:8080/	Redirige a /menu
localhost:8080/menu	Carga menu.html
"redirect:/menu"	Ya no se imprime, se interpreta como orden

Regla de oro en Spring:

Anotación	Uso
@RestController	APIs REST (JSON, texto, datos)
@Controller	Páginas HTML / Thymeleaf

Si mezclas ambos conceptos, Spring siempre tratará el retorno como texto plano.

Extra: si quieres que /logout también vaya a una vista:

```

@GetMapping("/logout")
public String logout() {
    return "redirect:/login";
}

```

Y ahí renderizas tu login.html.

Con ese simple cambio tu home() va a funcionar exactamente como esperas.

You said:

necesito la configuracion para las vistas. En spring solo tengo un api con la configuracion del login. Que más me hace falta

ChatGPT said:

Si ahora mismo tu proyecto está montado “solo como API” (tipo REST), para que te funcionen **vistas Thymeleaf (HTML en /templates)** necesitas 3 cosas: **dependencia**, **estructura**, y **config mínima** (y si usas Security, permitir estáticos y mapear login).

1) Dependencias (Maven/Gradle)

Maven (pom.xml)

Asegúrate de tener:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Ojo: si tienes spring-boot-starter-webflux, eso ya es otro stack. Para Thymeleaf clásico usa starter-web.

Gradle

```
implementation 'org.springframework.boot:spring-boot-starter-thymeleaf'
implementation 'org.springframework.boot:spring-boot-starter-web'
```

2) Carpetas correctas

- Vistas: src/main/resources/templates/*.html
- Estáticos: src/main/resources/static/css/app.css, static/js/app.js, etc.

Ejemplo:

```
src/main/resources/templates/menu.html
src/main/resources/static/css/app.css
```

3) Controladores MVC (no REST)

Para servir HTML necesitas @Controller (no @RestController):

```
@Controller
public class ViewController {
```

```

@GetMapping("/")
public String root() {
    return "redirect:/menu";
}

@GetMapping("/menu")
public String menu(Model model) {
    model.addAttribute("title", "Menú principal");
    return "menu"; // -> templates/menu.html
}

@GetMapping("/login")
public String login() {
    return "login"; // -> templates/Login.html
}
}

```

4) Configuración (application.properties / yml)

Con Spring Boot normalmente **no hace falta nada**, pero si quieres dejarlo explícito:

application.properties

```

spring.thymeleaf.prefix=classpath:/templates/
spring.thymeleaf.suffix=.html
spring.thymeleaf.cache=false

```

- cache=false viene bien en desarrollo (ves cambios sin reiniciar).

5) Spring Security (lo que normalmente te falta si tienes login SSO)

Si solo tenías seguridad para API, probablemente te está:

- bloqueando /menu
- bloqueando /css/app.css
- o no redirige bien post-login

Ejemplo típico **para vistas + API**:

```

@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests(auth -> auth
            // estáticos y login público
            .requestMatchers("/", "/login", "/error",

```

```

        "/css/**", "/js/**", "/img/**").permitAll()

        // si tienes endpoints API
        .requestMatchers("/api/**").authenticated()

        // vistas
        .anyRequest().authenticated()
    )
    // si estás con OAuth2/SSO:
    .oauth2Login(oauth -> oauth
        .loginPage("/login") // tu login.html
        .defaultSuccessUrl("/menu", true) // después de login
    )
    .logout(logout -> logout
        .logoutUrl("/logout")
        .logoutSuccessUrl("/login")
    );

    return http.build();
}

```

Si tu login es con OAuth2/SSO, esto es lo normal. Si es formLogin, cambia oauth2Login por formLogin.

6) Error común: tienes solo @RestControllerAdvice / config “API only”

No pasa nada: puedes convivir con ambos:

- Controladores REST: @RestController bajo /api/**
- Controladores de vistas: @Controller para /menu, /catalog, etc.

Ejemplo limpio:

- /api/... → JSON
- /... → HTML

Checklist rápido (para que funcione ya)

1. ☒ spring-boot-starter-thymeleaf añadido
2. ☒ templates/menu.html existe
3. ☒ Tienes un @Controller que devuelve "menu"

4. ☒ Security permite `/css/**` y `/js/**`
5. ☒ No estás usando `@RestController` para vistas

Si me pegas tu `SecurityConfig` (el del login que ya tienes) te lo adapto exactamente a tu caso (OAuth2/SSO o formLogin) sin romper tu API.

You said:
que es webflux

ChatGPT said:

WebFlux es el stack **reactivo** de Spring para hacer aplicaciones web no bloqueantes. Es la alternativa moderna a Spring MVC (el “clásico”).

En Spring hay dos mundos:

Stack	Modelo	Cómo trabaja
Spring MVC	Bloqueante	Un hilo por petición
Spring WebFlux	No bloqueante	Event loop + programación reactiva

Tú ahora mismo estás trabajando mentalmente en el modelo Spring MVC + Thymeleaf, que es el tradicional y el más sencillo para aplicaciones como la tuya.

En Spring MVC (lo que estás usando con Thymeleaf):

Petición → Controlador → Espera a la BD → Respuesta → HTML

Cada petición:

- Usa un hilo.
- Si la BD tarda, el hilo queda bloqueado.
- Es simple y muy estable.

En WebFlux:

Petición → Controlador → Registro de operación → el hilo se libera
→ cuando llega el dato → se continúa

Aquí:

- No hay bloqueo.
- Todo es asíncrono.

- Trabajas con `Mono<T>` y `Flux<T>` en vez de `T`.

Ejemplo:

Spring MVC:

```
@GetMapping("/books")
public List<Book> books() {
    return service.findAll();
}
```

WebFlux:

```
@GetMapping("/books")
public Flux<Book> books() {
    return service.findAll();
}
```

Y el servicio devuelve `Flux<Book>`, no una lista normal.

WebFlux es ideal cuando:

- Tienes miles de conexiones simultáneas.
- Haces streaming de datos.
- Microservicios muy cargados.
- Sistemas con IO muy lento (muchas APIs externas).

Pero tiene coste:

- Código más complejo.
- Debug más difícil.
- Thymeleaf casi no se usa ahí (aunque existe versión reactiva).
- La mayoría de aplicaciones “de negocio” no lo necesitan.

Regla clara:

Tu caso (biblioteca, SSO, formularios, CRUD)	Elección correcta
App web clásica con pantallas	Spring MVC
Thymeleaf	Funciona perfecto
Simplicidad	Máxima
WebFlux	Sobreingeniería

Otra pista rápida:

Si en tu `pom.xml` tienes esto:

```
spring-boot-starter-web
```

→ Estás en Spring MVC.

Si tienes esto:

```
spring-boot-starter-webflux
```

→ Estás en WebFlux.

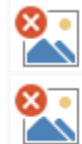
No deberías mezclar ambos.

En resumen sencillo:

WebFlux es el “Spring para programación reactiva y no bloqueante”.

Thymeleaf + pantallas HTML + tu proyecto actual → **Spring MVC**, no WebFlux.

You said:



quiero que modifiques el css teniendo en cuenta este estilo. Solo quiero tener en cuenta el estilo, no quiero añadir las imágenes que salen en la pagina.

ChatGPT said:

Perfecto. El estilo de la UPM que muestras es muy “institucional”:

- **Fondo blanco**
- **Header azul UPM** (barra superior)
- **Tipografía sencilla** (sans)
- **Contenido centrado con mucho blanco**
- **Panel lateral gris claro** (menú)
- **Links azules**, tablas con borde fino
- **Footer oscuro/gris** con links en blanco

Abajo tienes un app.css **nuevo** (reemplaza el que te pasé). Mantiene tus clases (.topbar, .container, .card, .btn, .list, .item, etc.) pero cambia la estética para parecerse a ese estilo. **No añade imágenes.**

src/main/resources/static/css/app.css (versión estilo UPM)

```
/* ===== UPM-like institutional style (no images) ===== */

:root{
  /* Palette */
  --upm-blue: #2f6fb6;      /* main header blue */
  --upm-blue-2: #1f5fa7;    /* hover/darker */
  --upm-link: #1a5fb4;

  --bg: #ffffff;
  --surface: #ffffff;
  --surface-2: #f2f4f7;     /* light gray panels */
  --surface-3: #e9edf2;     /* separators */
  --text: #1f2937;          /* dark gray */
  --muted: #6b7280;         /* medium gray */
  --border: #d7dde5;        /* soft border */

  --danger: #b42318;
  --warning: #b54708;
  --ok: #027a48;

  --radius: 6px;            /* more "rectangular" than before */
  --shadow: 0 1px 2px rgba(16,24,40,.08);

  --container: 1100px;
  --gap: 14px;
  --pad: 16px;

  --font: Arial, Helvetica, "Segoe UI", Roboto, system-ui, -apple-system, sans-serif;
}

*{ box-sizing:border-box; }
html,body{ height:100%; }
body{
  margin:0;
  font-family: var(--font);
  color: var(--text);
  background: var(--bg);
}
```



```

a{ color: var(--upm-link); text-decoration:none; }
a:hover{ text-decoration: underline; }
img{ max-width:100%; display:block; }

/* ===== Layout ===== */
.container{
  max-width: var(--container);
  margin: 0 auto;
  padding: 18px 16px 26px;
}

/* ===== Topbar ===== */
.topbar{
  position: sticky;
  top:0;
  z-index: 50;
  background: var(--upm-blue);
  border-bottom: 1px solid rgba(255,255,255,.20);
}
.topbar-inner{
  max-width: var(--container);
  margin:0 auto;
  padding: 14px 16px;
  display:flex;
  align-items:center;
  justify-content:space-between;
  gap: 12px;
}

.brand{
  display:flex;
  align-items:center;
  gap: 10px;
  font-weight: 700;
  letter-spacing: .2px;
  color: #fff;
}
.brand:hover{ text-decoration:none; }
.brand-badge{
  width: 34px;

```

```

    height: 34px;
    border-radius: 4px;
    background: rgba(255,255,255,.18); /* placeholder block (no image) */
    border: 1px solid rgba(255,255,255,.25);
}

.top-actions{
    display: flex;
    align-items: center;
    gap: 10px;
}

.top-actions .chip{
    background: rgba(255,255,255,.14);
    border-color: rgba(255,255,255,.22);
    color: #fff;
}

/* ===== Typography ===== */
.h1{
    font-size: 30px;
    line-height: 1.15;
    margin: 0 0 8px;
    color: #1f4f93; /* UPM-like heading tone */
    font-weight: 700;
}

.h2{
    font-size: 16px;
    margin: 0 0 10px;
    color: var(--text);
    font-weight: 700;
}

.p{
    margin: 0;
    color: var(--muted);
    line-height: 1.55;
}

.divider{
    height: 1px;
    background: var(--border);
    margin: 14px 0;
}

```

```

}

/* ===== Cards / Surfaces ===== */
.card{
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: var(--radius);
  box-shadow: var(--shadow);
}
.card.pad{ padding: var(--pad); }

/* "Panel" style like left menu blocks */
.panel{
  background: var(--surface-2);
  border: 1px solid var(--border);
  border-radius: var(--radius);
}

/* ===== Grid ===== */
.grid{
  display:grid;
  gap: var(--gap);
}
.grid.two{ grid-template-columns: 1fr 1fr; }
.grid.three{ grid-template-columns: repeat(3, 1fr); }
@media (max-width: 860px){
  .grid.two, .grid.three{ grid-template-columns: 1fr; }
}

/* Optional: 2-column layout (sidebar + content) if you use it */
.layout{
  display:grid;
  grid-template-columns: 260px 1fr;
  gap: 16px;
}
@media (max-width: 980px){
  .layout{ grid-template-columns: 1fr; }
}

/* ===== Forms ===== */
.field{ display:grid; gap: 8px; }

```

```

.label{
  font-size: 13px;
  color: var(--text);
  font-weight: 700;
}
.input, .select{
  width:100%;
  padding: 10px 10px;
  border-radius: 4px;
  border: 1px solid var(--border);
  background: #fff;
  color: var(--text);
  outline: none;
}
.input::placeholder{ color: #9aa4b2; }
.input:focus, .select:focus{
  border-color: var(--upm-blue);
  box-shadow: 0 0 0 3px rgba(47,111,182,.18);
}

/* ===== Buttons ===== */
.btn{
  display:inline-flex;
  align-items:center;
  justify-content:center;
  gap: 8px;
  padding: 9px 12px;
  border-radius: 4px;
  border: 1px solid var(--border);
  background: #fff;
  color: var(--text);
  cursor:pointer;
  font-weight: 700;
}
.btn:hover{
  background: #f8fafc;
  text-decoration:none;
}

.btn.primary{
  border-color: var(--upm-blue);

```

```

    background: var(--upm-blue);
    color: #fff;
}
.btn.primary:hover{
    background: var(--upm-blue-2);
    border-color: var(--upm-blue-2);
}

.btn.ghost{
    background: transparent;
    border-color: rgba(255,255,255,.32);
    color:#fff;
}
.btn.ghost:hover{
    background: rgba(255,255,255,.12);
}

.btn.danger{
    border-color: rgba(180,35,24,.35);
    background: rgba(180,35,24,.08);
    color: var(--danger);
}
.btn.danger:hover{ background: rgba(180,35,24,.12); }

/* ===== Chips / badges ===== */
.chip{
    font-size: 12px;
    padding: 4px 10px;
    border-radius: 999px;
    border: 1px solid var(--border);
    background: var(--surface-2);
    color: var(--text);
    font-weight: 700;
}
.chip.ok{ color: var(--ok); border-color: rgba(2,122,72,.25); background:
rgba(2,122,72,.08); }
.chip.warn{ color: var(--warning); border-color: rgba(181,71,8,.25); background:
rgba(181,71,8,.08); }
.chip.bad{ color: var(--danger); border-color: rgba(180,35,24,.25); background:
rgba(180,35,24,.08); }

```

```

/* ===== List items (catalog, loans, etc.) ===== */
.list{ display:grid; gap: 10px; }

.item{
  display:grid;
  grid-template-columns: 58px 1fr auto;
  gap: 12px;
  align-items:center;
  padding: 12px;
  border-radius: 4px;
  border: 1px solid var(--border);
  background: #fff;
}
@media (max-width: 620px){
  .item{ grid-template-columns: 48px 1fr; }
  .item .right{ grid-column: 1 / -1; justify-content:flex-start; }
}

.thumb{
  width: 58px;
  height: 58px;
  border-radius: 4px;
  background: var(--surface-2);
  border: 1px solid var(--border);
}

.meta .title{
  margin:0;
  font-weight: 700;
  color: var(--text);
}
.meta .sub{
  margin: 3px 0 0;
  color: var(--muted);
  font-size: 13px;
}

.row{
  display:flex;
  gap: 10px;
  align-items:center;

```

```

    flex-wrap:wrap;
}
.spread{ justify-content:space-between; }

.right{
    display:flex;
    gap: 10px;
    align-items:center;
    justify-content:flex-end;
    flex-wrap:wrap;
}

/* ===== Tables (UPM-like) ===== */
.table{
    width:100%;
    border-collapse: collapse;
    border: 1px solid var(--border);
    background:#fff;
}
.table th, .table td{
    padding: 10px 12px;
    border-bottom: 1px solid var(--border);
    vertical-align: top;
    font-size: 14px;
}
.table th{
    background: var(--surface-2);
    text-align:left;
    font-weight: 700;
}
.table tr:last-child td{ border-bottom: 0; }

/* ===== Sidebar menu Look (if you use a left nav) ===== */
.sidemenu{
    background: var(--surface-2);
    border: 1px solid var(--border);
    border-radius: var(--radius);
    overflow:hidden;
}
.sidemenu .sitem{
    display:flex;

```

```

    align-items:center;
    justify-content:space-between;
    padding: 10px 12px;
    border-bottom: 1px solid var(--border);
    color: var(--text);
    font-weight: 700;
}
.sidemenu .sitem:last-child{ border-bottom:0; }
.sidemenu .sitem:hover{ background: var(--surface-3); text-decoration:none; }

/* ===== Footer (dark institutional) ===== */
.footer{
    background: #3e3e3e;
    color: #fff;
    font-size: 13px;
    padding: 16px 10px;
    text-align:center;
    margin-top: 26px;
}
.footer a{ color:#fff; text-decoration:none; }
.footer a:hover{ text-decoration:underline; }

```

(Opcional) Cómo conseguir el “menú lateral” tipo UPM con tus páginas

Si quieres que alguna página (por ejemplo /catalog) tenga sidebar + contenido como en la captura, sin tocar demasiado HTML:

```

<div class="layout">
  <aside class="sidemenu">
    <a class="sitem" href="/menu">Menú</a>
    <a class="sitem" href="/catalog">Catálogo</a>
    <a class="sitem" href="/my">Mis préstamos</a>
  </aside>

  <section class="card pad">
    <!-- tu contenido actual -->
  </section>
</div>

```

Si me dices qué páginas quieres que lleven **sidebar** (solo catálogo, o todas salvo login), te ajusto la maqueta exacta sin que tengas que reescribir plantillas.

You said:

Deprecated unwrapped fragment expression "fragments/layout :: footer" found in template menu, line 61, col 13. Please use the complete syntax of fragment expressions instead ("~{fragments/layout :: footer}"). The old, unwrapped syntax for fragment expressions will be removed in future versions of Thymeleaf.

ChatGPT said:

Eso no es un error, es una **advertencia de Thymeleaf**. Te está diciendo que la sintaxis corta que estás usando está deprecada y que en futuras versiones dejará de funcionar.

Tú tienes algo así:

```
<footer th:replace="fragments/layout :: footer"></footer>
```

Y ahora Thymeleaf quiere que uses la sintaxis completa con ~{ }:

```
<footer th:replace="~{fragments/layout :: footer}"></footer>
```

Lo mismo para cualquier fragmento:

Antes:

```
<header th:replace="fragments/layout :: topbar(${userName}, ${role})"></header>
```

Ahora:

```
<header th:replace="~{fragments/layout :: topbar(${userName},  
${role})}"></header>
```

Y para el footer:

```
<footer th:replace="~{fragments/layout :: footer}"></footer>
```

Qué significa eso:

- ~{ . . . } es la forma “oficial” y completa de decirle a Thymeleaf: “Esto es una expresión de fragmento, no un texto normal”.
- Antes Thymeleaf permitía escribirla sin ~{ } por comodidad.
- Ahora lo mantiene solo por compatibilidad, pero lo va a eliminar.

Te recomiendo que hagas un buscar y reemplazar:

Buscar:

```
th:replace="fragments/
```

Reemplazar por:

```
th:replace=~{fragments/
```

Y lo mismo con `th:insert` si lo usas.

Después de eso:

- Se te van a ir los warnings
- Tu proyecto queda preparado para versiones nuevas de Thymeleaf
- Estás usando la sintaxis “correcta” y moderna

You said:

create rol in cognito user pool

ChatGPT said:

En Cognito hay una confusión muy común:

en un *User Pool* no se crean “roles” como tal, lo que se crean son **Groups**.

Los *IAM Roles* existen, pero están asociados al *Identity Pool*, no al *User Pool*.

Para aplicaciones web como la tuya (biblioteca, bibliotecario, usuario, etc.), lo correcto es:

- User Pool → autenticación + grupos (roles lógicos)
- Groups → USER, LIBRARIAN, ADMIN, etc.
- Spring Security → lees el grupo como rol

Es decir:

En Cognito el “rol” de tu aplicación = **Group del User Pool**

Paso 1. Crear un Group en el User Pool

1. Ve a AWS Console → Cognito
2. Entra en tu **User Pool**
3. En el menú lateral: **User management** → **Groups**
4. Click en **Create group**
5. Ejemplo:

Campo	Valor
Group name	LIBRARIAN
Description	Bibliotecario
Precedence	1

Crea también:

- USER
- PROFESSOR (si lo necesitas)
- ADMIN

Paso 2. Asignar un usuario a un Group

1. User management → Users
2. Selecciona el usuario
3. Pestaña **Groups**
4. Add to group → selecciona LIBRARIAN

Paso 3. Hacer que el Group viaje en el token JWT

Cognito mete los grupos en el JWT automáticamente:

Claim:

```
"cognito:groups": ["LIBRARIAN"]
```

Ejemplo real de token:

```
{
  "sub": "...",
  "email": "ana@upm.es",
  "cognito:groups": ["LIBRARIAN"]
}
```

Paso 4. Mapear grupos Cognito a roles Spring Security

En Spring Security:

```
@Bean
SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/librarian/**").hasRole("LIBRARIAN")
            .requestMatchers("/menu").authenticated()
            .anyRequest().authenticated()
        )
        .oauth2Login();

    return http.build();
}
```

Y un converter para que Spring lea `cognito:groups` como `ROLE_*`:

```
@Bean
public JwtAuthenticationConverter jwtAuthenticationConverter() {
    JwtGrantedAuthoritiesConverter converter = new
    JwtGrantedAuthoritiesConverter();
    converter.setAuthoritiesClaimName("cognito:groups");
    converter.setAuthorityPrefix("ROLE_");

    JwtAuthenticationConverter jwtConverter = new JwtAuthenticationConverter();
    jwtConverter.setJwtGrantedAuthoritiesConverter(converter);
    return jwtConverter;
}
```

Paso 5. Usar los roles en Thymeleaf

En tus vistas:

```
<div th:if="${#authorization.expression('hasRole(''LIBRARIAN'')')}">
    Opciones de bibliotecario
</div>
```

Resumen corto:

Cognito	Tu aplicación
User Pool Group	Rol lógico
cognito:groups	Authorities en Spring
LIBRARIAN	ROLE_LIBRARIAN

Solo necesitas IAM Roles si:

- Usas un Identity Pool
- Quieres que el usuario acceda a recursos AWS (S3, DynamoDB, etc.)

Para tu caso (login SSO + control de pantallas), **User Pool + Groups es exactamente lo que necesitas.**

You said:

diferencia entre identity pool y grupos

ChatGPT said:

En Cognito hay dos conceptos distintos que mucha gente mezcla:

- **User Pool**
- **Identity Pool**

Y dentro del User Pool están los **Groups**.

Cada uno sirve para algo diferente.

Primero: User Pool

Es para **autenticación**.

Sirve para:

- Login (usuario/contraseña, SSO, OAuth, etc.)
- Gestión de usuarios
- Tokens JWT
- Claims como email, nombre, etc.
- Grupos (roles lógicos)

Cuando un usuario se autentica contra un User Pool obtiene un token como este:

```
{
  "sub": "...",
  "email": "ana@upm.es",
  "cognito:groups": ["LIBRARIAN"]
}
```

Aquí es donde viven tus:

- USER
- LIBRARIAN
- ADMIN
- PROFESSOR

Los grupos solo sirven dentro de tu aplicación para decidir:
qué pantallas ve, qué endpoints puede usar, qué rol tiene.

Segundo: Identity Pool

Es para **autorización a servicios de AWS**.

Sirve para:

- Dar credenciales temporales de AWS
- Permitir acceso a:

- S3
- DynamoDB
- SNS
- etc.

Ejemplo típico:

“Este usuario autenticado puede subir ficheros a un bucket S3”.

Ahí aparecen los **IAM Roles reales**.

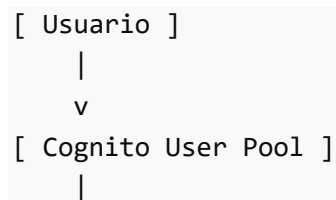
Diferencia clara:

Concepto	Para qué sirve	Se usa en tu app
User Pool	Autenticar personas	✅ Sí
Groups (en User Pool)	Roles de tu aplicación	✅ Sí
Identity Pool	Acceso a AWS (S3, Dynamo...)	❌ No (salvo que tu app use AWS directamente)
IAM Role	Permisos AWS reales	❌ No necesario para tu caso

Visualmente:



Solo si necesitas acceso a AWS:



```
    v
[ Cognito Identity Pool ]
    |
    v
[ IAM Role ]
    |
    v
[ S3 / DynamoDB / etc ]
```

En tu proyecto de biblioteca:

- Solo necesitas **User Pool + Groups**
- No necesitas Identity Pool
- No necesitas IAM Roles

Tus “roles” son simplemente:

```
Cognito Group → Spring Security ROLE_*
```

Y eso es exactamente lo correcto para una aplicación web clásica.

You said:

no tengo la clase `JwtGrantedAuthoritiesConverter` , lo tengo que crear o es un import

ChatGPT said:

No la tienes que crear. Es una clase que ya existe en Spring Security.

Si no te aparece es porque te falta el *starter* correcto o estás usando otro tipo de configuración.

La clase está en:

```
org.springframework.security.oauth2.server.resource.authentication.JwtGrantedAuth  
oritiesConverter
```

y la otra en:

```
org.springframework.security.oauth2.server.resource.authentication.JwtAuthenticat  
ionConverter
```

Import completo:

```
import  
org.springframework.security.oauth2.server.resource.authentication.JwtGrantedAuth  
oritiesConverter;  
import  
org.springframework.security.oauth2.server.resource.authentication.JwtAuthenticat  
ionConverter;
```

Si tu IDE no las encuentra, casi seguro te falta esta dependencia:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>
```

aunque estés usando login con Cognito, Spring necesita este starter para poder trabajar con JWT y converters.

Tu pom.xml mínimo para Cognito + Thymeleaf suele tener:

```
<dependencies>
  <!-- MVC -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <!-- Thymeleaf -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
  </dependency>

  <!-- Security -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
  </dependency>

  <!-- OAuth2 / Cognito -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-oauth2-client</artifactId>
  </dependency>

  <!-- JWT handling -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
  </dependency>
</dependencies>
```


Después de añadirlo:

- `mvn clean install`
- o recargar proyecto en tu IDE

Y ya podrás usar:

```
@Bean
public JwtAuthenticationConverter jwtAuthenticationConverter() {
    JwtGrantedAuthoritiesConverter converter = new
    JwtGrantedAuthoritiesConverter();
    converter.setAuthoritiesClaimName("cognito:groups");
    converter.setAuthorityPrefix("ROLE_");

    JwtAuthenticationConverter jwtConverter = new JwtAuthenticationConverter();
    jwtConverter.setJwtGrantedAuthoritiesConverter(converter);
    return jwtConverter;
}
```

Resumen corto:

- ✗ No es una clase que tú crees
- ✓ Es una clase de Spring Security
- 📦 Te falta el `spring-boot-starter-oauth2-resource-server`
- 🔄 La añades, recargas el proyecto, y aparece automáticamente

You said:



Tengo esa configuracion para el logout pero cuando le doi a cerrar sesion no pasa nada, parece que carga algo pero sigo en el menu, y no me hecha

ChatGPT said:

Stopped thinking

You said:



Tengo esa configuracion para el logout pero cuando le doi a cerrar sesion no pasa nada, parece que carga algo pero sigo en el menu, y no me echa

ChatGPT said:

Thought for 22s

Eso que describes es **típico cuando el logout NO se está ejecutando realmente**. En Spring Security, por defecto, /logout **solo funciona por POST** (para evitar CSRF). Si tu botón “Cerrar sesión” es un `<a href="/logout">`, está haciendo **GET**, y Spring Security normalmente **no cierra sesión** (o “parece que carga” pero no invalida nada).

1) Solución recomendada: cambiar el link por un formulario POST (con CSRF)

En tu topbar cambia esto:

```
<a class="btn_ghost" th:href="@{/logout}">Cerrar sesión</a>
```

por esto:

```
<form th:action="@{/logout}" method="post" style="margin:0;">
  <input type="hidden" th:name="${_csrf.parameterName}"
th:value="${_csrf.token}">
  <button class="btn_ghost" type="submit">Cerrar sesión</button>
</form>
```

Con eso Spring Security sí ejecuta el logout, invalida sesión, borra cookies, y entonces se llamará a tu `logoutSuccessHandler`.

2) En tu SecurityConfig: NO pongas a la vez `logoutSuccessUrl` y `logoutSuccessHandler`

Si usas `logoutSuccessHandler(cognitoLogoutSuccessHandler())`, quita el `.logoutSuccessUrl("/")`, porque no tiene sentido tener ambos (y puede liar el comportamiento).

Ejemplo limpio:

```
http
    .oauth2Login(oauth -> oauth.defaultSuccessUrl("/menu", true))
    .logout(logout -> logout
        .logoutUrl("/logout")
        .logoutSuccessHandler(cognitoLogoutSuccessHandler())
        .invalidateHttpSession(true)
        .clearAuthentication(true)
        .deleteCookies("JSESSIONID")
    );
```

3) Si “sale” de tu app pero vuelve automáticamente, es porque Cognito sigue con sesión

Si solo cierras sesión en tu app pero **NO cierras sesión en Cognito**, al volver a cualquier página protegida, Cognito te “reauth” y vuelves al menú sin ver login.

Tu handler va por el camino correcto (**redirigir a Cognito /logout**), pero asegúrate de dos cosas:

(a) URL de logout correcta (Hosted UI)

Debe ser algo así:

```
https://TU_DOMINIO.auth.REGION.amazoncognito.com/logout
?client_id=XXXX
&logout_uri=http%3A%2F%2Flocalhost%3A8080%2F
```

(b) logout_uri permitido en Cognito

En la consola:

App integration → App client settings / Hosted UI

- “Allowed sign-out URLs” debe incluir:
 - <http://localhost:8080/>
 - (y los que uses en producción)

Si no está permitido, Cognito no cierra bien y te devuelve “como si nada”.

4) Comprobación rápida (para confirmar el fallo)

Cuando pulses “Cerrar sesión” mira en el Network del navegador:

- Si ves **GET /logout** → no va a cerrar (por defecto).
- Debe ser **POST /logout** y normalmente responde con un **302** hacia Cognito.

Alternativa “rápida” (no recomendada): permitir logout por GET

Solo si quieres “que funcione ya”, puedes permitir GET:

```
.logout(logout -> logout
  .logoutRequestMatcher(new AntPathRequestMatcher("/logout", "GET"))
  .logoutSuccessHandler(cognitoLogoutSuccessHandler())
)
```

Pero lo correcto es **POST + CSRF** (la opción 1).

Si me pegas tu fragmento exacto del topbar (donde está el botón) y tu `authorizeHttpRequests( ... )`, te lo dejo perfecto y además verificamos que `/menu` esté protegido (para que después de logout no se pueda entrar).

You said:

Exception evaluating SpringEL expression: "`_csrf.parameterName`" (template: "fragments/layout" - line 22, col 38)Exception evaluating SpringEL expression: "`_csrf.parameterName`" (template: "fragments/layout" - line 22, col 38)

ChatGPT said:

Ese error significa que Thymeleaf no encuentra el objeto `_csrf` en el modelo. Spring solo lo añade automáticamente cuando:

1. CSRF está activado (por defecto lo está).
2. Estás usando integración web normal (no REST puro).
3. Tienes Thymeleaf + Spring Security bien integrados.

Lo más limpio es usar la integración oficial de Thymeleaf con Spring Security.

Añade esta dependencia:

```
<dependency>
  <groupId>org.thymeleaf.extras</groupId>
  <artifactId>thymeleaf-extras-springsecurity6</artifactId>
</dependency>
```

(Reinicia el proyecto después)

Y en tu layout (o donde esté el formulario de logout) usa esto:

```
<form th:action="@{/logout}" method="post" style="margin:0;">
  <input type="hidden" th:name="${_csrf.parameterName}"
th:value="${_csrf.token}" />
  <button class="btn ghost" type="submit">Cerrar sesión</button>
</form>
```

Con eso `_csrf` ya existe y no da error.

Si aun así sigue fallando, es porque tu app está funcionando como API y no como MVC completo. En ese caso puedes forzar el token así en el `<head>` de tu layout:

```
<meta name="_csrf" th:content="${_csrf.token}"/>
<meta name="_csrf_header" th:content="${_csrf.headerName}"/>
```

Y el formulario:

```
<form th:action="@{/logout}" method="post" style="margin:0;">
    <input type="hidden" name="_csrf" th:value="${_csrf.token}">
    <button class="btn ghost" type="submit">Cerrar sesión</button>
</form>
```

Pero ojo: si `_csrf` sigue sin existir incluso así, casi seguro tienes algo como esto en tu `SecurityConfig`:

```
http.csrf(csrf -> csrf.disable());
```

Si está eso, elimínalo.

Con CSRF desactivado:

- No existe `_csrf`
- Spring no acepta POST /logout correctamente
- El logout con OAuth2 + Cognito suele romperse

Resumen rápido:

1. Añade:

```
thymeleaf-extras-springsecurity6
```

2. Usa POST para logout con token CSRF
3. Asegúrate de no tener `csrf.disable()`

Después de eso, tu botón de “Cerrar sesión”:

- Invalidará sesión local
- Borrará cookies
- Redirigirá a Cognito
- Cerrará sesión real
- Y al volver no te dejará entrar sin loguearte otra vez

You said:

para que sirve csrf

ChatGPT said:

CSRF significa **Cross-Site Request Forgery** (falsificación de petición en sitios cruzados).

Es un ataque donde un usuario, sin darse cuenta, ejecuta una acción en tu aplicación porque ya está autenticado en ella.

Ejemplo sencillo:

1. Tú estás logueado en tu aplicación.

2. Abres otra página maliciosa en otra pestaña.
3. Esa página ejecuta automáticamente algo como:

```
<form action="https://tu-app.com/logout" method="post">
  <input type="submit">
</form>
<script>
  document.forms[0].submit();
</script>
```

Como tu navegador:

- ya tiene la cookie de sesión
- ya está autenticado

Spring recibe la petición y piensa:

“Esto viene de un usuario válido”

Y ejecuta la acción sin que tú lo sepas.

Eso puede ser grave si la acción es:

- borrar datos
- cambiar contraseña
- transferir dinero
- reservar libros
- dar de baja usuarios

CSRF evita eso usando un **token secreto por sesión**:

1. Cuando se carga una página, Spring genera un token:

```
_csrf = "a9f2d83e..."
```

2. Ese token se mete en el formulario:

```
<input type="hidden" name="_csrf" value="a9f2d83e...">
```

3. Cuando el formulario se envía:

- Spring comprueba que el token coincide con el de la sesión
- Si no coincide → la petición se rechaza (403)

Una página externa no puede conocer tu token, así que no puede falsificar la petición.

Por eso:

- Logout en Spring es por POST
- Con token CSRF obligatorio

- Para que nadie pueda “desloguearte” desde otra web

En tu proyecto:

Acción	¿Debe llevar CSRF?
Login OAuth2	No (lo maneja Cognito)
Logout	Sí
Reservar libro	Sí
Renovar préstamo	Sí
Cancelar reserva	Sí
APIs públicas GET	No

Regla práctica:

- Si una acción **cambia estado** → necesita CSRF
- Si solo **consulta datos** (GET) → no necesita CSRF

Por eso antes, cuando usabas:

```
<a href="/logout">
```

No funcionaba bien:

- Era GET
- No tenía token
- Spring no lo considera logout válido

Y por eso ahora:

```
<form method="post" action="/logout">
  <input type="hidden" name="_csrf" value="...">
</form>
```

sí funciona y es seguro.

CSRF es básicamente:

“La prueba de que esta petición la hizo *esta página* y no otra web externa.”

You said:

ahora no me da error. Pero no me cierra la sesion, si estoy en otra agina me devuelve a /menu pero no me manda a logearme ni nada

ChatGPT said:

Thought for 19s

Vale — si ya no te da error de CSRF pero **después de “Cerrar sesión” sigues entrando y te manda a /menu**, casi siempre es por **una de estas dos causas (o ambas)**:

1. **Tu ruta / (home) siempre redirige a /menu**, estés logueado o no.
2. **No estás cerrando sesión en Cognito**, así que al volver a cualquier URL protegida Spring te re-autentica automáticamente (SSO “silent login”) y acabas otra vez en /menu.

Vamos a dejarlo bien.

1) Arregla el “root” (/) para que NO redirija siempre a /menu

Si tienes algo como:

```
@GetMapping("/")
public String home() { return "redirect:/menu"; }
```

eso hace que **aunque estés deslogueado** y vayas a /, te mande a /menu.

Cámbialo a esto:

```
@Controller
public class HomeController {

    @GetMapping("/")
    public String root(Authentication auth) {
        if (auth == null || !auth.isAuthenticated()
            || auth instanceof AnonymousAuthenticationToken) {
            return "redirect:/login";
        }
        return "redirect:/menu";
    }
}
```

Imports:

```
import org.springframework.security.core.Authentication;
import org.springframework.security.authentication.AnonymousAuthenticationToken;
```

Y asegúrate de tener `@GetMapping("/login")` que renderice tu `login.html`.

2) Asegura que /menu esté protegido (NO permitAll)

En tu `SecurityFilterChain`, esto es clave:

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/", "/login", "/css/**", "/js/**", "/img/**",
```



```
"/error").permitAll()
    .requestMatchers("/menu", "/catalog/**", "/my/**",
"/librarian/**").authenticated()
    .anyRequest().authenticated()
)
```

Si por error tienes /menu en permitAll(), podrás entrar aunque no haya sesión.

3) Logout: no uses logoutSuccessUrl("/") si / redirige a /menu

En tu captura tienes .logoutSuccessUrl("/").

Si / te manda a /menu, eso explica perfecto el “me devuelve a /menu”.

Si quieres logout local (sin Cognito), usa:

```
.logout(logout -> logout
    .logoutUrl("/logout")
    .logoutSuccessUrl("/login")
    .invalidateHttpSession(true)
    .clearAuthentication(true)
    .deleteCookies("JSESSIONID")
)
```

Pero en tu caso (Cognito) lo correcto es **cerrar también Cognito**.

4) Logout real con Cognito (Hosted UI)

Tu logoutSuccessHandler está bien como idea, pero comprueba estas 2 cosas:

(a) Allowed sign-out URLs en Cognito

En tu App client / Hosted UI, añade EXACTAMENTE:

- <http://localhost:8080/login> (o / si quieres)
- <http://localhost:8080/>

Si no está, Cognito no hará sign-out bien.

(b) Usa una URL de vuelta que no te meta otra vez a /menu

En tu handler, pon logoutRedirect a /login, no /:

```
String logoutRedirect = "http://localhost:8080/login";
```

5) Verificación rápida (lo más importante)

Abre DevTools → Network y pulsa “Cerrar sesión”.

Debes ver:

- **POST /logout** → responde **302**
- luego navegación a <https://<tu-dominio>.auth.../logout?...>

Si NO ves eso, el logout no está pasando por Spring Security.

Config final recomendada (sin mezclar successUrl y handler)

```
http
    .oauth2Login(oauth -> oauth.defaultSuccessUrl("/menu", true))
    .logout(logout -> logout
        .logoutUrl("/logout")
        .logoutSuccessHandler(cognitoLogoutSuccessHandler())
        .invalidateHttpSession(true)
        .clearAuthentication(true)
        .deleteCookies("JSESSIONID")
    );
```

Y en Thymeleaf (ya lo tienes):

```
<form th:action="@{/logout}" method="post" style="margin:0;">
    <input type="hidden" th:name="${_csrf.parameterName}"
th:value="${_csrf.token}">
    <button class="btn ghost" type="submit">Cerrar sesión</button>
</form>
```

Si me pegas tu bloque completo de `authorizeHttpRequests(...)` y tu `HomeController` (la ruta /), te digo exactamente cuál de las 2 causas es la que te está pasando y te lo dejo “cerrado” sin bucles.

You said:



al llamar a logout tiene estos valores, están bien?

ChatGPT said:

No, **no están bien**: en tu debug se ve que `logoutRedirect` y `cognitoDomain` valen literalmente los **nombres de las propiedades**:

- `logoutRedirect` = `"library.cognito.logout-redirect"`
- `cognitoDomain` = `"library.cognito.domain"`

Eso pasa cuando esas variables **no se están inyectando** desde `application.properties/yml` (o las estás asignando como strings a mano). Entonces tu handler construye una URL inválida y el navegador no llega a Cognito → por eso “parece que carga” pero sigues logueado.

Cómo tiene que verse (valores reales)

Debería ser algo así:

- `cognitoDomain` = "<https://tu-dominio.auth.eu-west-1.amazoncognito.com>"
- `logoutRedirect` = "<http://localhost:8080/login>" (o /)

Y `clientId` debe ser el **id real** del App Client, no el nombre de la property.

Arreglo correcto (inyección de properties)

Opción A: @Value (rápida)

```
@Value("${library.cognito.domain}")
private String cognitoDomain;

@Value("${library.cognito.logout-redirect}")
private String logoutRedirect;

@Value("${spring.security.oauth2.client.registration.cognito.client-id}")
private String clientId;
```

Y en `application.properties`:

```
library.cognito.domain=https://TU_DOMINIO.auth.TU_REGION.amazoncognito.com
library.cognito.logout-redirect=http://localhost:8080/login
```

Ojo: el `cognito.domain` **NO** es “el pool id”, es el dominio del Hosted UI (el de ...auth...amazoncognito.com o tu custom domain).

Opción B: @ConfigurationProperties (mejor práctica)

```
@ConfigurationProperties(prefix = "library.cognito")
public record CognitoProps(String domain, String logoutRedirect) {}
```

Registrar:

```
@EnableConfigurationProperties(CognitoProps.class)
@Configuration
public class SecurityConfig { ... }
```

Y lo usas en el handler.

Además: URL de logout correcta

Cognito espera:

`{domain}/logout?client_id=...&logout_uri=...`

Tu construcción está bien *si* los valores están bien.

Ejemplo final:

```
@Bean
LogoutSuccessHandler cognitoLogoutSuccessHandler(
    @Value("${library.cognito.domain}") String cognitoDomain,
    @Value("${library.cognito.logout-redirect}") String logoutRedirect,
    @Value("${spring.security.oauth2.client.registration.cognito.client-id}")
String clientId
) {
    return (request, response, authentication) -> {
        String encoded = URLEncoder.encode(logoutRedirect, StandardCharsets.UTF_8);
        String url = cognitoDomain + "/logout?client_id=" + clientId + "&logout_uri="
+ encoded;
        response.sendRedirect(url);
    };
}
```

Y MUY importante: Allowed sign-out URLs en Cognito

En la consola de Cognito (Hosted UI / App client), añade exactamente el logout redirect:

- <http://localhost:8080/login>

Si no, Cognito ignora/deniega el sign-out.

Resumen

- ✓ Tu lógica está bien.
- ✗ Tus variables NO están inyectadas (tienen el nombre de la property, no el valor).
- ✓ Arregla @Value / ConfigurationProperties, y añade la URL a “Allowed sign-out URLs”.

Si me pegas tu `application.properties` (solo esas keys, sin secrets) te digo exactamente qué tienes mal escrito (nombres, guiones, prefijos, etc.).

You said:



porque no se están leyendo esos valores?

ChatGPT said:

En tu captura se ve la causa casi seguro:

✓ **Tu AwsSecurityConfig está condicionado por @Profile("aws")**

```
@Profile("aws")  
public class AwsSecurityConfig { ... }
```

Eso significa que **esta clase (y sus @Value)** solo se cargan si tu aplicación está arrancada con el perfil **aws** activo.

Si no tienes activo el perfil aws, Spring **ni crea ese bean**, por tanto:

- no se inyecta `clientId`
- no se inyecta `cognitoDomain`
- no se inyecta `logoutRedirect`

y acabas con strings “raros” o valores nulos/por defecto.

1) Activa el profile aws (opción rápida)

application.properties

```
spring.profiles.active=aws
```

o al arrancar:

Maven

```
mvn spring-boot:run -Dspring-boot.run.profiles=aws
```

Gradle

```
./gradlew bootRun --args='--spring.profiles.active=aws'
```

Jar

```
java -jar app.jar --spring.profiles.active=aws
```

2) Mejor práctica: usa perfiles por fichero de config

Crea `application-aws.properties` y mete ahí:

```
library.cognito.domain=...
library.cognito.logout-redirect=...
spring.security.oauth2.client.registration.cognito.client-id=...
```

Y arrancas con `--spring.profiles.active=aws`.

3) Si quieres que funcione SIEMPRE (local y aws)

Entonces **quita el `@Profile("aws")`** de la clase de seguridad, o separa:

- Seguridad común (sin profile)
- Props específicas AWS en `application-aws.properties`

Ejemplo recomendado:

- `SecurityConfig` (sin profile) → siempre se carga
- `application-local.properties` y `application-aws.properties` → cambian los valores

4) Otro detalle importante: en `@Value` te falta `${...}`

En la captura estás usando:

```
@Value("library.cognito.domain")
```

Eso NO lee properties. Eso mete literalmente el string `"library.cognito.domain"`.

Debe ser:

```
@Value("${library.cognito.domain}")
private String cognitoDomain;

@Value("${library.cognito.logout-redirect}")
private String logoutRedirect;

@Value("${spring.security.oauth2.client.registration.cognito.client-id}")
private String clientId;
```

🕒 Este punto por sí solo explica EXACTAMENTE tu debug anterior (valores literales).

Resumen (lo que tienes mal)

Tienes **2 problemas**:

1. `@Profile("aws")` hace que la clase no se cargue si no activas el perfil.
2. Tus `@Value` están mal escritos: falta `${...}`.

Arregla así:

```
@Profile("aws") // opcional, solo si quieres
public class AwsSecurityConfig {

    @Value("${spring.security.oauth2.client.registration.cognito.client-id}")
    private String clientId;

    @Value("${library.cognito.domain}")
    private String cognitoDomain;

    @Value("${library.cognito.logout-redirect}")
    private String logoutRedirect;
}
```

Con eso ya verás en debug valores reales, y el logout te redirigirá a Cognito correctamente.

You said:



ahora me ha echado, pero quiero que me mande al login, no a esa pagina de error

ChatGPT said:

Bien: ahora **sí está llamando al logout de Cognito**, pero te manda a “Invalid request” porque la URL que estás construyendo es incorrecta.

En tu captura estás yendo a:

```
.../login?client_id=...&logout_uri=http://localhost:8080/
```

Eso está mal por dos motivos:

1. Estás llamando a /login en Cognito con parámetro `logout_uri` (ese parámetro es para /logout, no para /login).
2. El parámetro correcto suele llamarse `logout_uri` (para logout) y **debe estar permitido** en “Allowed sign-out URLs”.

✓ Lo que quieres

Al cerrar sesión:

1. Spring hace POST /logout
2. Spring redirige a **Cognito** /logout
3. Cognito cierra sesión y vuelve a tu app en /login

1) Corrige tu LogoutSuccessHandler

Asegúrate de usar /logout y logout_uri:

```
@Bean
LogoutSuccessHandler cognitoLogoutSuccessHandler(
    @Value("${library.cognito.domain}") String cognitoDomain,
    @Value("${library.cognito.logout-redirect}") String logoutRedirect,
    @Value("${spring.security.oauth2.client.registration.cognito.client-id}")
String clientId
) {
    return (request, response, authentication) -> {
        String encoded = URLEncoder.encode(logoutRedirect, StandardCharsets.UTF_8);
        String url = cognitoDomain
            + "/logout?client_id=" + clientId
            + "&logout_uri=" + encoded;
        response.sendRedirect(url);
    };
}
```

Y en properties:

```
library.cognito.domain=https://us-east-1snnleupih.auth.us-east-1.amazonaws.com
library.cognito.logout-redirect=http://localhost:8080/login
```

2) Configura Cognito: Allowed sign-out URLs

En AWS Cognito → User pool → App integration → Hosted UI / App client:

En **Allowed sign-out URLs** añade EXACTAMENTE:

- <http://localhost:8080/login>
(la misma que estás usando en logout-redirect)

Si no coincide exacto, Cognito devuelve error.

3) Asegura que /login exista en tu app

Controlador MVC:

```
@Controller
public class LoginController {
    @GetMapping("/login")
```



```
public String login() {  
    return "login";  
}  
}
```

Y en security:

```
.authorizeHttpRequests(auth -> auth  
    .requestMatchers("/", "/login", "/css/**", "/js/**", "/error").permitAll()  
    .anyRequest().authenticated()  
)  
.oauth2Login(oauth -> oauth.loginPage("/login"))
```

Resultado final

- Le das a “Cerrar sesión”
- Cognito cierra sesión
- Te devuelve a <http://localhost:8080/login>
- Ves tu login.html (no la página de error)

Si después de esto sigue saliendo “Invalid request”, casi siempre es porque **la URL no está en Allowed sign-out URLs** o porque el dominio `library.cognito.domain` no incluye el <https://...auth....>

You said:

quiero que me haga redirect al login de cognito

ChatGPT said:

Ahora mismo tu flujo es:

Spring /logout

→ rediriges a Cognito /logout

→ Cognito intenta volver a tu app

→ tú quieres que en vez de volver a tu app vaya directamente al login de Cognito.

Eso se puede hacer. Simplemente el `logout_uri` no debe apuntar a tu aplicación, sino al endpoint de login de Cognito.

El login de Cognito es:

https://TU_DOMINIO.auth.REGION.amazoncognito.com/login

pero necesita parámetros:

```
/login
?client_id=XXX
&response_type=code
&scope=openid+email+profile
&redirect_uri=TU_REDIRECT_URI_DE_TU_APP
```

Por ejemplo:

```
https://us-east-1snnleupih.auth.us-east-1.amazonaws.com/login
?client_id=1h2dcs34d8ucv154k44joac1bc
&response_type=code
&scope=openid+email+profile
&redirect_uri=http://localhost:8080/login/oauth2/code/cognito
```

Ese `redirect_uri` es el que ya tienes configurado en Spring para OAuth2.

Entonces tu logout queda así:

1. Spring cierra sesión local.
2. Redirige a Cognito /logout.
3. Cognito cierra su sesión.
4. Cognito redirige directamente a su propio /login.
5. Aparece la pantalla de login de Cognito.

Tu `LogoutSuccessHandler` quedaría así:

```
@Bean
LogoutSuccessHandler cognitoLogoutSuccessHandler(
    @Value("${library.cognito.domain}") String cognitoDomain,
    @Value("${spring.security.oauth2.client.registration.cognito.client-id}")
String clientId,
    @Value("${spring.security.oauth2.client.registration.cognito.redirect-uri}")
String appRedirectUri
) {
    return (request, response, authentication) -> {

        String cognitoLoginUrl =
            cognitoDomain + "/login"
            + "?client_id=" + clientId
            + "&response_type=code"
            + "&scope=openid+email+profile"
            + "&redirect_uri=" + URLEncoder.encode(appRedirectUri,
StandardCharsets.UTF_8);
```

```

        String logoutUrl =
            cognitoDomain + "/logout"
            + "?client_id=" + clientId
            + "&logout_uri=" + URLEncoder.encode(cognitoLoginUrl,
StandardCharsets.UTF_8);

        response.sendRedirect(logoutUrl);
    };
}

```

Y en tu `application.properties` asegúrate de tener:

```

library.cognito.domain=https://us-east-1snnleupih.auth.us-east-1.amazoncognito.com
spring.security.oauth2.client.registration.cognito.client-id=TU_CLIENT_ID
spring.security.oauth2.client.registration.cognito.redirect-
uri=http://localhost:8080/login/oauth2/code/cognito

```

Y en Cognito (muy importante):

En tu App Client → Hosted UI → Allowed sign-out URLs añade:

<https://us-east-1snnleupih.auth.us-east-1.amazoncognito.com/login>

Sí, el propio login de Cognito debe estar permitido como destino tras logout.

Con esto el comportamiento será exactamente el que quieres:

Botón “Cerrar sesión”

→ se cierra sesión en Spring

→ se cierra sesión en Cognito

→ aparece automáticamente la pantalla de login de Cognito

→ el usuario vuelve a autenticarse desde ahí.

You said:

cuando yo entro en mi aplicacion directamente se abre el login de cognito. Quiero que al cerrar sesion pase lo mismo

ChatGPT said:

Perfecto: entonces al hacer logout quieres volver a “estado inicial” de tu app, que al entrar en / te redirige a Cognito automáticamente.

La forma más robusta es:

1. Logout en tu app (borra sesión/cookies)
 2. Logout en Cognito (Hosted UI /logout)
 3. Cognito vuelve a tu app en / (o /login)
 4. Spring Security, al no haber sesión, dispara de nuevo el login → Cognito login
- ✓ Eso hace exactamente “lo mismo que cuando entro en la app”.

1) Configura el LogoutSuccessHandler para volver a tu app (NO al login de Cognito)

No redirijas al /login de Cognito. Redirige a tu app en / (o /login local).

```
@Bean
LogoutSuccessHandler cognitoLogoutSuccessHandler(
    @Value("${library.cognito.domain}") String cognitoDomain,
    @Value("${library.cognito.logout-redirect}") String logoutRedirect,
    @Value("${spring.security.oauth2.client.registration.cognito.client-id}")
String clientId
) {
    return (request, response, authentication) -> {
        String encoded = URLEncoder.encode(logoutRedirect, StandardCharsets.UTF_8);
        String url = cognitoDomain + "/logout?client_id=" + clientId + "&logout_uri="
+ encoded;
        response.sendRedirect(url);
    };
}
```

application.properties

```
library.cognito.domain=https://us-east-1snnleupih.auth.us-east-1.amazonaws.com
library.cognito.logout-redirect=http://localhost:8080/
```

2) En Cognito añade esa URL en “Allowed sign-out URLs”

En AWS Cognito → Hosted UI → App client:

Allowed sign-out URLs incluye:

- <http://localhost:8080/>
(Exacto, con barra final)

3) Asegura que / esté protegido (o que redirija correctamente)

Lo más simple: haz que / sea una página protegida (requiere auth).

Así, cuando Cognito te devuelva a /, Spring verá que no hay sesión y te mandará a login de Cognito.

En tu security:

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/css/**", "/js/**", "/img/**", "/error").permitAll()
    .anyRequest().authenticated()
)
.oauth2Login(oauth -> oauth.defaultSuccessUrl("/menu", true))
```

✓ Nota: aquí **NO** pongas "/" en permitAll() si quieres que entrar a / dispare login.

4) Logout config (no mezcles successUrl y successHandler)

Deja solo el handler:

```
.logout(logout -> logout
    .logoutUrl("/logout")
    .logoutSuccessHandler(cognitoLogoutSuccessHandler())
    .invalidateHttpSession(true)
    .clearAuthentication(true)
    .deleteCookies("JSESSIONID")
)
```

Y tu botón logout debe ser POST con CSRF (como ya tienes).

Qué pasará con esto

- Estás en /menu
- Pulsas “Cerrar sesión”
- Se cierra sesión local
- Se cierra sesión en Cognito
- Cognito te devuelve a <http://localhost:8080/>
- Spring detecta que / requiere autenticación → te redirige automáticamente al login de Cognito

Es exactamente el mismo comportamiento que “entrar a la app”.

Si aun así te deja entrar sin loguearte

Entonces / o /menu están en permitAll() o tienes una redirección manual que te manda a /menu aunque estés anónimo. En ese caso, quita el redirect fijo en tu @GetMapping("/").

Si me pegas tu bloque authorizeHttpRequests(...) completo, te señalo exactamente qué matcher está dejando pasar.

You said:

como cargar datos de usuario autorizado automaticamente al modelo

ChatGPT said:

En Spring MVC + Thymeleaf puedes cargar datos del usuario autenticado al Model **automáticamente** de 3 formas (la más limpia suele ser la #1).

1) @ControllerAdvice + @ModelAttribute (recomendado)

Esto mete atributos en el modelo **para todas las vistas** sin repetir código.

@ControllerAdvice

```
public class GlobalModelAttributes {

    @ModelAttribute("userName")
    public String userName(@AuthenticationPrincipal OidcUser user) {
        if (user == null) return null;
        // Cognito suele tener "email" y a veces "name"
        String name = user.getFullName();
        if (name == null || name.isBlank()) name = user.getEmail();
        return name;
    }

    @ModelAttribute("role")
    public String role(@AuthenticationPrincipal OidcUser user) {
        if (user == null) return null;
        // Grupos Cognito llegan como claim "cognito:groups"
        Object groups = user.getClaims().get("cognito:groups");
        if (groups instanceof java.util.List<?> list && !list.isEmpty()) {
            return String.valueOf(list.get(0));
        }
        return "USER";
    }
}
```

```

@ModelAttribute("isLibrarian")
public boolean isLibrarian(Authentication auth) {
    if (auth == null) return false;
    return auth.getAuthorities().stream()
        .anyMatch(a -> a.getAuthority().equals("ROLE_LIBRARIAN"));
}
}

```

En Thymeleaf ya puedes usar en cualquier página:

```

<span th:text="${userName}">Usuario</span>
<span th:text="${role}">Rol</span>

<div th:if="${isLibrarian}">
    Opciones bibliotecario
</div>

```

✓ Ventaja: lo tienes en todas las vistas sin repetir.

2) Meter el Principal/Authentication en cada controller (rápido, pero repetitivo)

En un controlador:

```

@GetMapping("/menu")
public String menu(Model model, Authentication auth) {
    model.addAttribute("userName", auth.getName());
    return "menu";
}

```

✓ útil si solo lo necesitas en 1-2 pantallas.

3) Acceder al usuario directamente desde Thymeleaf (sin Model)

Si añades la dependencia:

```

<dependency>
    <groupId>org.thymeleaf.extras</groupId>
    <artifactId>thymeleaf-extras-springsecurity6</artifactId>
</dependency>

```

puedes hacer:

```

<span sec:authentication="name"></span>

```

y condicional por rol:

```
<div sec:authorize="hasRole('LIBRARIAN')">
  Opciones bibliotecario
</div>
```

✓ Ventaja: cero Java para atributos simples.

⚠ Inconveniente: a veces quieres un “userName” bonito (email vs name) y ahí #1 es mejor.

¿Cuál usar con Cognito?

- Si estás con oauth2Login (OIDC): usa `@AuthenticationPrincipal OidcUser`
- Si estás con JWT resource server puro: sería `Jwt` en lugar de `OidcUser`

Para tu caso (login Cognito Hosted UI), normalmente es **OIDC** → `OidcUser`.

Si me dices si en tu controller el principal llega como `OidcUser` o como `OAuth2User` (depende de config), te adapto el código exacto para tus claims reales (email, name, preferred_username, cognito:groups).

You said:



tengo este codigo, y quiero unirlo con el front. en el front tengo este form. <form class="row" th:action="@{/catalog}" method="get" style="gap:12px;" data-catalog-search>
<input class="input" name="q" placeholder="Buscar..." th:value="{q}" style="flex:1;min-width:220px;" data-q> <button class="btn primary" type="submit">Buscar</button>
</form>

ChatGPT said:

Perfecto: ahora mismo tienes **dos mundos**:

- Tu **frontend** (Thymeleaf) hace GET `/catalog?q=...`
- Tu **backend** (en la captura) devuelve `List<BookSummaryDto>` (JSON) en algún endpoint (seguramente `@RestController`), y además tu método recibe query (no q)

Para “unirlo” tienes 2 opciones correctas. Te dejo ambas, pero la más cómoda con tu HTML actual es la **A** (misma ruta `/catalog` devuelve HTML) + un endpoint `/api/catalog` para JS.

Opción A (recomendada): /catalog devuelve HTML y /api/catalog devuelve JSON

1) Controlador de vistas (HTML)

```
@Controller
public class CatalogPageController {

    @GetMapping("/catalog")
    public String catalogPage(@RequestParam(required = false, name = "q") String q,
        Model model) {
        model.addAttribute("q", q);
        model.addAttribute("books", (q == null || q.isBlank()) ? List.of() :
            catalogService.search(q)
                .stream().map(BookSummaryDto::from).toList());
        return "catalog"; // templates/catalog.html
    }
}
```

✓ Con esto, cuando envías el form, renderiza la página con resultados usando Thymeleaf.

2) Controlador REST (JSON) para búsqueda “live” con JS

```
@RestController
@RequestMapping("/api/catalog")
public class CatalogApiController {

    @GetMapping
    public List<BookSummaryDto> search(@RequestParam(required = false, name = "q")
        String q) {
        if (q == null || q.isBlank()) return List.of();
        return catalogService.search(q).stream()
            .map(BookSummaryDto::from)
            .toList();
    }
}
```

✓ Esto encaja con el JS que te di (que llama a /api/catalog?q=...).

Opción B (rápida): usa tu endpoint actual tal cual y adapta el front

Si tu método actual es el “/catalog” REST (devuelve JSON), entonces tu form **no debería apuntar a /catalog** porque el navegador recibirá JSON y no HTML.

En ese caso, mantén el HTML y haz que el form no navegue, sino que el JS haga fetch.

Cambios necesarios:

1. El endpoint debe aceptar q (o el front enviar query)
2. El JS debe llamar a esa ruta REST

Por ejemplo, si el backend usa query:

```
@GetMapping("/api/catalog")
public List<BookSummaryDto> search(@RequestParam(required = false) String query)
{ ... }
```

Entonces en el HTML cambia el input a:

```
<input class="input" name="query" ...>
```

Pero esto te rompe tu URL ?q=... que es más estándar. Mejor unificar a q.

Tu caso concreto (lo que veo en tu código)

Tu backend recibe String query, pero tu form manda q.

✓ Arreglo mínimo: cambia el backend a @RequestParam(name="q"):

```
@GetMapping("/api/catalog")
public List<BookSummaryDto> search(@RequestParam(required = false, name="q")
String query) {
    if (query == null || query.isBlank()) return List.of();
    return catalogService.search(query).stream()
        .map(BookSummaryDto::from)
        .toList();
}
```

(puedes llamarla q o query dentro, da igual, lo importante es name="q")

Qué necesitas también en catalog.html para que renderice resultados (server-side)

Debajo del form, asegúrate de tener:

```
<div class="list" data-results>
  <article class="item" th:each="book : ${books}">
    <div class="thumb" aria-hidden="true"></div>
    <div class="meta">
      <p class="title" th:text="${book.title}">Título</p>
      <p class="sub" th:text="${book.author}">Autor</p>
    </div>
    <div class="right">
      <a class="btn" th:href="@{/books/${book.id}|"}">Ver ejemplares →</a>
    </div>
  </article>

  <div class="card pad" th:if="${#lists.isEmpty(books)}">
    <p class="p">No hay resultados.</p>
  </div>
</div>
```

Resumen

- Si quieres que el form funcione “normal” (sin JS): /catalog **debe devolver HTML**.
- Si quieres búsqueda live con JS: añade /api/catalog que devuelva JSON.
- Unifica el parámetro: tu form envía q, tu endpoint debe leer q (no query).

Si me dices qué ruta exacta tiene tu @GetMapping del método search (/catalog o /api/catalog?), te dejo el código final exacto sin ambigüedad.

You said:

no necesito tener dos endpoints uno para front y otro para back

ChatGPT said:

Perfecto: entonces tienes que elegir **una sola cosa** para ese endpoint:

- **O devuelve HTML** (vista Thymeleaf) → el form funciona tal cual.
- **O devuelve JSON** (API REST) → el form NO debe navegar; el JS hace fetch y tú pintas resultados.

Como tú quieres **un solo endpoint**, la opción más práctica es: **JSON + JS** (endpoint único) y el HTML es “estático” (solo plantilla), sin render server-side.

Te lo dejo cerrado así:

1) Backend: un solo endpoint JSON (GET /catalog)

Tu método debe estar en un `@RestController` y aceptar el parámetro `q` (tu form envía `q`):

```
@RestController
@RequestMapping("/catalog")
public class CatalogController {

    @GetMapping
    public List<BookSummaryDto> search(@RequestParam(required = false, name = "q")
String q) {
        if (q == null || q.isBlank()) return List.of();

        return catalogService.search(q).stream()
            .map(BookSummaryDto::from)
            .toList();
    }
}
```

✓ Importante: `name="q"` para que coincida con el input del form.

2) Front: el form NO debe recargar la página (lo intercepta JS)

Tu form lo puedes dejar igual, solo añade un contenedor donde pintar resultados:

```
<form class="row" th:action="@{/catalog}" method="get" style="gap:12px;"
    data-catalog-search>
    <input class="input" name="q" placeholder="Buscar..." style="flex:1;min-
width:220px;" data-q>
    <button class="btn primary" type="submit">Buscar</button>
</form>

<div class="divider"></div>

<div class="list" data-results>
    <div class="card pad">
        <p class="p">Escribe para buscar en el catálogo.</p>
    </div>
</div>
```

3) JS: intercepta submit y hace fetch a /catalog?q=...

En tu app.js añade/usa esto:

```
(function initCatalogSearch() {
  const form = document.querySelector("[data-catalog-search]");
  if (!form) return;

  const input = form.querySelector("[data-q]");
  const results = document.querySelector("[data-results]");
  if (!input || !results) return;

  const render = (books) => {
    results.innerHTML = "";
    if (!books.length) {
      results.innerHTML = `<div class="card pad"><p class="p">No hay resultados.</p></div>`;
      return;
    }

    for (const b of books) {
      const el = document.createElement("article");
      el.className = "item";
      el.innerHTML = `
        <div class="thumb" aria-hidden="true"></div>
        <div class="meta">
          <p class="title"></p>
          <p class="sub"></p>
        </div>
        <div class="right">
          <a class="btn">Ver ejemplares -></a>
        </div>
      `;
      el.querySelector(".title").textContent = b.title ?? "(Sin título)";
      el.querySelector(".sub").textContent = b.author ?? "";
      el.querySelector("a").href = `/books/${b.id}`;
      results.appendChild(el);
    }
  };

  async function doSearch(q) {
    const res = await fetch(`/catalog?q=${encodeURIComponent(q)}`, {
```

```

        headers: { "Accept": "application/json" }
    });
    if (!res.ok) throw new Error("Error buscando en catálogo");
    return res.json();
}

form.addEventListener("submit", async (ev) => {
    ev.preventDefault(); // <-- evita recarga
    const q = input.value.trim();
    if (!q) return render([]);

    try {
        const data = await doSearch(q);
        render(Array.isArray(data) ? data : []);
    } catch (e) {
        results.innerHTML = `<div class="card pad"><p
class="p">${e.message}</p></div>`;
    }
});

// opcional: búsqueda en vivo mientras escribes
let t;
input.addEventListener("input", () => {
    clearTimeout(t);
    t = setTimeout(() => form.requestSubmit(), 250);
});
})();

```

4) Muy importante: NO puedes tener al mismo tiempo HTML y JSON en el mismo endpoint

Con un solo endpoint /catalog:

- si el método devuelve `List<BookSummaryDto>` → siempre será JSON
- si quieres HTML, el método debe devolver `String` (nombre de vista)

No puedes devolver ambos sin meter lógica por Accept header y eso ya sería “dos comportamientos” (y suele ser más lioso).

Qué te recomiendo para tu caso

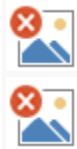
Si tu app es Thymeleaf “clásica”, lo más limpio suele ser HTML en `/catalog`.

Pero como has dicho que no quieres dos endpoints, entonces:

- ✓ `/catalog = JSON`
- ✓ el front renderiza con JS
- ✓ y tu página `catalog.html` solo contiene el esqueleto + el form.

Si me confirmas si `/catalog` ya lo estás usando para servir la página `catalog.html` o no, te digo cómo organizarlo sin duplicar rutas (porque si `/catalog` devuelve JSON, la página HTML tendrá que estar en otra ruta tipo `/catalog-page` o `/menu` que la enlace).

You said:



tengo este cacho en un fichero `catalog.html` la clase donde esta el controller tiene `@RequestMapping("/catalog")` he modificado el controller. Pero ahora cuando intento abrir la pagina `/catalog` me da error 404

ChatGPT said:

El 404 te está pasando por una combinación típica de dos cosas que se ven en tus capturas:

1. Tu clase tiene `@RequestMapping("/catalog")` y **además** tu método tiene `@GetMapping("/")`
→ eso hace que la ruta real sea `/catalog/` (con barra final).
En muchos casos `/catalog` y `/catalog/` se comportan distinto (depende de tu config), y te puede dar 404 al entrar sin la barra.
2. Muy probablemente la clase está anotada como `@RestController` (o tienes `@ResponseBody`), entonces "catalog" se devuelve como texto y no como vista (aunque en tu caso dices que es 404, pero igual conviene asegurar esto).

✓ Arreglo directo (el más simple)

Cambia el mapping del método a vacío en vez de `"/"`:

```
@Controller
@RequestMapping("/catalog")
public class CatalogController {
```

```

@GetMapping    // <-- NO "/"
public String search(@RequestParam(required = false, name = "q") String q,
Model model) {
    model.addAttribute("q", q);
    model.addAttribute("books",
        (q == null || q.isBlank())
            ? List.of()
            :
catalogService.search(q).stream().map(BookSummaryDto::from).toList()
    );
    return "catalog";
}
}

```

Con esto, la ruta es exactamente:

- GET /catalog ☒

y ya no depende de la barra final.

Revisa también estas 3 cosas (muy importantes)

A) Que sea @Controller, no @RestController

Si fuese @RestController, no renderiza catalog.html.

☒ Debe ser:

```
@Controller
```

B) Que el archivo esté en el sitio correcto

Debe estar en:

```
src/main/resources/templates/catalog.html
```

No en static/ ni en otra carpeta.

C) Que Security no lo esté bloqueando (y te lo “disfrazca”)

Si tienes Spring Security y no estás logueado, normalmente no es 404 sino redirect, pero revisa que /catalog esté permitido o autenticado según tu caso.

Tu HTML está bien, solo un detalle opcional

Tu form tiene:

```
<form th:action="@{/catalog}" method="get">
```

Perfecto: eso llama a GET /catalog?q=... que encaja con el controlador.

Checklist rápido para que /catalog abra sí o sí

1. CatalogController anotado con @Controller
2. @RequestMapping("/catalog") en clase
3. método con @GetMapping (sin "/")
4. templates/catalog.html existe
5. spring-boot-starter-thymeleaf está en dependencias

Si después de cambiar @GetMapping("/") a @GetMapping sigue dando 404, lo siguiente que miraría es: **si ese controller se está registrando** (paquete correcto para component scan).

Tu clase principal @SpringBootApplication debe estar en un paquete “padre” que incluya a tu controller. Por ejemplo:

- com.tuapp.Application (main)
- com.tuapp.web.CatalogController

Si el controller está en un paquete “fuera”, Spring no lo detecta y da 404.

Si me dices el paquete de tu clase @SpringBootApplication y el paquete del controller, te digo al instante si es eso.

You said:



tengo esto en el html, pero al endpoint no llega bien el book id. no manda el id

ChatGPT said:

El problema es el th:href que estás usando. Esa sintaxis:

```
th:href="@{/catalog/copies/${book.id}}"
```

no construye bien el path variable; Thymeleaf no “inyecta” \${book.id} así dentro del @{...} (te queda literal o vacío según el caso).

Tienes 2 formas correctas:

✓ Opción 1 (recomendada): path variable con {id}

```
<a class="btn"
  th:href="@{/catalog/copies/{id}(id=${book.id})}">
  Ver ejemplares →
</a>
```

Esto genera:

- /catalog/copies/123

✓ Opción 2: concatenación con |...|

```
<a class="btn"
  th:href="@{|/catalog/copies/${book.id}|}">
  Ver ejemplares →
</a>
```

Esto también genera:

- /catalog/copies/123

Importante: tu controller debe mapear el id como PathVariable

Ejemplo:

```
@GetMapping("/catalog/copies/{id}")
public String copies(@PathVariable Long id, Model model) {
    ...
    return "copies";
}
```

O si tu clase ya tiene @RequestMapping("/catalog"):

```
@GetMapping("/copies/{id}")
public String copies(@PathVariable Long id, Model model) { ... }
```

Tip extra de depuración

En el navegador, pon el ratón encima del link (o inspecciona) y mira el href real.

Si ves /catalog/copies/ sin número, es que book.id está llegando null (entonces hay que revisar tu BookSummaryDto::from).

Pero en 90% de casos aquí es solo la sintaxis del th:href.

Lo que te recomiendo cambiar ya mismo

Sustituye tu línea por esta:

```
<a class="btn" th:href="@{/catalog/copies/{id}(id=${book.id})}">Ver ejemplares</a>
```

Y listo.

You said:



tengo el endpoint copies que llega bien y genera el modelo, pero luego no me abre el html que se llama catalog-copies.html

ChatGPT said:

Si el endpoint entra y llena el Model pero **no te abre la vista** catalog-copies.html, casi siempre es por una de estas 3 causas (en tu captura ya se ve 1 muy probable):

1) Tu controlador es @RestController (o el método queda como response body)

Si tu clase está anotada con @RestController, Spring **no renderiza plantillas**: devuelve el string "catalog-copies" como texto.

✓ Solución: asegúrate de que la clase sea @Controller:

```
@Controller
@RequestMapping("/catalog")
public class CatalogController {
    ...
}
```

y que el método NO tenga @ResponseBody.

2) La ruta de la vista no coincide con el nombre que retornas

En tu código estás devolviendo:

```
return "catalog-copies";
```

Y el fichero (según tu panel) se llama:

catalog-copies.html ✓ (eso coincide)

Pero ojo: en la imagen también veo `catalog-copies.html` en templates, así que el nombre está bien.

Lo que sí veo raro es esto:

```
@GetMapping("/copies/{id}")
public String getCopyDetails(@PathVariable Long id, Model model) {
    model.addAttribute("book", bookRepository.findById(id));
    model.addAttribute("copies", copyRepository.findById(id)
        .stream().map(CopyDetailsDto::from).toList());
    return "catalog-copies";
}
```

`bookRepository.findById(id)` devuelve un **Optional<Book>**.

Eso no impide renderizar la vista, pero puede provocar errores en la plantilla si haces `${book.title}` (porque `book` es `Optional`). En ese caso verías error 500, no 404.

✓ Mejor: desempaqueta:

```
Book book = bookRepository.findById(id)
    .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND));

model.addAttribute("book", book);
```

Imports:

```
import org.springframework.http.HttpStatus;
import org.springframework.web.server.ResponseStatusException;
```

3) Estás usando la ruta `/copies/{id}` pero tu controller base `/catalog` no coincide

En tu captura el método está con:

```
@GetMapping("/copies/{id}")
```

Si tu clase tiene `@RequestMapping("/catalog")`, entonces la URL real es:

✓ `/catalog/copies/{id}`

Y tu link debe generar eso. Por ejemplo:

```
th:href="@{/catalog/copies/{id}(id=${book.id})}"
```

Esto ya lo tienes corregido.

Lo más probable en tu caso

Que tu clase siga siendo `@RestController` (o tenga `@ResponseBody` a nivel clase/método).

✂ Confírmalo así:

- Si al ir a `/catalog/copies/1` ves en pantalla el texto `catalog-copies` → es `@RestController`.
- Si te da una página de error “Whitelabel Error Page” → está intentando renderizar pero falla dentro.

Config correcto completo (lista para copiar)

```
@Controller
@RequestMapping("/catalog")
public class CatalogController {

    @GetMapping("/copies/{id}")
    public String getCopyDetails(@PathVariable Long id, Model model) {

        Book book = bookRepository.findById(id)
            .orElseThrow(() -> new RuntimeException(HttpStatus.NOT_FOUND));

        model.addAttribute("book", book);

        model.addAttribute("copies", copyRepository.findById(id).stream()
            .map(CopyDetailsDto::from)
            .toList());

        return "catalog-copies";
    }
}
```

Check rápido extra: ¿está Thymeleaf en dependencias?

En `pom.xml` debe existir:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

Si me pegas el encabezado de tu clase (las anotaciones arriba del controller: `@RestController/@Controller, @RequestMapping`, etc.), te digo exacto cuál de los 3 es y lo ajustamos en 1 minuto.

You said:

como poner en `th:if` 2 condiciones con OR

ChatGPT said:

En Thymeleaf usas exactamente el mismo operador lógico que en Java: `||`.

Ejemplo simple:

```
<div th:if="${condicion1 || condicion2}">
    ...
</div>
```

En tu caso típico con roles:

```
<div th:if="${#authorization.expression('hasRole(''ADMIN'')')
    || #authorization.expression('hasRole(''LIBRARIAN'')')}">
    Opciones de administración
</div>
```

O usando las variables que ya cargaste en el modelo:

```
<div th:if="${role == 'ADMIN' || role == 'LIBRARIAN'}">
    Opciones de administración
</div>
```

También puedes mezclar con AND:

```
<div th:if="${isLoggedIn && (role == 'ADMIN' || role == 'LIBRARIAN')}">
```

Paréntesis funcionan igual que en Java y son muy recomendables cuando mezclas operadores.

Ejemplos prácticos:

Mostrar si el usuario es bibliotecario o profesor:

```
<div th:if="${role == 'LIBRARIAN' || role == 'PROFESSOR'}">
```

Mostrar si hay resultados o si la búsqueda está vacía:

```
<div th:if="${#lists.isEmpty(books) || q == null}">
```

Regla mental:

- `||` → OR

- && → AND
- ! → NOT

You said:

org.h2.jdbc.JdbcSQLException: Violación de índice de Unicidad ó Clave primaria: "PUBLIC.CONSTRAINT_41 PRIMARY KEY ON PUBLIC.RESERVATIONS(ID) (/* key:1 */ DATE '2026-01-14', CAST(4 AS BIGINT), TIMESTAMP WITH TIME ZONE '2026-01-20 10:00:00+01', CAST(1 AS BIGINT), CAST(1 AS BIGINT), CAST('ACTIVE' AS ENUM('ACTIVE', 'CANCELED', 'EXPIRED', 'FULFILLED')))" Unique index or primary key violation: "PUBLIC.CONSTRAINT_41 PRIMARY KEY ON PUBLIC.RESERVATIONS(ID) (/* key:1 */ DATE '2026-01-14', CAST(4 AS BIGINT), TIMESTAMP WITH TIME ZONE '2026-01-20 10:00:00+01', CAST(1 AS BIGINT), CAST(1 AS BIGINT), CAST('ACTIVE' AS ENUM('ACTIVE', 'CANCELED', 'EXPIRED', 'FULFILLED')))" ; SQL statement: insert into reservations (copy_id,created_at,deadline,status,user_id,id) values (?,?,,?,default) [23505-240]

ChatGPT said:

Ese error significa: **estás intentando insertar una fila con un ID que ya existe** (colisión con la primary key). En H2 aparece como "PRIMARY KEY ... key:1".

Aunque en el SQL veas `id default`, en H2/JPA eso puede terminar siendo 1 otra vez por alguna de estas causas típicas:

Causas más comunes (y cómo arreglarlas)

1) Tu entidad Reservation no está generando el ID correctamente

Revisa que el `id` tenga `@GeneratedValue` y una estrategia compatible.

Recomendado (H2 + JPA):

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
```

Alternativa (más portable):

```
@Id
@GeneratedValue(strategy = GenerationType.SEQUENCE, generator =
"reservations_seq")
@SequenceGenerator(name = "reservations_seq", sequenceName = "reservations_seq",
```

```
allocationSize = 1)
private Long id;
```

✓ Si no tienes `@GeneratedValue`, JPA intenta insertar IDs “a mano” (o te mete 0/1 repetido).

2) Estás asignando tú el ID antes de guardar

Ejemplo típico de bug:

```
Reservation r = new Reservation();
r.setId(1L); // <- mal
reservationRepository.save(r);
```

O en un mapper/DTO que copia el id.

✓ Solución: **no setear** id al crear nuevas reservas.

3) Tienes datos “seed” / `data.sql` insertando IDs fijos

Si tienes `data.sql` o `import.sql` con:

```
insert into reservations(id, ...) values (1, ...);
```

y luego JPA intenta empezar en 1 otra vez → colisión.

✓ Soluciones:

- No insertar IDs manuales.
- O reiniciar el contador/sequence tras el seed (en H2 depende del tipo).

4) Estás usando H2 con tablas persistentes, pero el contador se reinicia

Si usas H2 en modo fichero (`jdbc:h2:file:...`) o mantienes DB entre reinicios, puede quedarte el registro con ID=1, pero el generador se resetea.

✓ Solución rápida en desarrollo:

En `application.properties`:

```
spring.jpa.hibernate.ddl-auto=create-drop
```

(o `update` si quieres mantener, pero entonces arregla el generador bien)

Qué haría yo en tu caso (pasos concretos)

Paso A) Comprueba tu entidad Reservation

Debe verse así (mínimo):

```
@Entity
@Table(name = "reservations")
public class Reservation {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    // ...
}
```

Paso B) Asegúrate de que cuando creas una reserva NO seteas id

Crea así:

```
Reservation r = new Reservation();
r.setId(null); // opcional
// set copyId, userId, deadline, status, createdAt...
reservationRepository.save(r);
```

Paso C) Si ya tienes la DB “sucia” en dev, bórrala o recrea

- Si H2 es en memoria: reinicia app
- Si H2 es en fichero: borra el archivo .mv.db o cambia a create-drop

Por qué el error muestra “key:1”

H2 te dice que ya existe una fila con ID = 1 (y te imprime el contenido de esa fila: fechas, copy_id=4, etc.). Luego intentas insertar otra que también acaba con ID=1.

Si me pegas tu entidad Reservation (solo el campo id y las anotaciones de la clase) y cómo estás creando/guardando la reserva, te digo exactamente cuál de los 4 casos te está pasando y el fix exacto.

You said:

tengo datos en data sql

ChatGPT said:

Vale, entonces el motivo del error cuadra al 100%: **tu data.sql está insertando reservas con IDs (o provoca que el contador empiece en 1)** y luego cuando tu app intenta crear una nueva reserva, H2 intenta volver a usar un id que ya existe (por ejemplo 1).

Tienes varias formas de solucionarlo. Te dejo las más típicas (elige una).

Opción A (la mejor): en data.sql NO insertes el id

Si tu entidad tiene:

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
```

Entonces en data.sql haz inserts sin id:

```
INSERT INTO reservations (copy_id, created_at, deadline, status, user_id)
VALUES (4, DATE '2026-01-14', TIMESTAMP WITH TIME ZONE '2026-01-20 10:00:00+01',
'ACTIVE', 1);
```

✓ Así H2 genera el id automáticamente y no choca.

Si ahora mismo tienes algo como:

```
INSERT INTO reservations (id, copy_id, ...) VALUES (1, 4, ...);
```

cámbialo como arriba.

Opción B: si por lo que sea quieres IDs fijos en data.sql, reinicia el contador después

Si insertas manualmente id=1,2,3..., luego debes decirle a H2 que el siguiente id empiece después del máximo.

Al final de data.sql añade:

```
ALTER TABLE reservations ALTER COLUMN id RESTART WITH 1000;
```

(pon el número que quieras, o el MAX(id)+1 que tenga sentido)

✓ Con esto, aunque tengas ya id=1..n, las nuevas reservas empezarán en 1000 y no habrá colisión.

Esto funciona si id es IDENTITY (lo más común con GenerationType.IDENTITY).

Opción C: si usas SEQUENCE (en vez de IDENTITY), reinicia la secuencia

Si tu entidad usa `@SequenceGenerator`, entonces resetea la secuencia:

```
ALTER SEQUENCE reservations_seq RESTART WITH 1000;
```

(El nombre real de la secuencia puede variar: `hibernate_sequence`, `reservations_seq`, etc.)

Opción D: en desarrollo, recrea la DB cada vez

Si te da igual perder datos en dev, esto evita “DB sucia”:

```
spring.jpa.hibernate.ddl-auto=create-drop
```

o incluso:

```
spring.jpa.hibernate.ddl-auto=create
```

✓ Cada arranque recrea tablas y vuelve a ejecutar `data.sql` limpio.

Recomendación para tu caso

- Si `data.sql` es solo datos de prueba → **Opción A** (no meter id) es lo más limpio.
- Si necesitas mantener IDs concretos por tests → **Opción B** (RESTART WITH) es lo más directo.

Tip importante (Spring Boot 2.5+ / 3.x)

Si usas JPA y `data.sql`, a veces necesitas esto para que `data.sql` se ejecute **después** de crear las tablas:

```
spring.jpa.defer-datasource-initialization=true  
spring.sql.init.mode=always
```

(No arregla el choque de IDs, pero evita problemas de orden).

Si me pegas el trozo de tu `data.sql` donde insertas en `reservations` y tu campo `id` de la entidad `Reservation`, te digo exactamente cuál opción encaja y te lo dejo listo con el SQL correcto (IDENTITY vs SEQUENCE).

You said:

```
Resolved [org.springframework.web.HttpMediaTypeNotSupportedException: Content-Type 'application/x-www-form-urlencoded; charset=UTF-8' is not supported]
```

ChatGPT said:

Ese error significa que tu endpoint **está recibiendo un POST con datos de formulario** (application/x-www-form-urlencoded) pero tu método Spring **espera otro tipo** (normalmente JSON con @RequestBody).

Esto pasa muchísimo cuando:

- envías un <form method="post">...
- y en el controller tienes @PostMapping con @RequestBody

Qué está pasando exactamente

Un <form> HTML envía por defecto:

Content-Type: application/x-www-form-urlencoded

Ejemplo:

```
copyId=123&userId=7
```

Pero tu endpoint probablemente está definido así:

```
@PostMapping("/reservations")
public ReservationDto create(@RequestBody CreateReservationDto dto) { ... }
```

Eso exige Content-Type: application/json, por eso Spring te dice “no soportado”.

Solución 1 (la más típica con formularios): usar @ModelAttribute o @RequestParam

Si vas a enviar desde <form>, haz:

Opción A: @RequestParam

```
@PostMapping("/reservations")
public String create(@RequestParam Long copyId) {
    reservationService.create(copyId);
    return "redirect:/my";
}
```

Opción B: @ModelAttribute

```
@PostMapping("/reservations")
public String create(@ModelAttribute CreateReservationForm form) {
    reservationService.create(form.copyId());
}
```

```
return "redirect:/my";
}
```

Solución 2: si quieres JSON, no uses <form> (usa fetch)

En el front:

```
fetch("/reservations", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ copyId: 123 })
});
```

Y en el backend mantienes:

```
@PostMapping(value="/reservations", consumes="application/json")
public ResponseEntity<?> create(@RequestBody CreateReservationDto dto) { ... }
```

Solución 3: hacer que tu endpoint acepte application/x-www-form-urlencoded

Si quieres mantener POST pero aceptar form-urlencoded, indica consumes y no uses @RequestBody:

```
@PostMapping(value="/reservations", consumes =
MediaType.APPLICATION_FORM_URLENCODED_VALUE)
public String create(CreateReservationForm form) {
  reservationService.create(form.getCopyId());
  return "redirect:/my";
}
```

Imports:

```
import org.springframework.http.MediaType;
```

¿Qué te recomiendo para tu proyecto?

Como estás usando Thymeleaf y formularios en varias pantallas (reservar, renovar, cancelar), lo más coherente es:

- ✅ endpoints que reciben formularios → @RequestParam / @ModelAttribute
- ✅ endpoints API JSON → @RequestBody + fetch

No mezcles un <form> con un endpoint que espera JSON.

Si me pegas tu método del controller que da este error (el `@PostMapping`), te digo exactamente qué línea cambiar (si es `@RequestBody`, `consumes`, etc.) según si quieres form o JSON.

You said:

el use case es: es una aplicación de librería, elijo un libro y le doy a reservar (se me genera la reserva), se me abre otra ventana para darle confirmar o no. Lo que falla es la parte de confirmar. A parte de solucionar esto, como sería mejor gestionarlo? mostrando solo los datos para la página de confirmar sin generar la reserva, y generarlo al confirmarlo?

ChatGPT said:

Lo que te falla en **confirmar** encaja con el error anterior: estás enviando un **POST de formulario** y tu endpoint probablemente espera **JSON** (`@RequestBody`) o tiene `consumes=application/json`. Solucionas eso cambiando el endpoint de confirmación a `@RequestParam/@ModelAttribute` (o mandando JSON por fetch). Te dejo el flujo bien montado y, además, cómo gestionarlo mejor.

Qué es “mejor” para tu caso de uso

Sí: **es mejor NO crear la reserva en el primer click.**

Motivos:

- Evitas reservas “fantasma” si el usuario cierra la pestaña o vuelve atrás.
- Evitas problemas de idempotencia (doble click) y de expiración.
- La confirmación tiene sentido: **solo confirmas lo que aún no existe.**

En tu dominio, el primer click “Reservar” debería ser:

“Quiero reservar este ejemplar” → mostrar pantalla de confirmación con datos.

Y al pulsar “Confirmar”:

“Crear reserva” (acción que cambia estado) → POST.

Diseño recomendado (simple y limpio)

1) GET /reserve/{copyId} → muestra pantalla de confirmación

- NO cambia estado.
- Solo carga datos para enseñar al usuario: libro, ejemplar, disponibilidad, etc.

2) POST /reserve/confirm → crea la reserva

- Cambia estado.
- Valida que el ejemplar sigue reservable.
- Maneja doble submit (idempotencia o restricción única).

Implementación Spring MVC + Thymeleaf (sin JSON)

Controller

```
@Controller
public class ReservationController {

    @GetMapping("/reserve/{copyId}")
    public String reserveConfirmPage(@PathVariable Long copyId, Model model,
    Authentication auth) {
        // 1) Carga copy + book (solo lectura)
        Copy copy = copyRepository.findById(copyId)
            .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND));

        Book book = bookRepository.findById(copy.getBookId())
            .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND));

        // 2) Datos para la vista
        model.addAttribute("copyId", copyId);
        model.addAttribute("book", book);
        model.addAttribute("copy", copy);
        return "reserve"; // reserve.html
    }

    @PostMapping("/reserve/confirm")
    public String confirmReservation(@RequestParam Long copyId, Authentication
    auth, RedirectAttributes ra) {

        Long userId = /* obtén userId desde auth o tu tabla users */;

        try {
            reservationService.createReservation(userId, copyId);
            ra.addFlashAttribute("toastOk", "Reserva creada ✅");
            return "redirect:/my";
        } catch (ReservationNotAllowedException ex) {
```

```

        ra.addFlashAttribute("toastErr", ex.getMessage());
        return "redirect:/reserve/" + copyId;
    }
}
}

```

Imports:

```

import org.springframework.http.HttpStatus;
import org.springframework.security.core.Authentication;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.server.ResponseStatusException;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;

```

reserve.html (confirmación)

```

<form th:action="@{/reserve/confirm}" method="post" class="row spread"
style="align-items: flex-end;">
    <input type="hidden" name="copyId" th:value="${copyId}"/>

    <!-- CSRF -->
    <input type="hidden" th:name="${_csrf.parameterName}"
th:value="${_csrf.token}"/>

    <button class="btn primary" type="submit">Confirmar reserva</button>
    <a class="btn" th:href="@{|/catalog/copies/${book.id}|}">Cancelar</a>
</form>

```

✓ Con esto **no hay JSON**, y no volverás a ver el error `application/x-www-form-urlencoded not supported`.

Qué validar en el service al confirmar

La confirmación es donde debes aplicar reglas:

- El ejemplar existe
- No es “referencia” si no se permite
- Está disponible o se permite cola
- El usuario no está sancionado
- No hay otra reserva activa duplicada del mismo usuario y ejemplar

Y muy importante: **evitar doble submit**.

Restricción única recomendada en DB

Para que un usuario no cree 2 reservas activas del mismo ejemplar:

- si tu DB soporta índices parciales: `unique (user_id, copy_id) where status= 'ACTIVE'`
- si no, como mínimo: `unique (user_id, copy_id, status)` y controlas en lógica.
En H2/JPA, una opción sencilla (no perfecta, pero útil):
- antes de insertar, compruebas si existe ACTIVE y si existe → no insertas.

Si quieres aún más robustez (token de confirmación)

Si te preocupa que alguien “fabrique” el POST con otro copyId, puedes añadir un token de confirmación (CSRF ya protege cross-site, pero no evita manipulación interna).

Opción práctica:

- En la página GET generas un `reservationIntentId` (UUID) guardado en sesión.
 - En POST confirmas usando ese intent.
- Pero para una librería típica, con CSRF + validación del servicio suele ser suficiente.

Si quisieras hacerlo con JS (opcional)

También se puede con fetch JSON, pero entonces:

- el `<form>` no debería postear normal
- tu endpoint debería ser `@RequestBody` JSON
- y manejas la UX en frontend

Como estás con Thymeleaf y flujos clásicos, **Spring MVC + form POST** es lo más simple y consistente.

Resumen

- ☒ Sí, es mejor: **mostrar confirmación sin crear reserva.**
- ☒ La reserva se crea **solo al confirmar** (POST).
- ☒ Tu error actual se arregla haciendo que el POST acepte formulario (`@RequestParam/@ModelAttribute`) o cambiando a fetch JSON.

Si me pegas tu método actual de “confirmar” (el `@PostMapping`) y tu `reserve.html`, te lo ajusto exactamente a tu código (sin reestructurar más de lo necesario).

You said:

RedirectAttributes RedirectAttributes para que sirve

ChatGPT said:

RedirectAttributes sirve para **pasar datos de forma segura cuando haces un redirect**: en Spring MVC.

Es decir:

👉 **mensajes o datos “temporales”** que solo existen **después del redirect**, no se quedan en la URL ni en sesión para siempre.

El problema que resuelve

Cuando haces:

```
return "redirect:/my";
```

el Model **se pierde**, porque:

- el redirect es **otra petición HTTP nueva**
- el navegador hace un GET nuevo a /my

Entonces esto **NO funciona**:

```
model.addAttribute("msg", "Reserva creada");  
return "redirect:/my"; // ✗ msg se pierde
```

Qué hace RedirectAttributes

Guarda atributos como **flash attributes**:

- viven solo **una redirección**
- no aparecen en la URL
- se eliminan automáticamente tras usarse

Uso típico (el más común)

```
@PostMapping("/reserve/confirm")  
public String confirm(RedirectAttributes ra) {  
  
    ra.addFlashAttribute("success", "Reserva creada correctamente");  
  
    return "redirect:/my";  
}
```

Y en /my (Thymeleaf):

```
<div th:if="${success}" class="alert ok">
  <span th:text="${success}"></span>
</div>
```

➡ El mensaje aparece **una vez** y desaparece al refrescar.

Diferencia entre addFlashAttribute y addAttribute

addFlashAttribute

```
ra.addFlashAttribute("msg", "OK");
```

- No va en la URL
- Vive solo 1 redirect
- Ideal para mensajes (success/error)

addAttribute

```
ra.addAttribute("id", 5);
```

- Se convierte en query param:
redirect:/catalog?id=5
- Visible en la URL
- Útil para IDs, filtros, paginación

Ejemplo completo en tu caso de biblioteca

Controller

```
@PostMapping("/reserve/confirm")
public String confirmReservation(
    @RequestParam Long copyId,
    RedirectAttributes ra
) {
    try {
        reservationService.create(copyId);
        ra.addFlashAttribute("toastOk", "Reserva confirmada ✅");
        return "redirect:/my";
    } catch (Exception e) {
        ra.addFlashAttribute("toastErr", "No se pudo crear la reserva");
        return "redirect:/reserve/" + copyId;
    }
}
```

Thymeleaf (en my.html)

```
<div th:if="${toastOk}" class="chip ok" th:text="${toastOk}"></div>
<div th:if="${toastErr}" class="chip bad" th:text="${toastErr}"></div>
```

Por qué NO usar sesión directamente

Podrías hacer:

```
session.setAttribute("msg", "OK");
```

Pero:

- se queda “pegado”
- tienes que limpiarlo a mano
- se filtra entre páginas

RedirectAttributes está hecho exactamente para esto.

Resumen rápido

- RedirectAttributes = datos **temporales** tras redirect
- addFlashAttribute = mensajes (lo normal)
- addAttribute = parámetros de URL
- Evita usar Model en redirects
- Evita usar sesión para mensajes

En apps como la tuya (formularios + confirmaciones) es **la forma correcta** de pasar feedback al usuario.

You said:

```
<div class="right"> <a class="btn danger" th:href="@{/reservations/{res.id}"}
method="delete">Cancelar reserva</a> </div>
```

como puedo poner eso bien?

ChatGPT said:

Buen ojo: **así como lo tienes no puede funcionar.**

Un `<a>` **no puede** hacer DELETE. El atributo `method="delete"` ahí **no existe en HTML** y el navegador siempre hará un **GET**.

En Spring MVC hay **2 formas correctas** de hacerlo. Te las dejo y te digo cuál recomiendo.

❑ Opción 1 (RECOMENDADA): <form method="post"> + DELETE lógico

Es la forma estándar con Spring + Thymeleaf.

HTML (correcto)

```
<div class="right">
  <form th:action="@{/reservations/{id}(id=${res.id})}"
        method="post"
        style="margin:0;">

    <!-- CSRF -->
    <input type="hidden"
          th:name="${_csrf.parameterName}"
          th:value="${_csrf.token}"/>

    <!-- Override del método -->
    <input type="hidden" name="_method" value="delete"/>

    <button class="btn danger" type="submit">
      Cancelar reserva
    </button>
  </form>
</div>
```

Controller

```
@DeleteMapping("/reservations/{id}")
public String cancelReservation(@PathVariable Long id,
                               RedirectAttributes ra) {
    reservationService.cancel(id);
    ra.addFlashAttribute("toastOk", "Reserva cancelada");
    return "redirect:/my";
}
```

Config necesaria (MUY IMPORTANTE)

Spring necesita este filtro para soportar _method=delete:

```
@Bean
HiddenHttpMethodFilter hiddenHttpMethodFilter() {
    return new HiddenHttpMethodFilter();
}
```

(Spring Boot suele traerlo, pero mejor asegurarlo).

✓ Opción 2: usar POST (más simple, también válida)

Si no quieres DELETE real (en muchas apps no es crítico):

HTML

```
<form th:action="@{/reservations/{id}/cancel(id=${res.id})}"
      method="post"
      style="margin:0;">

  <input type="hidden"
        th:name="${_csrf.parameterName}"
        th:value="${_csrf.token}"/>

  <button class="btn danger" type="submit">
    Cancelar reserva
  </button>
</form>
```

Controller

```
@PostMapping("/reservations/{id}/cancel")
public String cancelReservation(@PathVariable Long id,
                               RedirectAttributes ra) {
    reservationService.cancel(id);
    ra.addFlashAttribute("toastOk", "Reserva cancelada");
    return "redirect:/my";
}
```

👉 Para apps web clásicas esta opción es totalmente aceptable y más sencilla.

✗ Lo que NO funciona

```
<a href="/reservations/1" method="delete">✗</a>
```

HTML:

- <a> → solo GET
- method → ignorado
- Spring nunca recibe DELETE

Qué te recomiendo en tu proyecto

Como estás usando:

- Thymeleaf
- formularios
- CSRF
- flujos clásicos

👉 Opción 1 si quieres REST “correcto”

👉 Opción 2 si quieres simplicidad (yo usaría esta en una app de biblioteca)

Bonus: confirmación antes de cancelar

Puedes añadir sin romper nada:

```
<button class="btn danger"
        type="submit"
        onclick="return confirm('¿Cancelar esta reserva?');">
    Cancelar reserva
</button>
```

Resumen rápido

- <a> no sirve para DELETE ✗
- Usa <form> ✓
- CSRF obligatorio
- _method=delete si quieres DELETE real
- POST /cancel si quieres simplicidad

Si quieres, dime cuál opción eliges y te adapto **todo el flujo** (controller + service + vista) exactamente a tu estructura actual.

You said:



como puedo saber si me está pillando bien los roles en .hasRole("LIBRARIAN")

ChatGPT said:

Muy buena pregunta, porque cuando hasRole("LIBRARIAN") **no funciona**, casi siempre es porque Spring **sí recibe los roles**, pero **con otro nombre o prefijo**.

Vamos por partes y te doy **formas prácticas de comprobarlo**.

1 Comprueba qué authorities ve Spring de verdad

La forma más directa: **imprime las authorities del usuario autenticado.**

Opción A: en un controller

Añade temporalmente esto:

```
@GetMapping("/debug/roles")
@ResponseBody
public Object debugRoles(Authentication authentication) {
    return authentication.getAuthorities();
}
```

Entra logueado a:

/debug/roles

Deberías ver algo como:

```
[
  { "authority": "ROLE_LIBRARIAN" }
]
```

🔑 Si ves:

- ROLE_LIBRARIAN → hasRole("LIBRARIAN") está bien
- LIBRARIAN (sin ROLE_) → necesitas prefijo
- nada → el converter no está funcionando

Opción B: en logs (sin endpoint)

Pon un log temporal en cualquier controller:

```
log.info("Authorities: {}", authentication.getAuthorities());
```

2 Verifica el JWT que llega desde Cognito

Abre el token JWT (en jwt.io o similar) y busca:

```
"cognito:groups": ["LIBRARIAN"]
```

Si NO aparece:

- el usuario no está en ese grupo
- o el App Client no incluye el claim

3 Verifica tu JwtGrantedAuthoritiesConverter

Debe ser exactamente así (ya lo vimos antes, pero lo repito porque aquí suele estar el fallo):


```

@Bean
public JwtAuthenticationConverter jwtAuthenticationConverter() {
    JwtGrantedAuthoritiesConverter converter = new
    JwtGrantedAuthoritiesConverter();
    converter.setAuthoritiesClaimName("cognito:groups");
    converter.setAuthorityPrefix("ROLE_");

    JwtAuthenticationConverter jwtConverter = new JwtAuthenticationConverter();
    jwtConverter.setJwtGrantedAuthoritiesConverter(converter);
    return jwtConverter;
}

```

⚠ Si usas oauth2Login (OIDC), este converter **no se aplica automáticamente**.
En ese caso los roles vienen en OidcUser, no en Jwt.

4 Si usas oauth2Login (Cognito Hosted UI): ojo con esto

Con oauth2Login, Spring **NO** usa JwtAuthenticationConverter.

Ahí tienes dos opciones:

✓ Opción correcta con OIDC (recomendada)

Usa los grupos desde OidcUser y un GrantedAuthoritiesMapper.

```

@Bean
GrantedAuthoritiesMapper authoritiesMapper() {
    return authorities -> authorities.stream()
        .flatMap(a -> {
            if (a instanceof OidcUserAuthority oidc) {
                Object groups =
oidc.getUserInfo().getClaims().get("cognito:groups");
                if (groups instanceof Collection<?> g) {
                    return g.stream()
                        .map(Object::toString)
                        .map(r -> new SimpleGrantedAuthority("ROLE_" + r));
                }
            }
            return Stream.of(a);
        })
        .toList();
}

```

Y en security:

```
.oauth2Login(oauth -> oauth
    .userInfoEndpoint(u -> u.userAuthoritiesMapper(authoritiesMapper()))
)
```

5 Verificación rápida desde Thymeleaf

Si tienes thymeleaf-extras-springsecurity6:

```
<div sec:authorize="hasRole('LIBRARIAN')">
    SOLO BIBLIOTECARIO
</div>

<div sec:authorize="isAuthenticated()">
    AUTENTICADO
</div>
```

Si el primero **no aparece** pero el segundo sí → el rol no está bien mapeado.

6 Error típico: usar hasRole("ROLE_LIBRARIAN")

✗ Mal:

```
.hasRole("ROLE_LIBRARIAN")
```

hasRole **añade** ROLE_ automáticamente.

✓ Bien:

```
.hasRole("LIBRARIAN")
```

Resumen rápido de diagnóstico

Qué ves	Conclusión
ROLE_LIBRARIAN en /debug/roles	Todo OK
LIBRARIAN sin ROLE_	Falta prefijo
No hay authorities	Converter/mapper no aplicado
JWT no tiene cognito:groups	Usuario no está en grupo

Si quieres, dime **si estás usando** oauth2Login **o** resource-server (por la captura parece oauth2Login), y te dejo **la configuración exacta mínima** para Cognito + roles sin ambigüedades.

```
[{"attributes": {"at_hash": "4Spdh02DN2NxfVmuDoHu5Q", "sub": "94e8a428-40b1-709f-6365-a4a667c41a8d", "email_verified": true, "cognito:groups": ["LIBRARIAN"], "iss": "https://cognito-idp.us-east-1.amazonaws.com/us-east-1_SNNleUPIH", "cognito:username": "94e8a428-40b1-709f-6365-a4a667c41a8d", "nonce": "RkmREFN5csvNWmxuKn0UmU9DZzap9WMbJWj_2B1ryfl", "origin_jti": "82a64a1c-9c84-4182-bc0c-b0b93c899c3e", "aud": ["1h2dcs34d8ucv154k44joac1bc"], "event_id": "18794817-2e4a-4dd7-a67a-afc7d483bb47", "token_use": "id", "auth_time": "2026-02-03T10:42:47Z", "exp": "2026-02-03T11:42:47Z", "iat": "2026-02-03T10:42:47Z", "email": "prueba@prueba.com", "jti": "9a2cd886-3ad2-4f71-a412-dca1edd80344", "username": "94e8a428-40b1-709f-6365-a4a667c41a8d"}, "authority": "OIDC_USER", "idToken": {"tokenValue": "eyJraWQiOiI5S1E5TXVDRjJhZFJ4QXVTdlNoTFZKTWNPYWVxTjBKXC8reHY0S1hyRkV3QT0iLCJhbGciOiJSUzI1NiJ9.eyJhdF9oYXNoIjoiaWFNFwZGgwMkROMk54ZlZtdURvSHU1USIsInN1YiI6Ijk0ZThhNDI4LTQwYyEtNzA5Zi02MzY1LWE0YTZY2N2M0MWE4ZCIsmNvZ25pdG86Z3JvdXBzljpbIkxjQlJBUKlBTiJdLCJlbWFpbF92ZXJpZmllZCI6dHJ1ZSwiaXNzIjoiaHR0cHM6XC9icL2NvZ25pdG8taWRwLnVzLVVhc3QtMS5hbWF6b25hd3MuY29tXC91cy1lYXN0LTFffU05ubGVVUEIIiwiY29nbml0bzp1c2VybmFtZSI6Ijk0ZThhNDI4LTQwYyEtNzA5Zi02MzY1LWE0YTZY2N2M0MWE4ZCIsm5vbmlIjoiuUmttUkVGTjVjc3ZOv214dUtuMFVtVTlEWnphcDIXTWJlK2pfMklxcnlmSSIsIm9yaWdpbnl9qdGkiOiI4MmE2NGExYy05Yzg0LTQxODItYmMwYy1iMGi5M2M4OTljM2UiLCJhdWQiOiIxaDJkY3MzNGQ4dWN2MTU0azQ0am9hYzFiYyIsImV2ZW50X2lkIjoiaMTg3OTQ4MTctMmU0YS00ZGQ3LWE2N2EtYWZjN2Q0ODNiYjQ3IiwidG9rZW5fdXNlIjoiaWQiLCJhdXR0X3RpbWUiOiJlE3NzAxMTUzNjcsImV4cCI6MTc3MDExODk2NywiaWF0IjojoxNzczwMTE1MzY3LCJqdGkiOiI5YTJjZDg4Ni0zYWQyLTRmNzEtYTQxMi1kY2ExZWRkODA4ZDNDQiLCJlbWFpbGl6InBydWViYUBwcniYmEuY29tIn0.YPDm5eisxmlH5HnVzLOAKBkN3Q8F6x8PQRoimmvhNyKFqaFBIKW36Ud1UeGfdtrN-zBn8E00Fv53v9rG_la_W5k4en8VdKw8arhqf1E7dOiHGR8Wmsklxz1jKFhj_j4IG7zvUoq3ouXistQ3L9RCtL4VxsNgnt-L11cPI3nsdLg8rI94sPQR6xupBPTltrbsbldLU3yLIYhi_WGuz5_60pmF7X9RVBiz8k4PjH-WeWtMKxxwi0uuKlnSESNZ2rbDdvcoLuYQn559fe6waJXBPLGeVJC42NIrWRA6aPDNaVzK4hKkQhw2qsWYyPh-FaqyWQ4ZoVdcYsGjtKKLtqkPuA", "issuedAt": "2026-02-03T10:42:47Z", "expiresAt": "2026-02-03T11:42:47Z", "claims": {"at_hash": "4Spdh02DN2NxfVmuDoHu5Q", "sub": "94e8a428-40b1-709f-6365-a4a667c41a8d", "cognito:groups": ["LIBRARIAN"], "email_verified": true, "iss": "https://cognito-idp.us-east-1.amazonaws.com/us-east-1_SNNleUPIH", "cognito:username": "94e8a428-40b1-709f-6365-a4a667c41a8d", "nonce": "RkmREFN5csvNWmxuKn0UmU9DZzap9WMbJWj_2B1ryfl", "origin_jti": "82a64a1c-9c84-4182-bc0c-b0b93c899c3e", "aud": ["1h2dcs34d8ucv154k44joac1bc"], "event_id":
```

```
"18794817-2e4a-4dd7-a67a-afc7d483bb47", "token_use": "id", "auth_time": "2026-02-03T10:42:47Z", "exp": "2026-02-03T11:42:47Z", "iat": "2026-02-03T10:42:47Z", "jti": "9a2cd886-3ad2-4f71-a412-dca1ed80344", "email": "prueba@prueba.com", "accessTokenHash": "4Spdh02DN2NxfVmuDoHu5Q", "address": { "country": null, "formatted": null, "locality": null, "postalCode": null, "region": null, "streetAddress": null }, "audience": [ "1h2dcs34d8ucv154k44joac1bc" ], "authenticatedAt": "2026-02-03T10:42:47Z", "authenticationContextClass": null, "authenticationMethods": null, "authorizationCodeHash": null, "authorizedParty": null, "birthdate": null, "email": "prueba@prueba.com", "emailVerified": true, "familyName": null, "fullName": null, "gender": null, "givenName": null, "issuer": "https://cognito-idp.us-east-1.amazonaws.com/us-east-1_SNnleUPIH", "locale": null, "middleName": null, "nickName": null, "nonce": "RkmREFN5csvNWmxuKn0UmU9DZzap9WMbJWj_2B1ryfl", "phoneNumber": null, "phoneNumberVerified": null, "picture": null, "preferredUsername": null, "profile": null, "subject": "94e8a428-40b1-709f-6365-a4a667c41a8d", "updatedAt": null, "website": null, "zoneInfo": null }, "userInfo": { "claims": { "sub": "94e8a428-40b1-709f-6365-a4a667c41a8d", "email_verified": true, "email": "prueba@prueba.com", "username": "94e8a428-40b1-709f-6365-a4a667c41a8d" }, "address": { "country": null, "formatted": null, "locality": null, "postalCode": null, "region": null, "streetAddress": null }, "birthdate": null, "email": "prueba@prueba.com", "emailVerified": true, "familyName": null, "fullName": null, "gender": null, "givenName": null, "locale": null, "middleName": null, "nickName": null, "phoneNumber": null, "phoneNumberVerified": null, "picture": null, "preferredUsername": null, "profile": null, "subject": "94e8a428-40b1-709f-6365-a4a667c41a8d", "updatedAt": null, "website": null, "zoneInfo": null }, "userNameAttributeName": "username" } ]
```

ChatGPT said:

Esto es súper útil: tu salida confirma exactamente qué pasa.

Lo que estás viendo ahora mismo

En tu Authentication solo tienes una authority:

- "authority": "OIDC_USER"
- Y **no** aparece ROLE_LIBRARIAN.

Pero **sí** tienes el claim:

- "cognito:groups": ["LIBRARIAN"]

✓ Conclusión: **Cognito está enviando el grupo bien, pero Spring Security NO lo está convirtiendo a roles.**

Por eso `.hasRole("LIBRARIAN")` no te hace caso.

Solución correcta para oauth2Login (Cognito Hosted UI)

Como estás usando oauth2Login, el JwtGrantedAuthoritiesConverter **no se aplica**. Tienes que mapear los grupos a authorities con un GrantedAuthoritiesMapper.

1) Crea el mapper

```
import org.springframework.context.annotation.Bean;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.authority.mapping.GrantedAuthoritiesMapper;
import org.springframework.security.oauth2.core.oidc.user.OidcUserAuthority;

import java.util.*;
import java.util.stream.Stream;

@Bean
public GrantedAuthoritiesMapper cognitoAuthoritiesMapper() {
    return (authorities) -> authorities.stream()
        .flatMap(a -> {
            // Mantén las authorities existentes (OIDC_USER, etc.)
            Stream<GrantedAuthority> base = Stream.of(a);

            if (a instanceof OidcUserAuthority oidc) {
                Object groups =
                    oidc.getIdToken().getClaims().get("cognito:groups");

                if (groups instanceof Collection<?> list) {
                    Stream<GrantedAuthority> roles = list.stream()
                        .map(Object::toString)
                        .map(g -> new SimpleGrantedAuthority("ROLE_" + g));

                    return Stream.concat(base, roles);
                }
            }

            return base;
        })
    // evita duplicados
```

```
        .collect(java.util.stream.Collectors.toSet());
    }
```

2) Enchúfalo en tu oauth2Login

En tu SecurityFilterChain:

```
http
    .oauth2Login(oauth -> oauth
        .userInfoEndpoint(userInfo -> userInfo
            .userAuthoritiesMapper(cognitoAuthoritiesMapper()))
        )
    .defaultSuccessUrl("/menu", true)
);
```

Con eso, al autenticarte, Spring tendrá:

- OIDC_USER
- ROLE_LIBRARIAN

Y entonces:

```
.requestMatchers("/admin/**").hasRole("LIBRARIAN")
```

✔ funcionará.

Cómo comprobarlo rápido

Vuelve a tu endpoint de debug (el que te sacó ese JSON) y mira `authentication.getAuthorities()`.

Deberías ver algo como:

```
[
  {"authority": "OIDC_USER"},
  {"authority": "ROLE_LIBRARIAN"}
]
```

Nota importante

Tus grupos están en el **idToken** (se ve en `idToken.claims`).

Tu `userInfo.claims` no trae `cognito:groups`, así que **hay que leerlo del idToken**, como hago arriba.

Bonus: proteger rutas por varios roles

Una vez tengas roles bien, esto también funciona:

```
.requestMatchers("/admin/**").hasAnyRole("LIBRARIAN", "ADMIN")
```

Si me pegas tu bloque completo de `.oauth2Login(...)` actual, te lo devuelvo ya integrado con el mapper para que lo puedas copiar tal cual.

You said:

```
<!-- src/main/resources/templates/admin-loan.html --> <!doctype html> <html lang="es"
xmlns:th="http://www.thymeleaf.org"> <head> <meta charset="utf-8"/> <meta
name="viewport" content="width=device-width,initial-scale=1"/> <title>Registrar
préstamo</title> <link rel="stylesheet" th:href="@{/css/app.css}"> </head> <body>
<header th:replace="~{fragments/layout :: topbar(${userName}, ${role})}"></header>
<main class="container"> <section class="card pad"> <div class="row spread"> <div> <h1
class="h1">Bibliotecario → Registrar préstamo</h1> <p class="p">1) Buscar usuario · 2)
Escanear ejemplar</p> </div> <a class="btn" th:href="@{/menu}">← Volver</a> </div>
<div class="divider"></div> <div class="grid two"> <div class="card pad"> <h2
class="h2">1. Buscar usuario</h2> <form class="row" method="get"
th:action="@{/admin/loan}"> <input class="input" name="userQ" placeholder="Username
/ Email" th:value="${userQ}" style="flex:1;min-width:220px;"> <button class="btn
primary" type="submit">Buscar</button> </form> <div class="divider"></div> <div
th:if="${selectedUser != null}"> <p class="p"> <strong
th:text="${selectedUser.name}">María López</strong> <span class="chip"
th:text="${selectedUser.type}">Estudiante</span> </p> </div> <p class="p"
th:if="${selectedUser == null}">Sin usuario seleccionado.</p> </div> <div class="card
pad"> <h2 class="h2">2. Escanear ejemplar</h2> <form method="post"
th:action="@{/admin/loan/scan}" class="field"> <label class="label">Código de barras /
ID ejemplar</label> <input class="input" name="copyCode" placeholder="Escanear..." />
<button class="btn primary" type="submit">Continuar</button> </form> <div
class="divider"></div> <div th:if="${copy != null}"> <p class="p" th:text="'Ejemplar: ' +
${copy.id}">Ejemplar: 12345</p> <p class="p" th:text="'Estado: ' +
${copy.status}">Estado: Disponible</p> <p class="p" th:text="'Reserva para este usuario: '
+ (${copy.reservedForUser} ? 'Sí' : 'No')"> Reserva para este usuario: Sí/No </p> <p
class="p" th:text="'¿Referencia?: ' + (${copy.reference} ? 'Sí' : 'No')'">¿Referencia?: No</p>
<div class="divider"></div> <form method="post" th:action="@{/admin/loan/confirm}">
<button class="btn primary" type="submit">Confirmar préstamo</button> </form>
</div> <p class="p" th:if="${copy == null}">Escanee un ejemplar para ver detalles.</p>
</div> </div> </section> <footer th:replace="~{fragments/layout :: footer}"></footer>
</main> </body> </html>
```

Quiero que me modifiques esto para que el usuario 1- busque el usuario, 2- le sale una lista, 3- selecciona el user que quiere 4 - escanee/meta el id de ejemplar

ChatGPT said:

Aquí tienes el **mismo** `admin-loan.html` modificado para el flujo que quieres:

1. Buscar usuario
2. Mostrar lista de resultados
3. Seleccionar usuario (queda "fijado")
4. Escanear / meter id de ejemplar

He asumido estas variables de modelo (fáciles de montar en tu controller):

- `userQ` (String) → texto buscado
- `users` (List) → resultados de búsqueda
- `selectedUser` (objeto) → usuario elegido (o null)
- `copy` (objeto) → ejemplar escaneado (o null)
- `selectedUserId` (Long/String) → id del usuario seleccionado (opcional; también se puede sacar de `selectedUser.id`)

Y he hecho que:

- El formulario de búsqueda siga siendo GET `/admin/loan`
- Selección de usuario sea POST `/admin/loan/select`
- Escaneo sea POST `/admin/loan/scan` con `selectedUserId` escondido
- Confirmación sea POST `/admin/loan/confirm` con `selectedUserId` y `copyId`

Importante: añadí CSRF hidden en los POST (ya lo necesitas en tu app).

```
<!-- src/main/resources/templates/admin-loan.html -->
<!doctype html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="utf-8"/>
  <meta name="viewport" content="width=device-width,initial-scale=1"/>
  <title>Registrar préstamo</title>
  <link rel="stylesheet" th:href="@{/css/app.css}">
</head>
<body>
<header th:replace="~{fragments/layout :: topbar(${userName},
${role})}"></header>

<main class="container">
  <section class="card pad">
    <div class="row spread">
      <div>
```



```

        <h1 class="h1">Bibliotecario → Registrar préstamo</h1>
        <p class="p">1) Buscar usuario · 2) Seleccionar · 3) Escanear
ejemplar</p>
    </div>
    <a class="btn" th:href="@{/menu}">← Volver</a>
</div>

<div class="divider"></div>

<div class="grid two">

    <!-- ===== Columna izquierda: buscar + lista + seleccionar ===== -->
    <div class="card pad">
        <h2 class="h2">1) Buscar usuario</h2>

        <form class="row" method="get" th:action="@{/admin/loan}">
            <input class="input"
                name="userQ"
                placeholder="Username / Email"
                th:value="${userQ}"
                style="flex:1;min-width:220px;">
            <button class="btn primary" type="submit">Buscar</button>
        </form>

        <div class="divider"></div>

        <h2 class="h2">2) Resultados</h2>

        <!-- Sin búsqueda todavía -->
        <div class="card pad" th:if="${userQ == null ||
#strings.isEmpty(userQ)}">
            <p class="p">Introduce un texto para buscar usuarios.</p>
        </div>

        <!-- Búsqueda sin resultados -->
        <div class="card pad" th:if="${userQ != null
&& !#strings.isEmpty(userQ) && (users == null || #lists.isEmpty(users))}">
            <p class="p">No se encontraron usuarios.</p>
        </div>
    </div>

```

```

        <!-- Lista de usuarios -->
        <div class="list" th:if="${users != null
&& !#lists.isEmpty(users)}">
            <article class="item" th:each="u : ${users}">
                <div class="thumb" aria-hidden="true"></div>

                <div class="meta">
                    <p class="title" th:text="${u.name}">Nombre
Apellido</p>

                    <p class="sub">
                        <span th:text="${u.email}">mail@upm.es</span>
                        <span th:if="${u.type != null}"> · </span>
                        <span th:if="${u.type != null}"
th:text="${u.type}">Estudiante</span>
                    </p>
                </div>

                <div class="right">
                    <!-- Seleccionar usuario -->
                    <form th:action="@{/admin/loan/select}" method="post"
style="margin:0;">

                        <input type="hidden"
th:name="${_csrf.parameterName}" th:value="${_csrf.token}"/>
                        <input type="hidden" name="userId"
th:value="${u.id}"/>

                        <!-- conserva el texto buscado para volver a la
lista si quieres -->
                        <input type="hidden" name="userQ"
th:value="${userQ}"/>

                        <button class="btn"
                                type="submit"
                                th:classappend="${selectedUser != null &&
selectedUser.id == u.id} ? ' primary' : ''">
                            <span th:text="${selectedUser != null &&
selectedUser.id == u.id} ? 'Seleccionado' : 'Seleccionar'">
                                Seleccionar
                            </span>
                        </button>
                    </form>
                </div>

```

```

        </article>
    </div>

    <div class="divider"></div>

    <h2 class="h2">Usuario seleccionado</h2>
    <div th:if="${selectedUser != null}">
        <p class="p">
            <strong th:text="${selectedUser.name}">María
López</strong>
            <span class="chip" th:if="${selectedUser.type != null}"
th:text="${selectedUser.type}">Estudiante</span>
        </p>
        <p class="p" th:if="${selectedUser.email != null}"
th:text="${selectedUser.email}">mail@upm.es</p>
    </div>
    <p class="p" th:if="${selectedUser == null}">Ningún usuario
seleccionado.</p>
</div>

<!-- ===== Columna derecha: escanear ejemplar (solo si hay usuario)
===== -->

<div class="card pad">
    <h2 class="h2">3) Escanear ejemplar</h2>

    <!-- Bloqueado si no hay usuario -->
    <div class="card pad" th:if="${selectedUser == null}">
        <p class="p">Primero selecciona un usuario para
continuar.</p>
    </div>

    <div th:if="${selectedUser != null}">
        <form method="post" th:action="@{/admin/loan/scan}"
class="field">
            <input type="hidden" th:name="${_csrf.parameterName}"
th:value="${_csrf.token}"/>
            <input type="hidden" name="userId"
th:value="${selectedUser.id}"/>

            <label class="label">Código de barras / ID
ejemplar</label>

```

```

        <input class="input" name="copyCode"
placeholder="Escanear..." autofocus />
        <button class="btn primary" type="submit">Buscar
ejemplar</button>

    </form>

    <div class="divider"></div>

    <!-- Detalles del ejemplar -->
    <div th:if="${copy != null}">
        <p class="p" th:text="'Ejemplar: ' +
${copy.id}">Ejemplar: 12345</p>
        <p class="p" th:text="'Estado: ' +
${copy.status}">Estado: Disponible</p>
        <p class="p" th:text="'Reserva para este usuario: ' +
(${copy.reservedForUser} ? 'Sí' : 'No')">
            Reserva para este usuario: Sí/No
        </p>
        <p class="p" th:text="'¿Referencia?: ' +
(${copy.reference} ? 'Sí' : 'No')">¿Referencia?: No</p>

        <div class="divider"></div>

        <!-- Confirmar préstamo -->
        <form method="post" th:action="@{/admin/loan/confirm}">
            <input type="hidden" th:name="${_csrf.parameterName}"
th:value="${_csrf.token}"/>
            <input type="hidden" name="userId"
th:value="${selectedUser.id}"/>
            <input type="hidden" name="copyId"
th:value="${copy.id}"/>

            <button class="btn primary" type="submit">Confirmar
préstamo</button>

        </form>
    </div>

    <p class="p" th:if="${copy == null}">Introduce o escanea un
ejemplar para ver detalles.</p>
    </div>
</div>

```

```

        </div>
    </section>

    <footer th:replace="{fragments/layout :: footer}"></footer>
</main>
</body>
</html>

```

Qué cambia a nivel backend (mínimo)

Solo para que encaje con el HTML, necesitarás endpoints como:

- GET /admin/loan?userQ=... → carga users y selectedUser (si lo guardas en sesión o lo pasas)
- POST /admin/loan/select → recibe userId y deja “selectedUser” (yo lo guardaría en sesión o redirigiría con query param)
- POST /admin/loan/scan → recibe userId + copyCode y devuelve copy
- POST /admin/loan/confirm → recibe userId + copyId

Si no quieres sesión, lo más simple es que select y scan hagan redirect a: /admin/loan?userQ=...&selectedUserId=... y reconstruyes el modelo.

Si quieres, te dejo también el controller completo con ese enfoque (sin sesión).

You said:

si

ChatGPT said:

Perfecto — aquí tienes un **controller completo** para ese admin-loan.html **sin usar sesión**, usando solo **redirects con query params** (userQ, selectedUserId, copyId) para reconstruir el modelo en cada GET.

La idea:

- **GET /admin/loan**
Renderiza la página, carga:
 - lista users si hay userQ
 - selectedUser si hay selectedUserId
 - copy si hay copyId

- **POST** /admin/loan/select
Recibe userId y redirige a GET con selectedUserId.
- **POST** /admin/loan/scan
Recibe copyCode y redirige a GET con copyId (si lo encuentra).
- **POST** /admin/loan/confirm
Crea el préstamo (y redirige de vuelta con flash msg).
OJO: Ajusta repositorios/DTOs a tus clases reales (UserRepository, CopyRepository, etc.). Te dejo una estructura limpia.

Controller (sin sesión)

```
package com.tuapp.web;

import com.tuapp.domain.*; // ajusta
import org.springframework.http.HttpStatus;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.server.ResponseStatusException;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;

import java.util.List;

@Controller
@RequestMapping("/admin/loan")
@PreAuthorize("hasRole('LIBRARIAN')") // requiere que ya tengas ROLE_LIBRARIAN
mapeado
public class AdminLoanController {

    private final UserRepository userRepository;
    private final CopyRepository copyRepository;
    private final LoanService loanService;

    public AdminLoanController(UserRepository userRepository,
                               CopyRepository copyRepository,
                               LoanService loanService) {
        this.userRepository = userRepository;
        this.copyRepository = copyRepository;
    }
}
```

```

        this.loanService = loanService;
    }

    /**
     * Pantalla única (render HTML).
     * Params:
     * - userQ: texto de búsqueda
     * - selectedUserId: usuario seleccionado
     * - copyId: ejemplar encontrado/escaneado
     */
    @GetMapping
    public String page(@RequestParam(required = false) String userQ,
                      @RequestParam(required = false) Long selectedUserId,
                      @RequestParam(required = false) Long copyId,
                      Model model) {

        // 1) input buscado
        model.addAttribute("userQ", userQ);

        // 2) Lista de usuarios
        List<User> users = List.of();
        if (userQ != null && !userQ.isBlank()) {
            // Ajusta a tu método real: buscar por username/email, etc.
            users = userRepository.searchByUsernameOrEmail(userQ.trim());
        }
        model.addAttribute("users", users);

        // 3) usuario seleccionado
        User selectedUser = null;
        if (selectedUserId != null) {
            selectedUser = userRepository.findById(selectedUserId)
                .orElseThrow(() -> new
ResponseStatusException(HttpStatus.NOT_FOUND, "Usuario no encontrado"));
        }
        model.addAttribute("selectedUser", selectedUser);

        // 4) ejemplar escaneado/encontrado
        Copy copy = null;
        if (copyId != null) {
            copy = copyRepository.findById(copyId)
                .orElseThrow(() -> new

```

```

ResponseStatusException(HttpStatus.NOT_FOUND, "Ejemplar no encontrado"));
    }
    model.addAttribute("copy", copy);

    return "admin-loan";
}

/**
 * Seleccionar usuario desde la lista.
 */
@PostMapping("/select")
public String selectUser(@RequestParam Long userId,
                        @RequestParam(required = false) String userQ,
                        RedirectAttributes ra) {

    ra.addAttribute("userQ", userQ);
    ra.addAttribute("selectedUserId", userId);
    // al seleccionar usuario, limpiamos el ejemplar si había uno previo
    // (si quieres mantenerlo, añade copyId)
    return "redirect:/admin/loan";
}

/**
 * Escanear/introducir ejemplar (copyCode).
 * Aquí lo convertimos a copyId y redirigimos a GET para renderizar.
 */
@PostMapping("/scan")
public String scanCopy(@RequestParam Long userId,
                      @RequestParam(required = false) String userQ,
                      @RequestParam String copyCode,
                      RedirectAttributes ra) {

    if (copyCode == null || copyCode.isBlank()) {
        ra.addFlashAttribute("toastErr", "Introduce un código de ejemplar.");
        ra.addAttribute("userQ", userQ);
        ra.addAttribute("selectedUserId", userId);
        return "redirect:/admin/loan";
    }

    // Si tu copyCode ES el ID numérico:
    Long copyId;

```



```

        try {
            copyId = Long.parseLong(copyCode.trim());
        } catch (NumberFormatException e) {
            // Si copyCode no es numérico y tienes otro campo (barcode), busca
            por barcode:
            // Copy copy =
            copyRepository.findByBarcode(copyCode.trim()).orElse(null);
            // if (copy == null) { ... }
            ra.addFlashAttribute("toastErr", "El código no es válido.");
            ra.addAttribute("userQ", userQ);
            ra.addAttribute("selectedUserId", userId);
            return "redirect:/admin/loan";
        }

        // valida que exista (para poder mostrar detalles)
        boolean exists = copyRepository.existsById(copyId);
        if (!exists) {
            ra.addFlashAttribute("toastErr", "No se encontró el ejemplar " +
copyId);
            ra.addAttribute("userQ", userQ);
            ra.addAttribute("selectedUserId", userId);
            return "redirect:/admin/loan";
        }

        ra.addAttribute("userQ", userQ);
        ra.addAttribute("selectedUserId", userId);
        ra.addAttribute("copyId", copyId);
        return "redirect:/admin/loan";
    }

    /**
     * Confirmar préstamo.
     */
    @PostMapping("/confirm")
    public String confirm(@RequestParam Long userId,
                        @RequestParam Long copyId,
                        @RequestParam(required = false) String userQ,
                        RedirectAttributes ra) {
        try {
            loanService.createLoan(userId, copyId); // tu lógica
            ra.addFlashAttribute("toastOk", "Préstamo registrado ");
        }
    }

```

```

        // después de confirmar, puedes limpiar selección o mantener user.
        // Yo mantendría el usuario y limpiaría el ejemplar:
        ra.addAttribute("userQ", userQ);
        ra.addAttribute("selectedUserId", userId);
        return "redirect:/admin/loan";
    } catch (Exception ex) {
        ra.addFlashAttribute("toastErr", ex.getMessage() != null ?
ex.getMessage() : "No se pudo registrar el préstamo.");
        ra.addAttribute("userQ", userQ);
        ra.addAttribute("selectedUserId", userId);
        ra.addAttribute("copyId", copyId);
        return "redirect:/admin/loan";
    }
}
}
}

```

Repositorios: ejemplos de métodos

Tú adapta a tu modelo real.

```

public interface UserRepository extends JpaRepository<User, Long> {
    // ejemplo: ajusta query según tu schema
    @Query("""
        select u from User u
        where lower(u.email) like lower(concat('%', :q, '%'))
        or lower(u.username) like lower(concat('%', :q, '%'))
        or lower(u.name) like lower(concat('%', :q, '%'))
    """)
    List<User> searchByUsernameOrEmail(@Param("q") String q);
}

```

Y CopyRepository:

```

public interface CopyRepository extends JpaRepository<Copy, Long> {
    // si tu "copyCode" es barcode:
    // Optional<Copy> findByBarcode(String barcode);
}

```

En tu HTML: pequeño ajuste recomendado (para conservar userQ)

En los forms POST (/scan y /confirm) mete userQ oculto para que no se pierda al redirigir.

En scan ya puse userId. Añade:

```
<input type="hidden" name="userQ" th:value="${userQ}">
```

Y en confirm:

```
<input type="hidden" name="userQ" th:value="${userQ}">
```

Así el buscador se mantiene.

Mensajes (toastOk/toastErr)

Si ya usas RedirectAttributes, puedes pintarlos en tu layout o en la página:

```
<div th:if="${toastOk}" class="chip ok" th:text="${toastOk}"></div>
<div th:if="${toastErr}" class="chip bad" th:text="${toastErr}"></div>
```

Ventajas de este enfoque (sin sesión)

- Muy fácil de depurar: la URL refleja el estado (selectedUserId, copyId)
- Sin estado servidor
- Re-entradas y refresh funcionan sin problemas

Si me dices cómo es tu User (campos: name, email, type, etc.) y si copyCode es ID o barcode real, te ajusto los queries/métodos exactamente a tu modelo.

You said:



tengo ese controller y el html se llama admin-loan.html. pero al entrar en la pagina me da error 404 not found. la direccion que aparece en el explorador es correcta

<http://localhost:8080/admin/loan>

ChatGPT said:

Viendo tu captura, la ruta del método **no coincide** con la URL que estás abriendo.

Tú estás abriendo:

```
/admin/loan
```

Pero tu controller tiene:

- A nivel clase: `@RequestMapping("/admin")`
 - A nivel método: `@GetMapping("/loan")`
- ✓ Eso **sí debería** mapear exactamente a `/admin/loan`.

Así que si te da 404, casi siempre es por **una de estas 4 causas reales** (te marco la más probable primero, y te doy cómo comprobarlo sin adivinar):

1) Estás arrancando con el profile aws y tu SecurityConfig solo existe con ese profile (y quizá el resto no)

Esto ya te pasó antes con `@Profile("aws")`. Si tu `@SpringBootApplication` está en un paquete que no escanea el de `LoanAdminController`, también da 404.

✓ Comprueba esto en 10 segundos

En consola, al arrancar, mira si sale algo como:

- Tomcat started on port(s): 8080
- y luego entra a:

<http://localhost:8080/actuator/mappings>

(si tienes actuator) o abre el panel **Mappings** del IDE (en tu captura abajo aparece “Mappings”).

Si **NO aparece** `/admin/loan` en mappings, el controller **no se está registrando**.

2) Tu LoanAdminController está en un paquete fuera del component scan

Esta es la causa nº1 de 404 cuando “todo parece correcto”.

Spring solo escanea paquetes **a partir del paquete donde está tu clase main** (la que tiene `@SpringBootApplication`).

Ejemplo:

- Main: `com.app.LibraryApplication`
- Controller: `es.upm.web.LoanAdminController` ✗ (no lo escanea)

✓ Solución

Mueve el controller al mismo paquete o subpaquete del main, o fuerza el scan:

```
@SpringBootApplication(scanBasePackages = "tu.paquete.base")
public class LibraryApplication { ... }
```

O pon el main en un paquete “raíz” superior.

3) Tu método está anotado con @GetMapping("/loan") (ojo con espacios/typos) o no se compila

En tu screenshot parece @GetMapping("/loan"), pero asegúrate de que no haya:

- comillas raras
- espacios invisibles
- imports incorrectos (muy raro, pero pasa)

✓ En IntelliJ: **Navigate** → **Declaration** sobre @GetMapping y verifica que es `org.springframework.web.bind.annotation.GetMapping`.

4) Spring Security te está bloqueando y tú lo ves como "404"

Normalmente Security devuelve **302 a Cognito** o **403**, pero dependiendo de tu config puede parecer 404.

En tu SecurityFilterChain (captura anterior) tienes:

```
.requestMatchers("/admin/**").hasRole("LIBRARIAN")
```

Si tus roles aún no se están mapeando a `ROLE_LIBRARIAN`, entonces `/admin/loan` no te deja entrar.

Pero normalmente eso no es 404.

✓ Para descartar esto rápido:
temporalmente permite `/admin/loan`:

```
.requestMatchers("/admin/loan").permitAll()
```

Si con eso "aparece", el problema era roles/security.

La comprobación definitiva (sin especular)

Entra en el panel "Mappings" (lo tienes en tu IDE, lo veo en las capturas anteriores) y busca `loan`.

- Si no existe `/admin/loan` → controller no cargado (causa 1 o 2)
- Si existe pero al abrir te falla → es security / redirect

También revisa el nombre de la vista

Tu método devuelve:

```
return "admin-loan";
```

Y tu plantilla debe estar aquí:

```
src/main/resources/templates/admin-loan.html
```

✓ Eso lo tienes bien.

Cambios mínimos recomendados

1. Asegúrate de que el controller sea detectado (package):
 - a. Tu clase main debe estar en un paquete padre.
2. Si quieres evitar dudas, puedes hacer el mapping explícito en una sola anotación:

```
@GetMapping("/admin/loan")
public String createLoan(...) { ... }

(sin @RequestMapping("/admin"))
```

Esto no arregla component-scan, pero evita confusiones.

Dime esto y te lo clavo en 1 mensaje

¿Cuál es el **package** de:

1. tu clase LibraryApplication (la main con @SpringBootApplication)
2. tu LoanAdminController

Con eso te digo al instante si es el problema de component scan (que es lo más probable cuando todo parece correcto y aún así es 404).

You said:

```
You said: @GetMapping("/loan") public String createLoan(@RequestParam(required =
false, name = "userQ") String userQ, @RequestParam(required = false, name = "userId")
Long userId, @RequestParam(required = false, name = "copyCode") Long copyId, Model
model) { model.addAttribute("userQ", userQ); model.addAttribute("users", (userQ == null
|| userQ.isBlank()) ? List.of() :
loanService.searchUsers(userQ).stream().map(UserSummaryDto::from).toList());
if(userId != null) { model.addAttribute("selectedUser",
UserSummaryDto.from(userRepository.findById(userId).orElseThrow(() -> new
NotFoundException("User not found.")))); } if(copyId != null){ model.addAttribute("copy",
CopyDetailsDto.from(copyRepository.findById(copyId).orElseThrow(() -> new
NotFoundException("User not found."))));
if(reservationRepository.findByIdAndCopyId(userId, copyId).isEmpty())
model.addAttribute("reservedForUser", false); else model.addAttribute("reservedForUser",
true); } return "admin-loan"; } como mejorarias eso
```

ChatGPT said:

Tu método funciona, pero ahora mismo está mezclando **demasiadas responsabilidades** en un solo handler y además tiene varios riesgos (NPE, error messages incorrectos, varias llamadas a repo, etc.). Yo lo mejoraría así:

Problemas concretos del código actual

- userRepository, copyRepository, reservationRepository aparecen dentro del controller: el controller queda “gordo”. Mejor que el controller solo coordine y el servicio haga la lógica.
- copyCode lo tipas como Long copyId: si llega vacío o no numérico, revienta. Mejor recibir String copyCode y parsear/validar.
- Si copyId != null estás usando userId sin comprobar que no sea null → posible NPE en findByUserIdAndCopyId(userId, copyId).
- NotFoundException("User not found.") también para copy: mensaje incorrecto.
- Para saber si hay reserva haces findBy...().isEmpty(): mejor existsBy... (más eficiente).
- reservedForUser lo metes suelto en el modelo cuando ya tienes copy: mejor que CopyDetailsDto incluya ese flag (reservedForSelectedUser) y solo pasas un objeto.

Versión mejorada (misma idea, más limpia y robusta)

1) Controller “thin”

```
@GetMapping("/loan")
public String createLoan(
    @RequestParam(required = false) String userQ,
    @RequestParam(required = false) Long userId,
    @RequestParam(required = false) String copyCode,
    Model model
) {
    AdminLoanView vm = loanService.buildAdminLoanView(userQ, userId, copyCode);

    model.addAttribute("userQ", vm.userQ());
    model.addAttribute("users", vm.users());
    model.addAttribute("selectedUser", vm.selectedUser());
    model.addAttribute("copy", vm.copy()); // aquí ya viene el flag
    reservedForSelectedUser

    return "admin-loan";
}
```

2) Un “view model” para la pantalla

```
public record AdminLoanView(  
    String userQ,  
    List<UserSummaryDto> users,  
    UserSummaryDto selectedUser,  
    CopyDetailsDto copy  
) {}
```

3) Lógica en el service (donde debe estar)

```
public AdminLoanView buildAdminLoanView(String userQ, Long userId, String  
copyCode) {  
    String q = (userQ == null) ? null : userQ.trim();  
  
    List<UserSummaryDto> users =  
        (q == null || q.isBlank())  
        ? List.of()  
        : searchUsers(q).stream().map(UserSummaryDto::from).toList();  
  
    UserSummaryDto selectedUser = null;  
    if (userId != null) {  
        var user = userRepository.findById(userId)  
            .orElseThrow(() -> new NotFoundException("User not found."));  
        selectedUser = UserSummaryDto.from(user);  
    }  
  
    CopyDetailsDto copy = null;  
    if (copyCode != null && !copyCode.isBlank()) {  
        Long copyId = parseCopyCodeToId(copyCode); // validación centralizada  
  
        var copyEntity = copyRepository.findById(copyId)  
            .orElseThrow(() -> new NotFoundException("Copy not found."));  
  
        boolean reservedForSelectedUser = false;  
        if (userId != null) {  
            reservedForSelectedUser = reservationRepository  
                .existsByUserIdAndCopyIdAndStatus(userId, copyId,  
ReservationStatus.ACTIVE);  
        }  
  
        copy = CopyDetailsDto.from(copyEntity, reservedForSelectedUser);  
    }  
}
```



```

    }

    return new AdminLoanView(q, users, selectedUser, copy);
}

private Long parseCopyCodeToId(String copyCode) {
    try {
        return Long.parseLong(copyCode.trim());
    } catch (NumberFormatException e) {
        throw new BadRequestException("Invalid copy code.");
    }
}
}

```

4) Mejorar el repo para no cargar reservas completas

En ReservationRepository:

```
boolean existsByUserIdAndCopyIdAndStatus(Long userId, Long copyId,
ReservationStatus status);
```

Y ya no necesitas `findBy...().isEmpty()`.

5) Meter el flag dentro del DTO del ejemplar

```

public record CopyDetailsDto(Long id, String status, boolean reference, boolean
reservedForSelectedUser) {
    public static CopyDetailsDto from(Copy c, boolean reserved) {
        return new CopyDetailsDto(c.getId(), c.getStatus().name(),
c.isReference(), reserved);
    }
}

```

En el HTML usarías:

```
<p class="p" th:text="'Reserva para este usuario: ' +
(${copy.reservedForSelectedUser} ? 'Sí' : 'No')"></p>
```

Bonus: separa “leer pantalla” de “acciones”

Este GET está bien para **renderizar**.

Pero las acciones deberían ser POST:

- seleccionar usuario
- escanear ejemplar
- confirmar préstamo

Tú querías hacerlo sin sesión: perfecto. El patrón ideal es:

- GET /admin/loan?userQ=...&userId=...©Code=... → render
- POST /admin/loan/select → redirect al GET con userId
- POST /admin/loan/scan → redirect al GET con copyCode
- POST /admin/loan/confirm → crea el préstamo y redirect

Así el GET queda solo “lectura” y no hace side-effects.

Cambios mínimos si no quieres refactor grande

Si quieres mejorar sin crear VM/service:

1. Cambia copyCode a String + parse seguro
2. Comprueba userId != null antes del repo de reservas
3. Usa existsBy...
4. Mensajes correctos

Ejemplo mínimo:

```
@RequestParam(required=false) String copyCode

if(copyCode != null && !copyCode.isBlank()){
    Long copyId = Long.parseLong(copyCode.trim());
    ...
    boolean reservedForUser = userId != null &&
reservationRepository.existsByIdAndCopyId(userId, copyId);
    model.addAttribute("reservedForUser", reservedForUser);
}
```

Si me dices si copyCode es siempre numérico (id) o puede ser un “barcode” alfanumérico, te ajusto el parseCopyCodeToId y el repo (findByBarcode) para que sea realista.

You said:

quiero que me hagas el endpoint para guardar el prestamo y que eso lo visualice de alguna forma el bilbiotecario, si se ha guardado correctamente o no. Me da igual que sea en otra pagina de html o una ventana emergente, lo que mejor te parezca.

ChatGPT said:

Vale. Te propongo la opción más simple y “Spring/Thymeleaf-friendly”: **POST que crea el préstamo + mensaje flash** (éxito/error) que se muestra arriba en la misma página admin-loan.html. No necesitas JS ni popups raros, y queda muy limpio.

Abajo te dejo:

1. Endpoint POST /admin/loan/confirm (guardar préstamo)
2. Ejemplo de `LoanService.createLoan(...)` con validaciones mínimas
3. Cambios en `admin-loan.html` para mostrar mensaje (tipo “toast”)
4. (Opcional) Evitar doble submit

1) Endpoint para guardar el préstamo

```
@PostMapping("/loan/confirm")
public String confirmLoan(
    @RequestParam Long userId,
    @RequestParam Long copyId,
    @RequestParam(required = false, name = "userQ") String userQ,
    RedirectAttributes ra
) {
    try {
        Loan loan = loanService.createLoan(userId, copyId);

        ra.addFlashAttribute("toastOk",
            "Préstamo registrado  (Usuario " + userId + ", Ejemplar " +
copyId + ")");
        // volvemos a la misma pantalla manteniendo búsqueda y usuario
seleccionado
        ra.addAttribute("userQ", userQ);
        ra.addAttribute("userId", userId);
        // opcional: limpiar copyId para “nuevo escaneo”
        // ra.addAttribute("copyId", copyId);
        return "redirect:/admin/loan";

    } catch (Exception ex) {
        ra.addFlashAttribute("toastErr",
            (ex.getMessage() != null && !ex.getMessage().isBlank())
            ? ex.getMessage()
            : "No se pudo registrar el préstamo.");
        ra.addAttribute("userQ", userQ);
        ra.addAttribute("userId", userId);
        ra.addAttribute("copyId", copyId); // mantenemos el ejemplar para que lo
vea
        return "redirect:/admin/loan";
    }
}
```

Nota: como tu clase es `@RequestMapping("/admin")`, la ruta final es `/admin/loan/confirm`.

2) Servicio createLoan (ejemplo típico)

Ajusta nombres/repos a los tuyos.

```
@Transactional
public Loan createLoan(Long userId, Long copyId) {

    var user = userRepository.findById(userId)
        .orElseThrow(() -> new NotFoundException("Usuario no encontrado."));

    var copy = copyRepository.findById(copyId)
        .orElseThrow(() -> new NotFoundException("Ejemplar no encontrado."));

    if (copy.isReference()) {
        throw new IllegalStateException("No se puede prestar un ejemplar de referencia.");
    }

    if (!copy.getStatus().equals(CopyStatus.AVAILABLE)) {
        throw new IllegalStateException("El ejemplar no está disponible.");
    }

    // si tienes reservas: permitir solo si está reservado por ese usuario
    // (opcional)
    boolean reservedForUser = reservationRepository
        .existsByUserIdAndCopyIdAndStatus(userId, copyId,
ReservationStatus.ACTIVE);

    boolean hasActiveReservationForCopy = reservationRepository
        .existsByCopyIdAndStatus(copyId, ReservationStatus.ACTIVE);

    if (hasActiveReservationForCopy && !reservedForUser) {
        throw new IllegalStateException("El ejemplar está reservado para otro usuario.");
    }

    // evita doble préstamo del mismo ejemplar
```

```

    if (loanRepository.existsByCopyIdAndStatus(copyId, LoanStatus.ACTIVE)) {
        throw new IllegalStateException("El ejemplar ya está prestado.");
    }

    // crear préstamo
    Loan loan = new Loan();
    loan.setUser(user);
    loan.setCopy(copy);
    loan.setStatus(LoanStatus.ACTIVE);
    loan.setCreatedAt(OffsetDateTime.now());
    loan.setDueAt(OffsetDateTime.now().plusDays(14)); // o la regla que uses

    Loan saved = loanRepository.save(loan);

    // actualizar estado ejemplar
    copy.setStatus(CopyStatus.LOANED);
    copyRepository.save(copy);

    // opcional: si había reserva del mismo usuario, marcarla FULFILLED
    reservationRepository.findActiveByUserIdAndCopyId(userId, copyId)
        .ifPresent(r -> {
            r.setStatus(ReservationStatus.FULFILLED);
            reservationRepository.save(r);
        });

    return saved;
}

```

3) Mostrar resultado al bibliotecario en admin-loan.html

Añade esto al principio del <section class="card pad"> (justo después del título, por ejemplo):

```

<div th:if="${toastOk}" class="card pad" style="border-color: rgba(2,122,72,.35);
background: rgba(2,122,72,.08);">
    <p class="p" th:text="${toastOk}">OK</p>
</div>

<div th:if="${toastErr}" class="card pad" style="border-color:
rgba(180,35,24,.35); background: rgba(180,35,24,.08);">
    <p class="p" th:text="${toastErr}">Error</p>
</div>

```

```
<div class="divider" th:if="${toastOk || toastErr}"></div>
```

Con esto:

- Si se guarda bien: mensaje verde ✓
- Si falla: mensaje rojo ✗
- No necesitas JS y es muy usable.

4) El formulario de confirmar (asegúrate de mandar userId + copyId + CSRF)

En tu HTML:

```
<form method="post" th:action="@{/admin/loan/confirm}">
  <input type="hidden" th:name="${_csrf.parameterName}"
th:value="${_csrf.token}"/>
  <input type="hidden" name="userId" th:value="${selectedUser.id}"/>
  <input type="hidden" name="copyId" th:value="${copy.id}"/>
  <input type="hidden" name="userQ" th:value="${userQ}"/>

  <button class="btn primary" type="submit">Confirmar préstamo</button>
</form>
```

Recomendación final

Para bibliotecarios, la UX más rápida es:

- Mensaje flash arriba
- Mantener usuario seleccionado
- Limpiar el ejemplar para escanear el siguiente

(Así procesan préstamos en cadena sin salir de la pantalla.)

Si me pasas tus entidades/repos (Loan, CopyStatus, LoanStatus, cómo guardas fechas), te adapto el createLoan exactamente a tu modelo para que compile tal cual.

You said:

```
2026-02-06T15:54:44.614+01:00 ERROR 1260 --- [library] [nio-8080-exec-7]
o.a.c.c.C.[.[./].[dispatcherServlet] : Servlet.service() for servlet [dispatcherServlet] in
context with path [] threw exception [Request processing failed:
org.thymeleaf.exceptions.TemplateInputException: An error happened during template
parsing (template: "class path resource [templates/admin-loan.html]] with root cause
```

org.springframework.expression.spel.SpelEvaluationException: EL1001E: Type conversion problem, cannot convert from null to boolean at
org.springframework.expression.spel.ast.OpOr.assertValueNotNull(OpOr.java:71)
~[spring-expression-7.0.2.jar:7.0.2]

ChatGPT said:

Ese error viene de Thymeleaf/SpringEL cuando haces un **OR (| |)** y uno de los operandos es null, y Spring intenta convertir ese null a boolean → peta con:

cannot convert from null to boolean

OpOr...

Qué lo provoca típicamente

Alguna expresión en tu `admin-loan.html` como:

```
th:if="${toastOk || toastErr}"
```

o

```
th:if="${copy.reservedForUser || ...}"
```

donde `toastOk`, `toastErr` (o cualquier variable) puede ser null.

En SpringEL, **|| requiere booleanos**, y null no se convierte a boolean automáticamente.

Arreglo rápido (recomendado): compara contra null

Cambia cualquier OR tipo “variable suelta” por checks explícitos:

✓ En vez de:

```
<div th:if="${toastOk || toastErr}"></div>
```

usa:

```
<div th:if="${toastOk != null || toastErr != null}"></div>
```

O si son strings, mejor:

```
<div th:if="${toastOk != null || toastErr != null}"></div>
```

(Con flash attributes normalmente llegan como String o no llegan)

Si es para strings “vacíos”

Puedes hacerlo más robusto así:

```
<div th:if="${toastOk != null and !#strings.isEmpty(toastOk) || toastErr != null and !#strings.isEmpty(toastErr)}"></div>
```

Si el OR está en otro sitio (ej. `copy.reservedForUser`)

Si tienes algo como:

```
th:if="${copy.reservedForUser || copy.reference}"
```

y `copy` puede ser null, también revienta.

✅ Forma segura:

```
th:if="${copy != null and (copy.reservedForUser or copy.reference)}"
```

(en Thymeleaf puedes usar `or` además de `||`)

Cómo localizar exactamente la línea

En tu log sale “template parsing”, pero no la línea.

Revisa en `admin-loan.html` cualquier `th:if` que tenga:

- `||` o `or`
- variables que pueden ser null (`toastOk`, `toastErr`, `copy`, `selectedUser`, etc.)

El culpable más común es este:

```
th:if="${toastOk || toastErr}"
```

Recomendación para tu caso (mensajes flash)

Usa esto:

```
<div th:if="${toastOk != null}" class="card pad" style="border-color:
rgba(2,122,72,.35); background: rgba(2,122,72,.08);">
  <p class="p" th:text="${toastOk}">OK</p>
</div>

<div th:if="${toastErr != null}" class="card pad" style="border-color:
rgba(180,35,24,.35); background: rgba(180,35,24,.08);">
  <p class="p" th:text="${toastErr}">Error</p>
</div>

<div class="divider" th:if="${toastOk != null || toastErr != null}"></div>
```

Con eso no hay conversión a boolean de null.

Si me pegas el bloque de `admin-loan.html` donde usas `th:if` con `||`/or, te digo exactamente cuál es la expresión que está rompiendo y la dejo corregida.

You said:

necesito la pagina de admin return

ChatGPT said:

Aquí tienes una **página nueva** para “Admin → Return” (devolución de préstamo) siguiendo el mismo estilo que `admin-loan.html` y pensada para un flujo típico:

1. Escanear/introducir **ID del ejemplar** (o código)
2. Mostrar detalles del préstamo activo (usuario, libro, fechas, estado, retraso)
3. Botón **Confirmar devolución** (POST)
4. Mensaje de éxito/error (flash `toastOk/toastErr`)

Guárdala como:

`src/main/resources/templates/admin-return.html`

```
<!-- src/main/resources/templates/admin-return.html -->
<!doctype html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="utf-8"/>
  <meta name="viewport" content="width=device-width,initial-scale=1"/>
  <title>Registrar devolución</title>
  <link rel="stylesheet" th:href="@{/css/app.css}">
</head>
<body>
<header th:replace="~{fragments/layout :: topbar(${userName},
${role})}"></header>

<main class="container">
  <section class="card pad">
    <div class="row spread">
      <div>
        <h1 class="h1">Bibliotecario → Registrar devolución</h1>
        <p class="p">1) Escanear ejemplar · 2) Revisar préstamo activo · 3)
Confirmar devolución</p>
        </div>
        <a class="btn" th:href="@{/menu}">← Volver</a>
      </div>

      <!-- Mensajes -->
      <div class="divider"></div>
```

```

<div th:if="${toastOk != null}" class="card pad"
      style="border-color: rgba(2,122,72,.35); background:
rgba(2,122,72,.08);">
  <p class="p" th:text="${toastOk}">Devolución registrada ✓</p>
</div>

<div th:if="${toastErr != null}" class="card pad"
      style="border-color: rgba(180,35,24,.35); background:
rgba(180,35,24,.08);">
  <p class="p" th:text="${toastErr}">Error</p>
</div>

<div class="divider" th:if="${toastOk != null || toastErr != null}"></div>

<div class="grid two">
  <!-- Columna izquierda: escaneo -->
  <div class="card pad">
    <h2 class="h2">1) Escanear ejemplar</h2>

    <form method="get" th:action="@{/admin/return}" class="field">
      <label class="label">Código de barras / ID ejemplar</label>
      <input class="input" name="copyCode" placeholder="Escanear..."
th:value="${copyCode}" autofocus />
      <button class="btn primary" type="submit">Buscar préstamo</button>
    </form>

    <div class="divider"></div>

    <p class="p">
      Introduce el identificador del ejemplar para localizar el préstamo
      activo asociado.
    </p>
  </div>

  <!-- Columna derecha: detalle préstamo + confirmar -->
  <div class="card pad">
    <h2 class="h2">2) Préstamo activo</h2>

    <!-- Si NO hay préstamo cargado -->

```

```

<div th:if="${loan == null}" class="card pad">
  <p class="p">Escanea un ejemplar para ver el préstamo activo.</p>
</div>

<!-- Si hay préstamo -->
<div th:if="${loan != null}">
  <!-- Datos principales -->
  <table class="table">
    <tbody>
      <tr>
        <th>Ejemplar</th>
        <td th:text="${loan.copyId}">12345</td>
      </tr>
      <tr>
        <th>Libro</th>
        <td>
          <div>
            <strong th:text="${loan.bookTitle}">Título del libro</strong>
          </div>
          <div class="p" style="margin-top:4px;">
            <span th:text="${loan.bookAuthor}">Autor</span>
          </div>
        </td>
      </tr>
      <tr>
        <th>Usuario</th>
        <td>
          <div>
            <strong th:text="${loan.userName}">Usuario</strong>
          </div>
          <div class="p" style="margin-top:4px;">
            <span th:text="${loan.userEmail}">email@upm.es</span>
          </div>
        </td>
      </tr>
      <tr>
        <th>Inicio</th>
        <td th:text="${loan.createdAt}">2026-02-01</td>
      </tr>
      <tr>
        <th>Vence</th>
        <td th:text="${loan.dueAt}">2026-02-15</td>
      </tr>
    </tbody>
  </table>

```

```

        </tr>
        <tr>
            <th>Retraso</th>
            <td>
                <span class="chip ok" th:if="${loan.overdueDays != null &&
loan.overdueDays == 0}">Sin retraso</span>
                <span class="chip bad" th:if="${loan.overdueDays != null &&
loan.overdueDays > 0}"
                    th:text="${loan.overdueDays} + ' días'">3 días</span>
                <span class="chip" th:if="${loan.overdueDays == null}">—</span>
            </td>
        </tr>
    </tbody>
</table>

<div class="divider"></div>

<h2 class="h2">3) Confirmar devolución</h2>

<form method="post" th:action="@{/admin/return/confirm}" class="row"
style="gap:12px;">
    <input type="hidden" th:name="${_csrf.parameterName}"
th:value="${_csrf.token}"/>

    <!-- Reenvío de datos mínimos -->
    <input type="hidden" name="loanId" th:value="${loan.id}"/>
    <input type="hidden" name="copyCode" th:value="${copyCode}"/>

    <button class="btn primary" type="submit"
        onclick="return confirm('¿Confirmar devolución del
ejemplar?');">
        Confirmar devolución
    </button>

    <a class="btn" th:href="@{/admin/return}">Limpiar</a>
</form>

<p class="p" style="margin-top:10px;">
    Al confirmar, el ejemplar pasará a <strong>disponible</strong> y el
préstamo quedará marcado como <strong>devuelto</strong>.
</p>

```

```

        </div>
    </div>
</div>
</section>

<footer th:replace="{fragments/layout :: footer}"></footer>
</main>
</body>
</html>

```

Variables que espera esta vista

- copyCode (String) → lo que escanea el bibliotecario
- loan (objeto/DTO) o null, con campos típicos:
 - id
 - copyId
 - bookTitle, bookAuthor
 - userName, userEmail
 - createdAt, dueAt
 - overdueDays (Integer)
- toastOk / toastErr (String) → flash attributes

Si quieres, también te dejo el **controller completo** (GET /admin/return + POST /admin/return/confirm) igual que el de préstamos: sin sesión y con RedirectAttributes para mostrar el resultado.

You said:

```
// If RESERVED, MVP accepts it (real rule may require "reserved by same user") if
(copy.getStatus() != CopyStatus.AVAILABLE && copy.getStatus() != CopyStatus.RESERVED)
{ throw new BusinessException("Copy cannot be loaned in its current status."); } tengo
esta condicion en prestamos quiero que Se pueda prestar si el estado es available o si el
status esta reserved y el usuario q ue ha reservado es el mismo que lo intenta coger
prestado
```

ChatGPT said:

Perfecto, esa regla está muy bien definida 👍
 Vamos a dejarla **correcta, clara y a prueba de bugs.**

Regla de negocio que quieres

Un ejemplar **se puede prestar** si:

1. Está AVAILABLE, o
2. Está RESERVED y la reserva activa es del **mismo usuario** que intenta cogerlo

En cualquier otro caso → **error**.

Implementación recomendada (clara y explícita)

Asumo:

- userId = usuario que intenta el préstamo
- copyId = ejemplar
- ReservationStatus.ACTIVE = reserva válida

Código final (service)

```
// AVAILABLE → OK
if (copy.getStatus() == CopyStatus.AVAILABLE) {
    // permitido
}

// RESERVED → solo si está reservado por el mismo usuario
else if (copy.getStatus() == CopyStatus.RESERVED) {

    boolean reservedBySameUser =
        reservationRepository.existsByUserIdAndCopyIdAndStatus(
            userId,
            copy.getId(),
            ReservationStatus.ACTIVE
        );

    if (!reservedBySameUser) {
        throw new BusinessException(
            "Copy is reserved for another user."
        );
    }
}

// Cualquier otro estado → NO
else {
    throw new BusinessException(
```

```

        "Copy cannot be loaned in its current status."
    );
}

```

Versión compacta (misma lógica, menos legible)

Si lo quieres en una sola condición:

```

boolean canLoan =
    copy.getStatus() == CopyStatus.AVAILABLE
    || (copy.getStatus() == CopyStatus.RESERVED
        && reservationRepository.existsByUserIdAndCopyIdAndStatus(
            userId,
            copy.getId(),
            ReservationStatus.ACTIVE
        ));

if (!canLoan) {
    throw new BusinessException("Copy cannot be loaned in its current
status.");
}

```

Funciona igual, pero la versión “explícita” es mejor para mantenimiento.

Importante: qué NO hacer ✕

No hagas esto:

```

if (copy.getStatus() != AVAILABLE && copy.getStatus() != RESERVED) { ... }

```

porque:

- permitiría prestar copias RESERVED **por cualquier usuario**
- rompe la regla real de negocio

Bonus (muy recomendable)

1 Asegura consistencia con la base de datos

Para evitar carreras:

- pon el método en `@Transactional`
- bloquea el ejemplar (si usas JPA):

```

@Lock(LockModeType.PESSIMISTIC_WRITE)
Optional<Copy> findById(Long id);

```

2 Mensajes de error más claros

En vez de uno genérico:

```
throw new BusinessException("Copy cannot be loaned in its current status.");
```

usa mensajes distintos:

- “Copy is reserved for another user”
- “Copy is already loaned”
- “Copy is unavailable”

Eso ayuda mucho al bibliotecario.

3 Qué hacer después de prestar una copia reservada

Si el usuario era el que reservó:

- marca la reserva como FULFILLED
- cambia el estado del ejemplar a LOANED

Resumen

✓ AVAILABLE → se puede prestar

✓ RESERVED + reservado por el mismo usuario → se puede prestar

✗ RESERVED + otro usuario → NO

✗ cualquier otro estado → NO

Si quieres, pásame tu método completo `createLoan(...)` y te lo dejo **cerrado y consistente** con reservas, estados y transacciones.

You said:

quiero añadir "Cola" en la reserva de libros.

ChatGPT said:

Genial. “Cola” en reservas (waitlist) encaja perfecto en una biblioteca. Te propongo un diseño **simple (MVP) pero sólido**, que no te obligue a reescribir media app y que evite reservas “fantasma”.

Qué significa “cola” aquí

Cuando un usuario reserva un libro:

- **Si hay algún ejemplar disponible** → se le asigna uno y queda **RESERVADO para ese usuario** (hold) con una **fecha límite** para recogerlo.

- Si no hay ejemplares disponibles → el usuario entra en **cola** y cuando se devuelve un ejemplar, se asigna automáticamente al primero de la cola.

1) Modelo de datos recomendado

Opción recomendada: cola por Libro (no por ejemplar)

La cola tiene sentido por libro: el usuario quiere “un ejemplar cualquiera” del libro.

Tabla reservations (o holds)

Campos típicos:

- id
- book_id ☒ (para la cola)
- copy_id (nullable) ☒ (se llena cuando ya se asigna ejemplar)
- user_id
- created_at
- deadline (nullable; solo cuando hay copy asignado)
- status enum:
 - QUEUED (en cola, sin copy)
 - ACTIVE (tiene copy asignado y está reservado)
 - FULFILLED (se convirtió en préstamo)
 - CANCELED
 - EXPIRED (se pasó deadline)

Reglas/constraints útiles

- Un usuario no puede tener dos reservas vivas del mismo libro:
 - Unique lógico: (user_id, book_id) para status en {QUEUED, ACTIVE}
 - Si tu DB no soporta índice parcial, lo controlas en código.

2) Flujo de negocio

A) Reservar un libro (entra o asigna)

POST /reservations (o tu confirm)

1. Validar usuario no sancionado / etc.
2. Si el usuario ya tiene QUEUED o ACTIVE para ese libro → error “ya estás en cola”
3. Buscar un ejemplar libre:
 - a. AVAILABLE y **no referencia**
4. Si existe:

- a. crear reserva con status=ACTIVE, copy_id=..., deadline=now+48h
- b. setear copy.status = RESERVED
5. Si no existe:
 - a. crear reserva con status=QUEUED, copy_id=null, deadline=null

B) Al devolver un ejemplar (return)

Cuando un copy pasa a AVAILABLE (o justo antes de marcarlo):

1. Buscar la **primera** reserva QUEUED del book_id (orden por created_at)
2. Si existe:
 - a. asignar ese copy_id a esa reserva
 - b. pasar reserva a ACTIVE
 - c. poner deadline = now + 48h
 - d. setear copy.status = RESERVED
3. Si no hay cola:
 - a. copy queda AVAILABLE

C) Expiración de reservas activas

Si se pasa el deadline:

1. Marca reserva EXPIRED
2. El ejemplar vuelve a AVAILABLE
3. Se intenta asignar al siguiente QUEUED (misma lógica que devolución)
(MVP: puedes hacerlo manual/endpoint admin o job programado)

3) Cambios mínimos en tu lógica actual

Ahora mismo tú estás reservando por **copy** (ejemplar) directamente. Para cola por libro, cambia a:

- Reservar desde /catalog/copies/{bookId} enviando bookId
- En “confirmar” creas **QUEUED** o **ACTIVE** dependiendo de disponibilidad

4) Código guía (service) — reserva con cola

Repos que te hacen la vida fácil

- CopyRepository.findFirstAvailableForBook(bookId)
- ReservationRepository.existsActiveOrQueued(userId, bookId)
- ReservationRepository.findFirstQueued(bookId)

Ejemplo de service (pseudo-Java muy cercano a real):

```

@Transactional
public Reservation reserveBook(Long userId, Long bookId) {

    if (reservationRepository.existsQueuedOrActiveByUserIdAndBookId(userId,
bookId)) {
        throw new BusinessException("Ya tienes una reserva/cola para este
libro.");
    }

    // 1) intenta asignar ejemplar
    Copy copy = copyRepository.findFirstReservableCopy(bookId) // AVAILABLE y no
referencia
        .orElse(null);

    Reservation r = new Reservation();
    r.setUserId(userId);
    r.setBookId(bookId);
    r.setCreatedAt(OffsetDateTime.now());

    if (copy != null) {
        r.setStatus(ReservationStatus.ACTIVE);
        r.setCopyId(copy.getId());
        r.setDeadline(OffsetDateTime.now().plusHours(48));
        copy.setStatus(CopyStatus.RESERVED);
        copyRepository.save(copy);
    } else {
        r.setStatus(ReservationStatus.QUEUED);
        r.setCopyId(null);
        r.setDeadline(null);
    }

    return reservationRepository.save(r);
}

```

5) Asignación de cola al devolver (return)

```

@Transactional
public void handleReturn(Long copyId) {

    Copy copy = copyRepository.findByIdForUpdate(copyId) // ideal con Lock
        .orElseThrow(() -> new NotFoundException("Copy not found"));
}

```

```

Long bookId = copy.getBookId();

// Marca copy retornado como AVAILABLE de momento
copy.setStatus(CopyStatus.AVAILABLE);

Reservation next = reservationRepository.findFirstQueuedForBookId(bookId)
    .orElse(null);

if (next == null) {
    copyRepository.save(copy);
    return;
}

// Asignar a siguiente de la cola
next.setStatus(ReservationStatus.ACTIVE);
next.setCopyId(copy.getId());
next.setDeadline(OffsetDateTime.now().plusHours(48));

copy.setStatus(CopyStatus.RESERVED);

reservationRepository.save(next);
copyRepository.save(copy);

// opcional: notificación/email
}

```

Importante: usa transacción y, si puedes, **locking** del ejemplar y/o de la cola para evitar carreras.

6) UI: qué mostrar al usuario

En /catalog/copies/{bookId}

- Botón: “Reservar”
- Si no hay disponibilidad: muestra “Entrar en cola”
- Si ya está en cola: “Ya estás en cola (posición X)”

Posición en cola (simple)

$pos = 1 + \text{count}(\text{QUEUED con created_at} < \text{la mía})$
(MVP: se calcula en query, no hace falta columna position)

En /my

Mostrar:

- Reservas **ACTIVE**: con deadline y copy asignado
- Reservas **QUEUED**: con posición estimada y fecha de solicitud

7) Tu regla actual de préstamo con RESERVED

Tu regla de préstamo ya se alinea con la cola:

- Si copy está **RESERVED**, **solo** lo puede prestar el usuario para quien está reservado (porque su reserva **ACTIVE** tiene ese `copy_id`).

Eso encaja perfecto.

8) MVP vs “bien hecho”

MVP:

- Cola por libro
- Asignar al devolver
- Posición calculada
- Expiración manual (endpoint admin “procesar expiradas”)

Siguiente paso:

- Job programado (cada X minutos) para expirar
- Notificaciones
- Mejoras de locking

Dime 1 cosa y te lo dejo ya “aterrizado” a tu código

¿Tu reserva actual es por:

- **bookId** (libro) o por **copyId** (ejemplar)?

Con eso te escribo exactamente:

- DTOs + repos JPA (queries)
- endpoints `POST /reservations` y `POST /admin/return/confirm`
- y los cambios concretos en tus HTML (botón “Entrar en cola” + posición).

